

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

IE2104 – Digital Electronics

School of Electrical & Electronic Engineering

Dr See Kye Yak
Associate Professor
Office: S2-B2C-112
Email: ekysee@ntu.edu.sg

IE2104 Digital Electronics

- **Text book:**
  - Wakerly John F, Digital Design: Principles and Practices, 4th Edition, Pearson Prentice Hall, 2006.
- **Reference books:**
  - Roth Charles H and Kinney Larry L, Fundamentals of Logic Design, 7th Edition, Cengage Learning, 2014.
  - Marcovitz Alan B, Introduction to logic design, 3rd Edition, McGraw-Hill, 2010.
  - Mano M Morris and Ciletti Michael D, Digital Design, 5th Edition, Pearson Prentice Hall, 2013.

Topics

Week	Name of topic
Week 1	Digital Fundamentals – Number System
Week 2	Digital Circuits – CMOS Circuits
Week 3	Digital Circuits – Electrical Behaviour
Week 4	Boolean Algebra
Week 5	Karnaugh Map
Week 6	Combinational Logic Modules – PLDs, Decoders and Encoders, Multiplexers
Week 7	Combinational Logic Modules – Comparators and Adders
Week 8	Flip Flops
Week 9	Flip Flops continue and State Machines
Week 10	State Machines Analysis
Week 11	State Machines Design
Week 12	Counter and Shift Registers
Week 13	Memory

Topic : Digital Fundamentals

- By the end of this topic, we will know:
  - Terminology used in Number Systems
  - Number representations in Binary, Octal and Hexadecimal
  - How to convert numbers among Decimal, Binary, Octal and Hexadecimal
  - Signed number representation with 2's Complement
  - How to convert between Decimal numbers and 2's Complement numbers
  - How to perform Binary Addition and Subtraction
  - Weighted codes, e.g. BCD
  - Non-weighted code, e.g. Gray Code, ASCII code

Digital Device and Design levels

- Digital systems or digital computers use digits or numbers to represent and process information: data, text, graphics, sound, video, etc.
- **Binary numbers** are used in digital systems.
- Binary numbers consist **two digits** (**0** and **1**).
- Each digit is called a BIT (**BI**nary dig**IT**).
- **BITs** are implemented with digital electronic circuits and have voltage levels: **LOW & HIGH.**
- These circuits are called **Logic Gates**, which operates on 0 & 1 (**False & True**).

Number Systems

- A number system specifies how values are represented.
- Human uses DECIMAL Number System.
- There are ten digits in Decimal Number System: 0,1,2,3,4,5,6,7,8,9.
- Digital Computers use Binary Numbers, which have only two digits: 0,1.
- There are other number systems, including:
  - **Octal**
  - **Hexadecimal**

Number System Examples

Binary	Decimal	Octal	3-Bit String	Hexadecimal	4-Bit String
0	0	0	000	0	0000
1	1	1	001	1	0001
10	2	2	010	2	0010
11	3	3	011	3	0011
100	4	4	100	4	0100
101	5	5	101	5	0101
110	6	6	110	6	0110
111	7	7	111	7	0111
1000	8	10	—	8	1000
1001	9	11	—	9	1001
1010	10	12	—	A	1010
1011	11	13	—	B	1011
1100	12	14	—	C	1100
1101	13	15	—	D	1101
1110	14	16	—	E	1110
1111	15	17	—	F	1111

Positional Number System

- Values are represented by the digits and their positions in the number.
- This type of number system is called **Positional Number System.**
- E.g., in Decimal Number System, 8973 is eight thousand nine hundred and seventy three:

$8 = 8000 = 8 \times 10^3$ (Thousands Place)
$9 = 900 = 9 \times 10^2$ (Hundreds Place)
$7 = 70 = 7 \times 10^1$ (Tens Place)
$3 = 3 = 3 \times 10^0$ (Units Place)

Number System Terminology

- In Number System, a value in decimal of an **n-digit number $a_{n-1}a_{n-2}\ldots a_1a_0$** is:

$$N = a_{n-1} \times r^{n-1} + a_{n-2} \times r^{n-2} + \ldots + a_1 \times r^1 + a_0 \times r^0$$

where $a_{n-1}, a_{n-2}, \ldots, a_1, a_0$ are coefficients.

- **r** is called the **Base** or **Radix**

Decimal is Base-10 system, r = 10
Binary is Base-2 system, r = 2
Octal is Base-8 system, r = 8
Hexadecimal is Base-16 system, r = 16

Binary Number System

- An **n-bit binary number** $a_{n-1}a_{n-2}\ldots a_1a_0$ has a value:

  $$N = a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \ldots + a_1 \times 2^1 + a_0 \times 2^0$$

- e.g. A **4-bit binary number** 1011_2 is:

  $N = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$

  $= 8 + 0 + 2 + 1 = 11_{10}$

- e.g. A **6-bit binary number** 110010_2 is:

  $N = 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

  $= 32 + 16 + 0 + 0 + 2 + 0$

  $= 50_{10}$

- The leftmost bit is called the ***high-order*** or ***most significant bit (MSB)***; the rightmost is the ***low-order*** or ***least significant bit (LSB)***.

Binary To Decimal Number Conversion

- Binary numbers can be converted to Decimal numbers by using:

$$N = a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \ldots + a_1 \times 2^1 + a_0 \times 2^0$$

- e.g. $10100_2 = 1\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 0\times2^0$

  $= 16 + 4 = \underline{20_{10}}$

  (16 8 4 2 1)

- e.g. $11001001_2 = 128 + 64 + 8 + 1 = \underline{201_{10}}$

  (128 64 32 16 8 4 2 1)

Decimal To Binary Number Conversion

- Decimal numbers can be converted to binary numbers by finding the coefficients of

$$N = a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \ldots + a_1 \times 2^1 + a_0 \times 2^0$$

- e.g. 17_{10} $= 16 + 1$

$= 1 \times 2^4 + 1 \times 2^0$

$= \underline{1\ 0\ 0\ 0\ 1}_2$

(16 8 4 2 1)

- A more systematical method is to divide the decimal number by 2 successively.

Decimal To Binary by Division of 2

- **For example:**
- **$168_{10} = 10101000_2$**
- **Steps:**
  - Divide the decimal by 2 until the quotient is 0
  - The remainders are the coefficient
  - The binary number is the reverse order of the remainders

Quotient	Remainder	Coefficient
168		
84	0	a_0
42	0	a_1
21	0	a_2
10	1	a_3
5	0	a_4
2	1	a_5
1	0	a_6
0	1	a_7

Octal Number System

- Octal Numbers are **base 8.**
- Octal has **8 digits: 0, 1, 2, 3, 4, 5, 6, 7.**
- An n-bit octal number $a_{n-1}a_{n-2}\ldots a_1a_0$ has a decimal value:

$$\mathbf{a_{n-1} \times 8^{n-1} + a_{n-2} \times 8^{n-2} + \ldots + a_1 \times 8^1 + a_0 \times 8^0}$$

- The conversion of octal to decimal can be done with the above equation.
- e.g. $\mathbf{263_8} = 2 \times 8^2 + 6 \times 8^1 + 3 \times 8^0$

$= 128 + 48 + 3 = \underline{\mathbf{179_{10}}}$

Hexadecimal Number System

- Hexadecimal Numbers are **base 16.**
- There are **16 digits**: **0 to 9, A, B, C, D, E, F.**
- An n-bit hexadecimal number $a_{n-1}a_{n-2}\ldots a_1a_0$ has a decimal value:

$$\mathbf{a_{n-1} \times 16^{n-1} + a_{n-2} \times 16^{n-2} + \ldots + a_1 \times 16^1 + a_0 \times 16^0}$$

- The conversion of hexadecimal to decimal can be done with the above equation.
- e.g. $\mathbf{B5E_{16}} = 11\times16^2 + 5\times16^1 + 14\times16^0$
  $= 2816 + 80 + 14 = \underline{\mathbf{2910_{10}}}$

Decimal To Octal Number Conversion

- Can be done by successive division of 8.
- e.g. $938_{10} = \underline{1652_8}$
- Steps:
  - Divide the decimal by 8 until the quotient is 0
  - The remainders are the coefficients
  - The octal number is the reverse order of the remainders

Quotient	Remainder	Coefficient
938		
117	2	a_0
14	5	a_1
1	6	a_2
0	1	a_3

Decimal To Hexadecimal Conversion

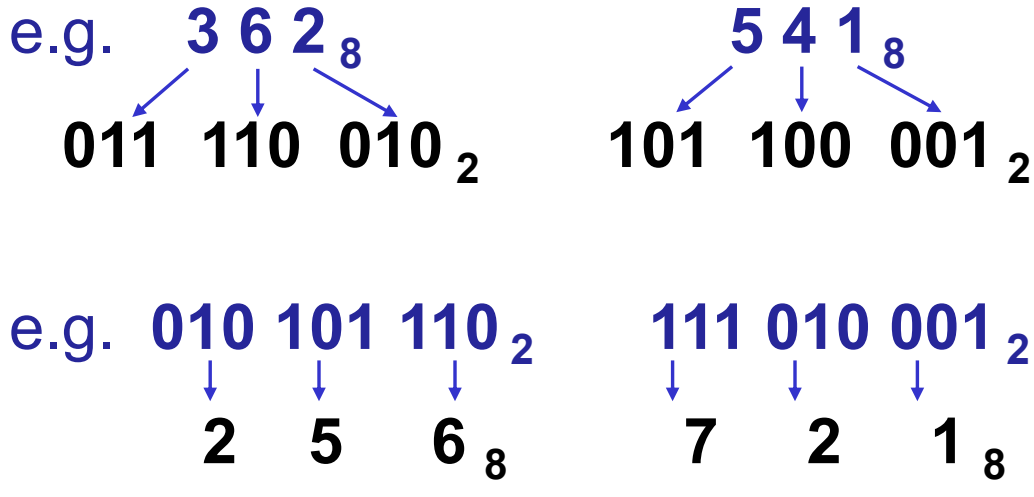
- Can be done by successive division of 16.
- e.g. $2793_{10} = AE9_{16}$
- Steps:
  - Divide the decimal by 16 until the quotient is 0
  - The remainders are the coefficients
  - The hexadecimal number is the reverse order of the remainders

Quotient	Remainder	Coefficient
2793		
174	9	a_0
10	E (14)	a_1
0	A (10)	a_2

Conversion between Binary and Octal

- Conversion between Binary and Octal is convenient.
- Each Octal digit equals to **3 bits.**

0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

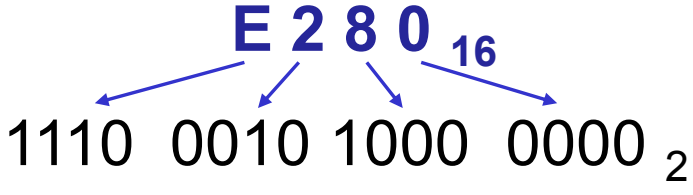


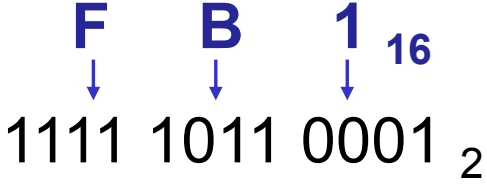
Conversion between Binary and Hex

- Conversion between Binary and Hexadecimal is also convenient.
- Each Hexadecimal digit equals to **4 bits.**

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111





Conversion Among Number Systems

- $267_{10} = 256 + 11$

  $= 1\times16^2 + 0\times16^1 + 11\times16^0$

  $= \mathbf{1\ 0\ B}_{16}$

  $= 0001\ 0000\ 1011_2$

  $= 000\ 100\ 001\ 011_2$

  $= \mathbf{0\ 4\ 1\ 3}_{8}$

  $= 4\times8^2 + 1\times8^1 + 3\times8^0$

  $= 256 + 8 + 3$

  $= \underline{267_{10}}$

Conversions Among Number Systems

- $731_8 = 111\ 011\ 001_2$

  $= 0001\ 1101\ 1001_2$

  $= \mathbf{1\ D\ 9}_{16}$

  $= 1 \times 16^2 + 13 \times 16^1 + 9 \times 16^0$

  $= 256 + 208 + 9$

  $= \underline{473_{10}}$

- $731_8 = 7 \times 8^2 + 3 \times 8^1 + 1 \times 8^0$

  $= 448 + 24 + 1$

  $= \underline{473_{10}}$

Representation of Signed Numbers

- In ordinary arithmetic, a plus / minus sign **“+” / “-”** is used to represent **positive / negative numbers**, e.g. +38 / -38.
- In digital electronic circuits, everything is represented with bits (0, 1).
- There are several ways to represent the signed binary numbers with just bits, e.g.:
  - Sign & Magnitude Representation
  - 1’s Complement Representation
  - 2’s Complement Representation

Sign-Magnitude Representation

- The most significant bit (MSB) is a sign bit.
- If the MSB is 0, the number is positive.
- If the MSB is 1, the number is negative.
- e.g. 01101 = +13
  11101 = -13
- e.g. 00000 = +0
  10000 = -0

not such a good method

- Disadvantages: (a) 2 patterns represent 0
  (b) Handle sign bit separately

Ones' Complement Representation

- Positive numbers and the corresponding negative numbers complement each other.
- Complement is inversion (Logic NOT):
  - The complement of 0 is 1.
  - The complement of 1 is 0.
- e.g. 01101 = +13
  16 8 4 2
  10010 = -13
- e.g. 00000 = +0
  11111 = -0
- Advantage: Its symmetry and the ease of complementation.
- Disadvantage: 2 patterns represent 0.

also not so good

Two's Complement Representation

- Positive numbers and Negative numbers are their corresponding 1's complement number + 1.
- e.g. 01101 = +13
  10010 **+ 1** = 10011 = -13
- e.g. 00000 = +0
  11111 **+ 1** = 00000 = +0

since its a 5 bit number anything more than 11111 is cut off

- Advantages: (a) only 1 pattern represents 0.
  (b) handle sign bit as other bits.

Two's Complement Number Conversion

- To obtain the negative value of a number in 2's complement system, we **invert ALL the bits and add 1 to it.**
- e.g. 5 = 00101

  -5 = 11010 + 1 = 11011
- e.g. -5 = 11011

  5 = 00100 + 1 = 00101

This process is called: **"take the 2's complement".**

Value of 2's Complement Number

- An **n-bit 2's complement number** $a_{n-1}a_{n-2}\ldots a_1a_0$ has a decimal value:

  $$N = (-1)a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \ldots + a_1 \times 2^1 + a_0 \times 2^0$$

- The equation is the same as the one for binary numbers except the MSB has a negative weight.
- E.g.: A 5-bit 2's complement number **11011** has the equivalent decimal value of:

  $N = (-1)1\times2^4 + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$

  $= -16 + 8 + 0 + 2 + 1 = \mathbf{-5}$

- Any number having its MSB = 1 is negative.
- Any number having its MSB = 0 is positive.

Two's Complement Number Conversion

- To find the 2's complement number for a negative decimal, we can find the binary of the positive decimal and then take its 2's complement.

e.g. -9_{10}

Add a "**0**" bit for MSB for +ve number

+9 = 01001

Invert bits: 10110

Plus 1: 10111 = $\underline{-9_{10}}$

Check: -16 + 0 + 4 + 2 + 1 = $\underline{-9}$

Non-Decimal Numbers Addition and Subtraction

- The arithmetic of non-decimal numbers is carried out in the same way as the decimal arithmetic.
- When adding 2 numbers, a carry is produced when the sum exceeds the base.
- For decimal, when a sum is 10 or more, a carry is produced.
- For binary, a sum of 2 will produce a carry.
- For Octal and Hexadecimal, a sum of 8 and 16 produces a carry, respectively.

Binary Addition

- Add the LSBs of the numbers with an initial carry (c_{in}) of 0, producing carry (c_{out}) and sum (S) bits.
- Continue processing the bits from right to left, adding the carry out of each column into the next column's sum.

E.g. 1

Carry in

		1	1	1	1				
X	190	1	0	1	1	1	1	1	0
Y	+ 141	+ 1	0	0	0	1	1	0	1
$X+Y$	331	1	0	1	0	0	1	0	1	1

E.g. 2

			1		1	1			
X	173	1	0	1	0	1	1	0	1
Y	+ 44	+ 0	0	1	0	1	1	0	0
$X+Y$	217	1	1	0	1	1	0	0	1

E.g. 3

Initial carry in

C		011111110
X	127	01111111
Y	+ 63	+ 00111111
$S = X + Y$	190	10111110

E.g. 4

C		000000000
X	170	10101010
Y	+ 85	+ 01010101
$S = X + Y$	255	11111111

Binary Subtraction

- Binary subtraction is performed by using addition in digital electronic circuits, so that it does not require another circuit.
- X – Y = X + (–Y)
- –Y can be obtained by using the two's complement of Y
- e.g. $4_{10} - 7_{10}$ is done with

  0100 – 0111 = 0100 + 1001 = 1101

  1101 is –3 in two's complement

Two's Complement Addition & Subtraction

- Two's complement numbers can be added in the same way as the ordinary addition.
- Any carry beyond the MSB is ignored.
- The result will always be the correct sum as long as the range of the number system is not exceeded.
- Examples of 4-bit two's complement additions are:

+3	0011	-2	1110
+ +4	+ 0100	+ -6	+ 1010
+7	0111	-8	~~1~~1000
			Discard carry
+6	0110	+4	0100
+ -3	+ 1101	+ -7	+ 1001
+3	~~1~~0011	-3	1101
Discard carry			

Octal & Hexadecimal Addition/Subtraction

$$\begin{array}{r} {\scriptstyle 1\ \ 1\ \ 1\ \ 0} \leftarrow \text{Carry in} \\ 1\ 4\ 7\ 6_8 \\ +\ \ 3\ 5\ 5\ 6_8 \\ \hline 5\ 2\ 5\ 4_8 \end{array} \qquad \begin{array}{r} {\scriptstyle 1\ \ 1\ \ 0\ \ 0} \leftarrow \text{Carry in} \\ 1\ 9\ B\ 9_{16} \\ +\ C\ 7\ E\ 6_{16} \\ \hline E\ 1\ 9\ F_{16} \end{array}$$

Ex. Subtract the following hexadecimal numbers using 2's complement method.

(a) $85_{16} - 3B_{16}$ (Ans. $4A_{16}$) (b) $E4_{16} - 0D_{16}$ (Ans. $D7_{16}$)

Solution

(a) $3B_{16} = 00111011$

2's complement of $3B_{16}$ = 11000101

= $C5_{16}$

$$\begin{array}{r} 8\ 5_{16} \\ +\ C\ 5_{16} \\ \hline 1\ 4\ A_{16} \end{array}$$

Answer

Discard carry

Value Range

- For **4-bit binary**, the range is:

Unsigned	0000 → 1111	0 → 15
1's Complement	1000 → 1111 0000 → 0111	-7 → -0 0 → 7
2's Complement	1000 → 1111 0000 → 0111	-8 → -1 0 → 7

Value Range

- For **n-bit binary,** the range is:

Unsigned	**$0 \rightarrow +2^n - 1$**
1's Complement	**$-(2^{n-1} - 1) \rightarrow +2^{n-1} - 1$**
2's Complement	**$-2^{n-1} \rightarrow +2^{n-1} - 1$**

- e.g. 5-bit binary

Unsigned	**0 → 31**
1's Complement	**-15 → +15**
2's Complement	**-16 → +15**

Overflow

- If an addition operation produces a result that exceeds the range of the number system, ***overflow*** is said to occur.
- Addition of two numbers with different signs can never produce overflow, but addition of two numbers of the same sign can.
- e.g.

-3	1101			+5	0101		
+ -6	+ 1010			+ +6	+ 0110		
-9	10111	= +7		+11	1011	= -5	
-8	1000			+7	0111		
+ -8	+ 1000			+ +7	+ 0111		
-16	10000	= +0		+14	1110	= -2	

Binary Codes for Decimal Numbers

- Binary numbers are used by computers, but human beings are familiar with decimal numbers.
- A set of n-bit strings in which different bit strings represent different numbers or other things is called a code.
- At least 4 bits are needed to represent ten decimal digits. There are billions of different ways to choose ten 4-bit code words and some of the common decimal codes are listed in the table on next slide.

Examples of Codes

Decimal digit	*BCD (8421)*	*2421*	*Excess-3*	*Biquinary*	*1-out-of-10*
0	0000	0000	0011	0100001	1000000000
1	0001	0001	0100	0100010	0100000000
2	0010	0010	0101	0100100	0010000000
3	0011	0011	0110	0101000	0001000000
4	0100	0100	0111	0110000	0000100000
5	0101	1011	1000	1000001	0000010000
6	0110	1100	1001	1000010	0000001000
7	0111	1101	1010	1000100	0000000100
8	1000	1110	1011	1001000	0000000010
9	1001	1111	1100	1010000	0000000001
		Unused code words			
	1010	0101	0000	0000000	0000000000
	1011	0110	0001	0000001	0000000011
	1100	0111	0010	0000010	0000000101
	1101	1000	1101	0000011	0000000110
	1110	1001	1110	0000101	0000000111
	1111	1010	1111	. . .	. . .

Note: BCD is covered in this course.

Binary-Coded Decimal (BCD)

- To facilitate the easy conversion between binary and decimal, ***binary-coded decimal (BCD)*** is used.
- In **BCD number system**, each decimal digit is represented by **4 bits.**
- BCD encodes the digits 0 through 9 by their 4-bit unsigned binary representations, 0000 to 1001. The codes 1010 to 1111 are not used, i.e., six 4-bit binary numbers not used in BCD.
- The weights for the BCD bits are 8, 4, 2 and 1, and for this reason it is sometimes called the ***8421 code***.

Decimal To BCD Conversion

0	0000
1	0001
2	0010
3	0011
4	0100

5	0101
6	0110
7	0111
8	1000
9	1001

Note: binary 1010 to 1111 are not used in BCD

e.g. 386_{10} is 0011 1000 0110 in BCD

e.g. 7589_{10} is 0111 0101 1000 1001 in BCD

Gray Code

- Binary numbers have more than 1 bits changing when increasing or decreasing continuously.
- e.g. 0011 to 0100 (3 to 4) ➡ 3 bits change
- In digital electronic circuits, sometimes it is desired to have only 1 bit changed.
- In this case, **Gray Code** can be used, where the adjacent numbers have only 1 bit different.
- Gray code is not weighted code.

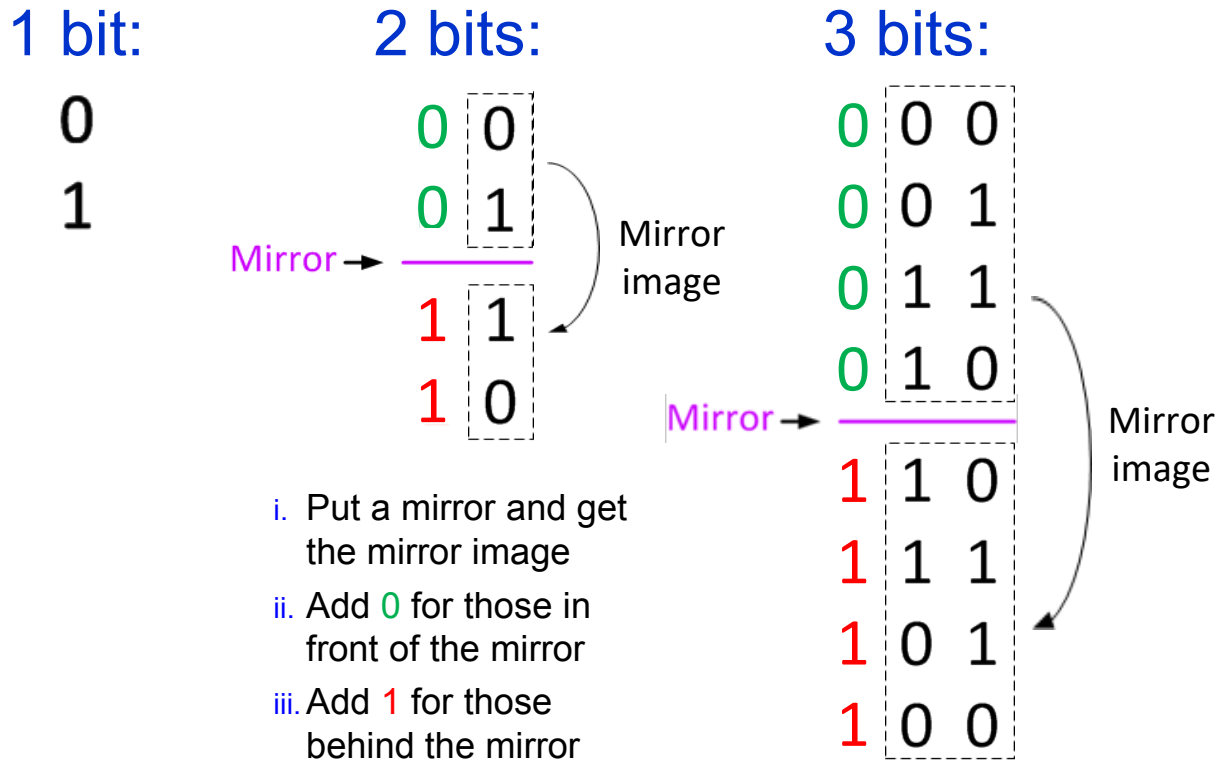
4-bit Gray Code

Gray Code	Decimal
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7

Gray Code	Decimal
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

Producing Gray Code

- Gray Code can be produced in this way:



ASCII Code

- Many applications of computers require processing of data which contains number, letters, and other symbols such as punctuation marks.
- To transmit such alphanumeric data to or from a computer or store it internally, each symbol must be represented by a binary code.
- One common alphanumeric code is the ASCII code (American Standard Code for Information Interchange).
- It is a 7-bit code, so 2^7 (128) different code combinations are available to represent letter, numbers, and other symbols. A portion of the ASCII code are shown in the following Table.

Table: ASCII code (incomplete)

Character	ASCII Code A_6	A_5	A_4	A_3	A_2	A_1	A_0	Character	ASCII Code A_6	A_5	A_4	A_3	A_2	A_1	A_0
space	0	1	0	0	0	0	0	@	1	0	0	0	0	0	0
!	0	1	0	0	0	0	1	A	1	0	0	0	0	0	1
"	0	1	0	0	0	1	0	B	1	0	0	0	0	1	0
#	0	1	0	0	0	1	1	C	1	0	0	0	0	1	1
$	0	1	0	0	1	0	0	D	1	0	0	0	1	0	0
%	0	1	0	0	1	0	1	E	1	0	0	0	1	0	1
&	0	1	0	0	1	1	0	F	1	0	0	0	1	1	0
'	0	1	0	0	1	1	1	G	1	0	0	0	1	1	1
(	0	1	0	1	0	0	0	H	1	0	0	1	0	0	0
)	0	1	0	1	0	0	1	I	1	0	0	1	0	0	1
*	0	1	0	1	0	1	0	J	1	0	0	1	0	1	0
+	0	1	0	1	0	1	1	K	1	0	0	1	0	1	1
,	0	1	0	1	1	0	0	L	1	0	0	1	1	0	0
−	0	1	0	1	1	0	1	M	1	0	0	1	1	0	1
.	0	1	0	1	1	1	0	N	1	0	0	1	1	1	0
/	0	1	0	1	1	1	1	O	1	0	0	1	1	1	1
0	0	1	1	0	0	0	0	P	1	0	1	0	0	0	0
1	0	1	1	0	0	0	1	Q	1	0	1	0	0	0	1
2	0	1	1	0	0	1	0	R	1	0	1	0	0	1	0
3	0	1	1	0	0	1	1	S	1	0	1	0	0	1	1
4	0	1	1	0	1	0	0	T	1	0	1	0	1	0	0

Work Example on ASCII Code

- Represent the word “IE2104” in ASCII code.
- **Solution:**

I	E	2	1	0	4
1001001	1000101	0110010	0110001	0110000	0110100

- What is the message represented by the ASCII codes below?

1010100 1001000 1000101 0100000 1000101 1001110 1000100

What do we know by now?

- By now, we should know:
  - Terminology used in Number Systems
  - Number representations in Binary, Octal and Hexadecimal
  - How to convert numbers among Decimal, Binary, Octal and Hexadecimal
  - How to perform Binary Addition and Subtraction
  - Signed number representation with 2's Complement
  - How to convert between Decimal numbers and 2's Complement numbers
  - What BCD, Gray Code and ASCII Code are

Topic: Digital Circuits

- By the end of this topic, we will know:
  - How to recognize logic signals
  - Know the operations of logic gates:
    - **NOT, AND, OR, NAND, NOR, XOR and XNOR**
  - Truth table
  - Different logic families
  - CMOS logic level
  - Switch model for CMOS logic gates
  - How to draw CMOS logic gates
  - AND-OR-INVERT and OR-AND-INVERT gates

Logic Signals and Logic Gates

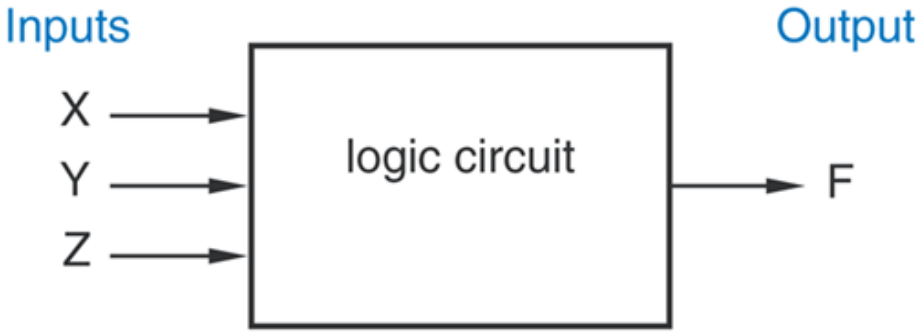
- Digital logic maps the infinite set of a physical quantity into two subsets corresponding to just two possible numbers or logic values — 0 and 1.
- Digital logic circuits can be analyzed and designed functionally, using switching algebra, tables, and other abstract means to describe the operation of well-behaved 0s and 1s in a circuit.
- A logic value, 0 or 1, is often called a binary digit, or a bit. If an application requires more than two discrete values, more bits may be used, with a set of n bits representing 2^n different values.

Examples of Physical States Representing Bits

Technology	State Representing Bit	
	0	1
Pneumatic logic	Fluid at low pressure	Fluid at high pressure
Relay logic	Circuit open	Circuit closed
Complementary metal-oxide semiconductor (CMOS) logic	0–1.5 V	3 .5–5.0 V
Transistor-transistor logic (TTL)	0–0.8 V	2 .0–5.0 V
Dynamic memory	Capacitor discharged	Capacitor charged
Nonvolatile, erasable memory	Electrons trapped	Electrons released
Microprocessor on-chip serial number	Fuse blown	Fuse intact
Polymer memory	Molecule in state A	Molecule in state B
Fiber optics	Light off	Light on
Magnetic disk or tape	Flux direction “north”	Flux direction “south”
Compact disc (CD)	No pit	Pit
Writeable compact disc (CD-R)	Dye in crystalline state	Dye in noncrystalline state

Logic Circuit

- A logic circuit can be represented as a “black box” with a certain number of inputs and outputs.
- e.g. A “black box” representation of a 3-input, 1-output logic circuit:



- The circuit’s “logical” operation can be described with a table that ignores electrical behavior and lists only the discrete 0 and 1 values.

Combinational Circuit & Truth Table

- A logic circuit whose outputs depend only on its current inputs is called a ***combinational circuit***.
- Its operation is fully described by a ***truth table*** that lists all combinations of input values and the corresponding output value(s) produced.

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Example of Truth Table of a 3-input 1-output Combinational Circuit

Truth Table

- A Truth Table is a table that lists **ALL** the combinations of the variables (inputs) and the result (output) of the logic operation.
- The number of rows in a truth table is 2^n, where **n** is the number of variables.

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

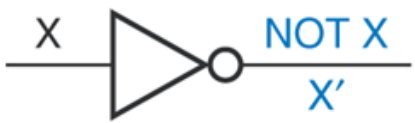
- The binary combinations are obtained by counting from **0 to 2^n - 1** in binary.

Binary Logic

- Binary Logic consists of **binary variables** and **logic operations**.
- **Binary variables** are designated by alphabets such as **a, b, c, x, y, z,** etc.
- Binary variables have **only 2 values: 0, 1.**
- Operations performed on the binary variables are called **Logic Operations.**
- Just three basic logic operations/functions, **AND**, **OR**, and **NOT**, can be used to build any combinational digital logic circuit.

Logic Operation NOT

- This operation is represented by an overbar ($\overline{X}$) or a prime (X'). e.g. $F = \overline{X}$ or F = X'.
- It produces the opposite value. i.e.:
  - If X is 0 then $\overline{X}$ is 1.
  - If X is 1 then $\overline{X}$ is 0.
- Also called ***complement*** or ***inversion*** operation.



Logic Symbol

X	NOT X
0	1
1	0

Truth Table

Logic Operation AND

- The AND operation is represented by a **dot** (•).
- e.g. F = X • Y
- Usually the dot is omitted: F = XY
- The operation result is: if **ALL** the variables are 1, then the result is 1. Otherwise, the result is 0.



Logic Symbol

X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

Truth Table

Logic Operation OR

- The OR operation is represented by a **plus sign.**
- e.g. F = X + Y
- The operation result is: if **ANY** of the variables is 1, then the result is 1. Otherwise, the result is 0.



Logic Symbol

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

Truth Table

Inverting Gates – NAND, NOR

- The small circle at the output is called an ***inversion bubble*** and is used in logic symbols to denote "inverting" behavior.



X	Y	X NAND Y
0	0	1
0	1	1
1	0	1
1	1	0



X	Y	X NOR Y
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive OR and Exclusive NOR Gates

- **OR:**

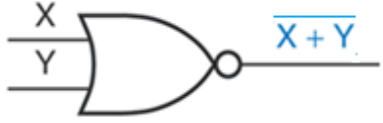
$F = X + Y$



a	b	F
0	0	0
0	1	1
1	0	1
1	1	1

- **NOR:**

$F = \overline{X + Y}$



a	b	y
0	0	1
0	1	0
1	0	0
1	1	0

- **XOR: Exclusive-OR**

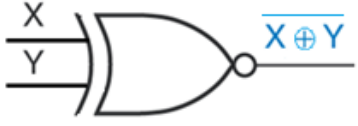
$F = X \oplus Y$



a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

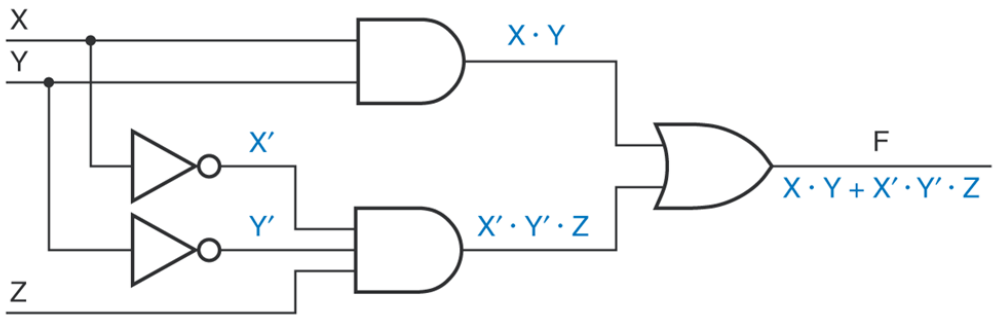
- **XNOR: Exclusive-NOR**

$F = \overline{X \oplus Y}$



a	b	y
0	0	1
0	1	0
1	0	0
1	1	1

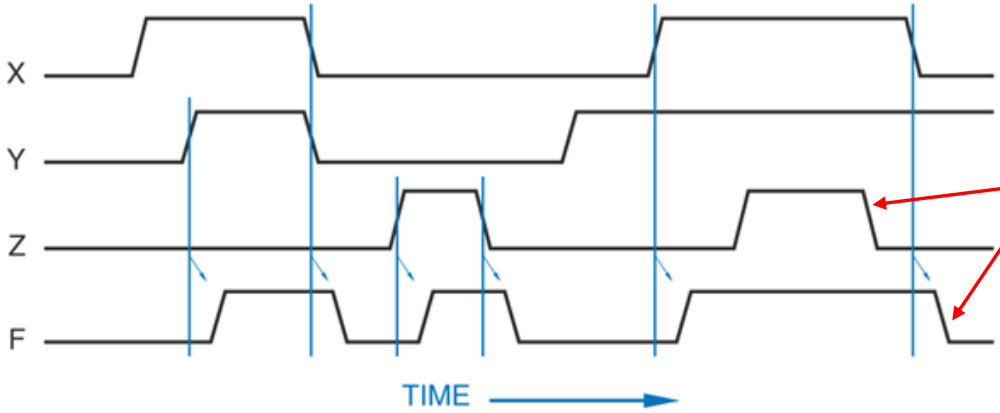
Logic Circuit & Timing Diagram



(a) Gate-level logic diagram

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

(b) Truth table



Slanted line to show the change is not instant.

(c) Timing diagram of the above circuit

Pin Diagram Examples of ICs

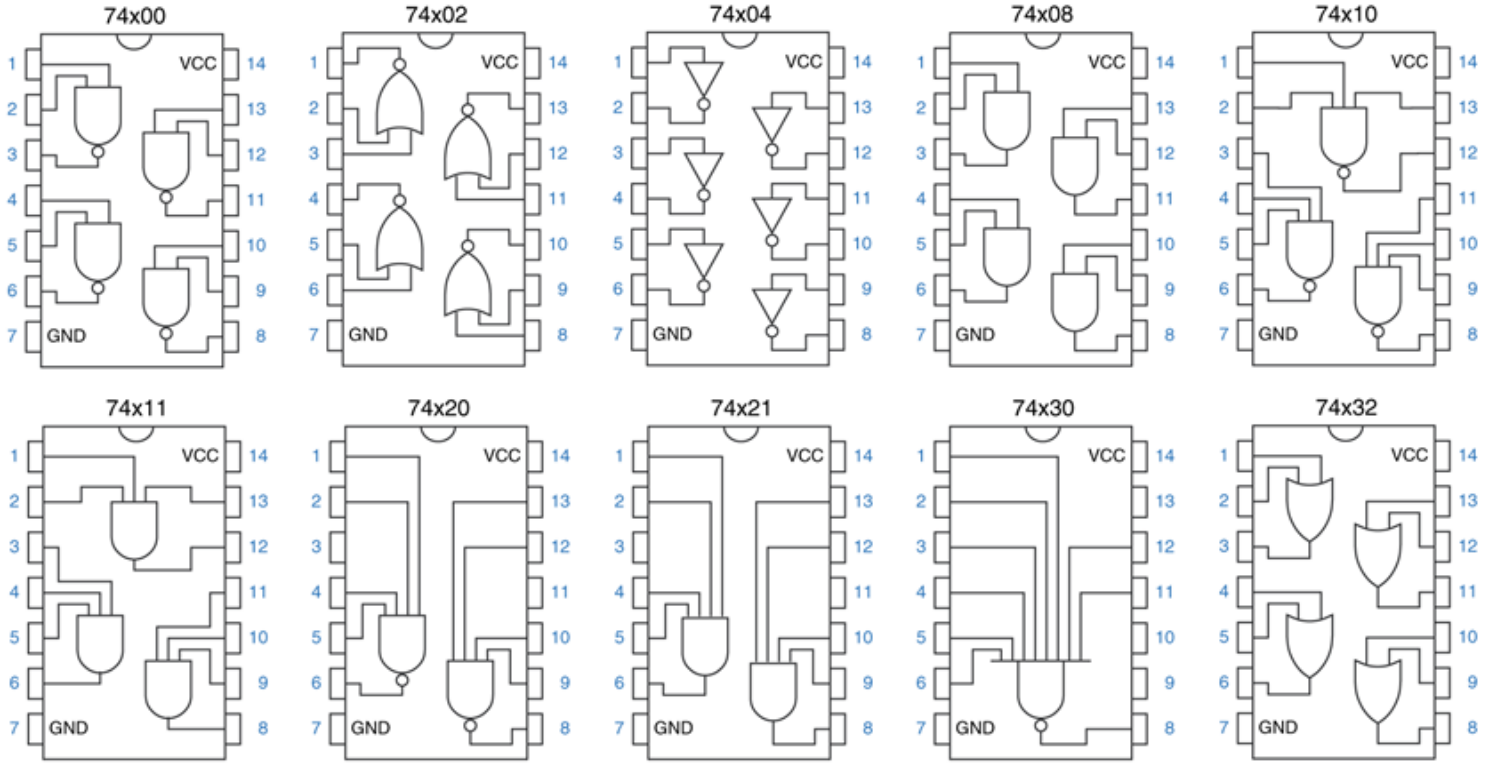
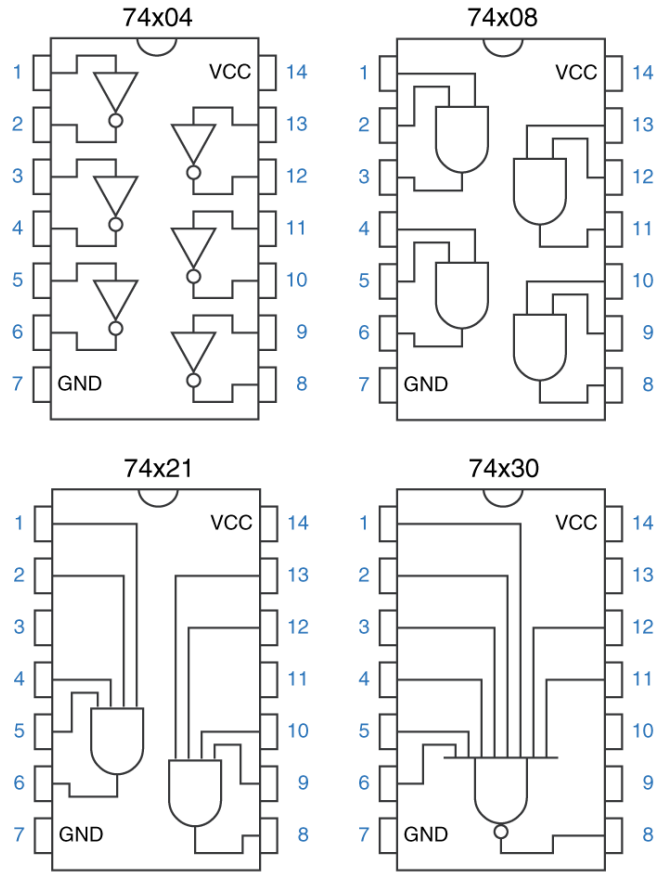


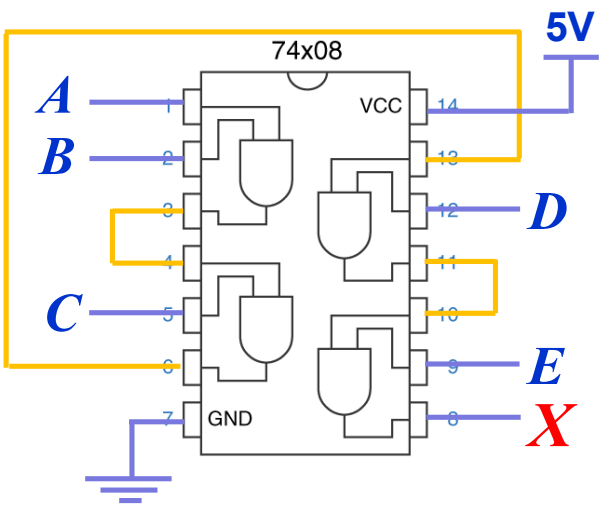
Figure 1-5 Pin diagrams for a few 7400-series SSI ICs.

Work Example on Logic Gate

Connect a circuit to realise the function, $X = ABCDE$. You may use any of the ICs on the right.



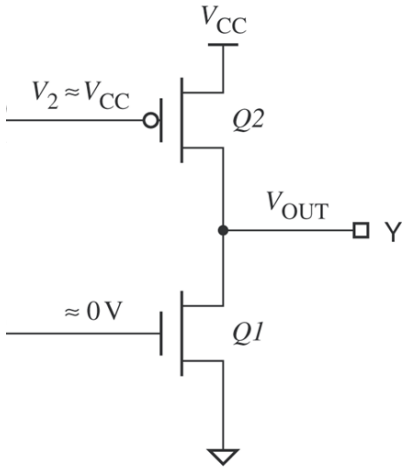
One Possible solution:



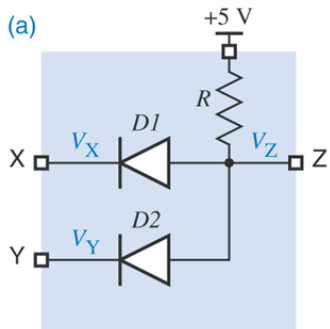
Logic Families

- The invention of *semiconductor diode* and the *bipolar junction transistor* allowed the development of smaller, faster, and more capable computers in the late 1950s.
- The invention of *integrated circuit (IC)* in 1960s allowed multiple diodes, transistors, and other components to be fabricated on a single chip, and hence better computer.
- A *logic family* is a collection of different IC chips that have similar characteristics for their inputs, outputs, and internal circuits but perform different logic functions, e.g., *transistor-transistor logic (TTL)*, *diode logic*, *CMOS logic, etc*.

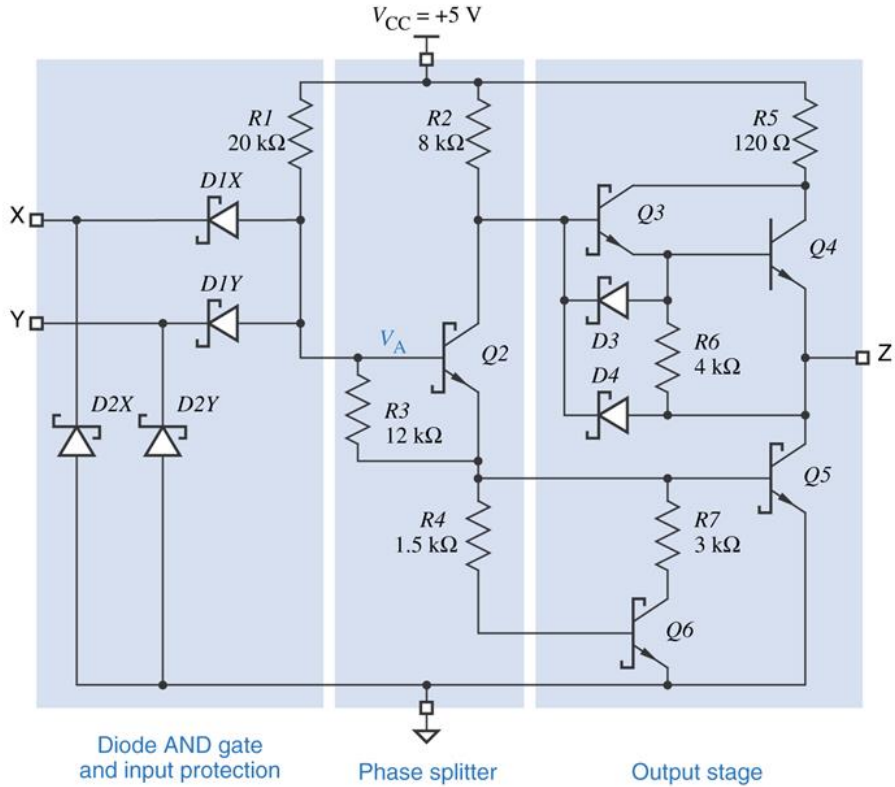
Examples of Logic Families



CMOS logic gate (Inverter)



Diode logic (AND gate)



TTL logic (NAND gate)

CMOS Logic

- Almost all new large-scale integrated circuits, such as microprocessors and memories, use **CMOS**.
- **CMOS** stands for **C**omplementary **M**etal-**O**xide **S**emiconductor **F**ield-**E**ffect **T**ransistor (**MOSFET**).
- CMOS offers higher speed and lower power consumption.
- CMOS logic is both the most capable and easy to understand commercial logic technology.

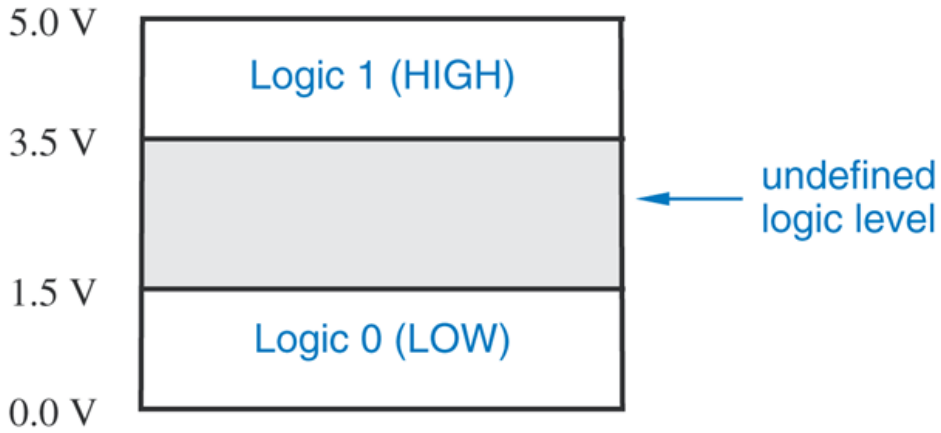
CMOS Logic Level

- Abstract logic element process binary digits, 0 and 1, but real logic circuits process electrical signals such as voltage levels.
- In any logic circuit, there is a range of voltages (or other circuit conditions) interpreted as a logic 0, and another non-overlapping range that is interpreted as a logic 1.

CMOS Logic Level (contd.)

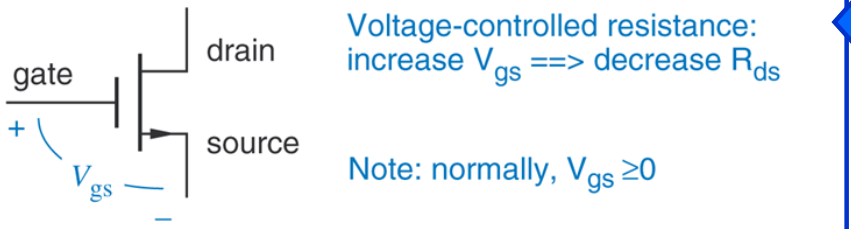
- A typical CMOS logic circuit operates from a 5-volt power supply. Any voltage in the range 0–1.5 V may be interpreted as logic 0, and in the range 3.5–5.0 V as a logic 1.
- Voltage in the intermediate range (1.5–3.5 V) are not expected to occur except during signal transitions, and yield undefined logic values (Could be either 0 or 1).

Logic levels for typical CMOS logic circuits

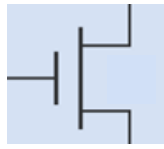


MOS Transistors

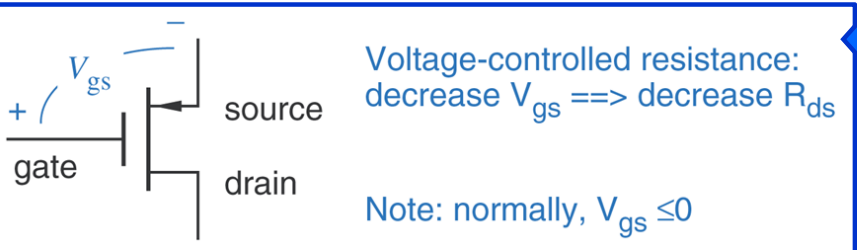
- A MOS transistor can be modeled as a 3-terminal device that acts like a voltage-controlled resistance.



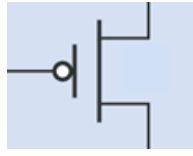
Circuit symbol for an *n*-channel MOS (NMOS) transistor



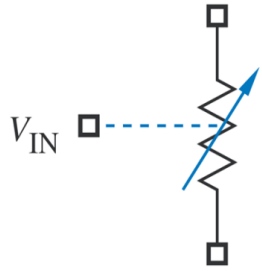
Alternative symbol for NMOS



Circuit symbol for an *p*-channel MOS (PMOS) transistor



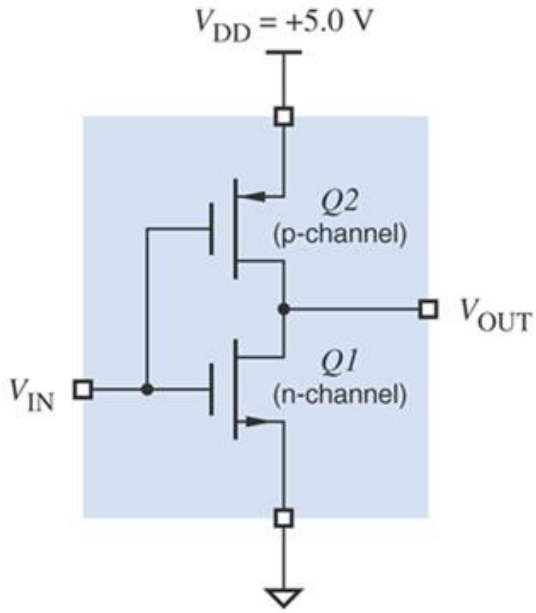
Alternative symbol for PMOS



The MOS transistor as a voltage-controlled resistance.

Basic CMOS Inverter Circuit

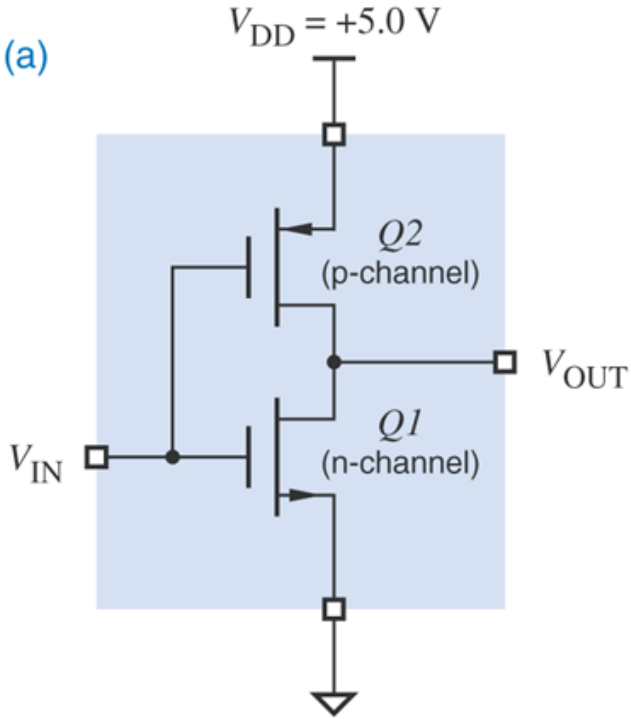
- NMOS and PMOS transistors are put together in a complementary manner to form ***CMOS logic circuits***.
- The simplest CMOS circuit, a logic inverter, requires only one of each type of transistor.
- The power-supply voltage, V_{DD}, typically in the range of 1 – 6 V and is often set at 5 V for compatibility with the older TTL family.



CMOS Inverter

CMOS Inverter

(a)



(b)

V_{IN}	Q1	Q2	V_{OUT}
0.0 (L)	off	on	5.0 (H)
5.0 (H)	on	off	0.0 (L)

(c)

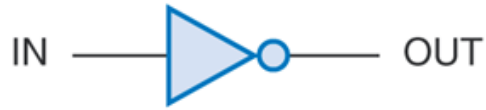


Figure 3-10

CMOS inverter: (a) circuit diagram; (b) functional behavior; (c) logic symbol.

Switch Model for CMOS Inverter

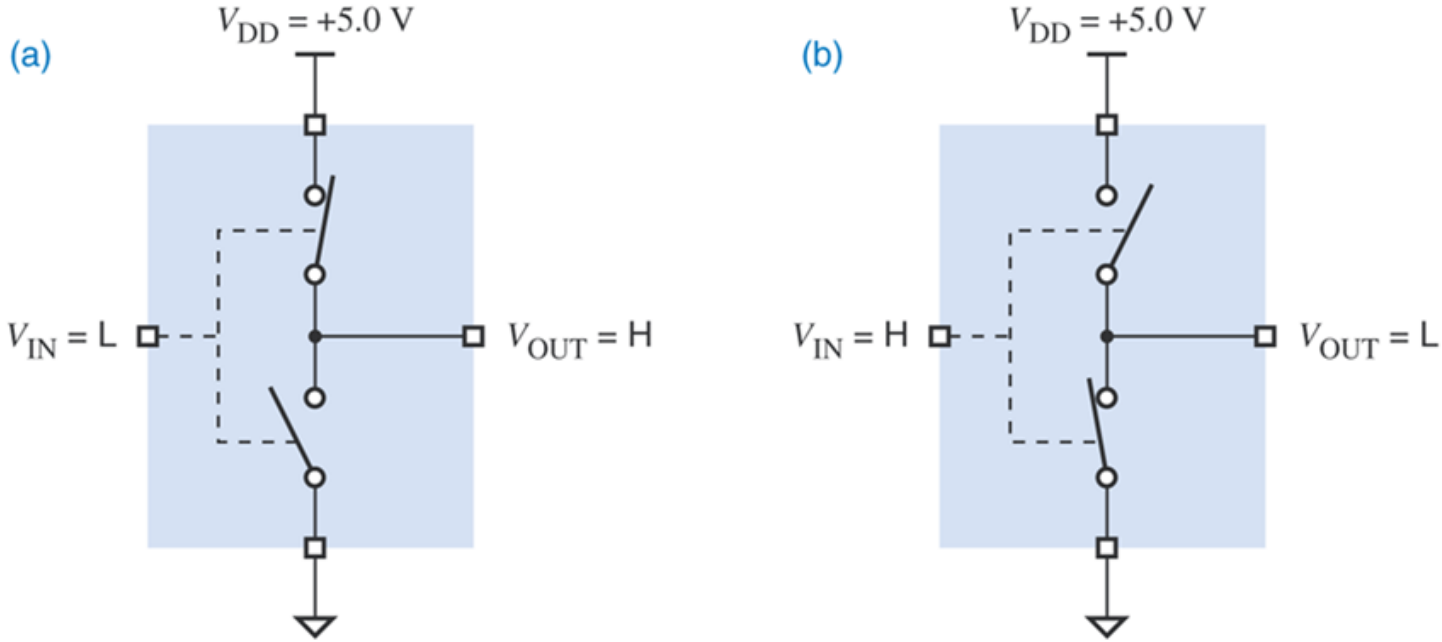
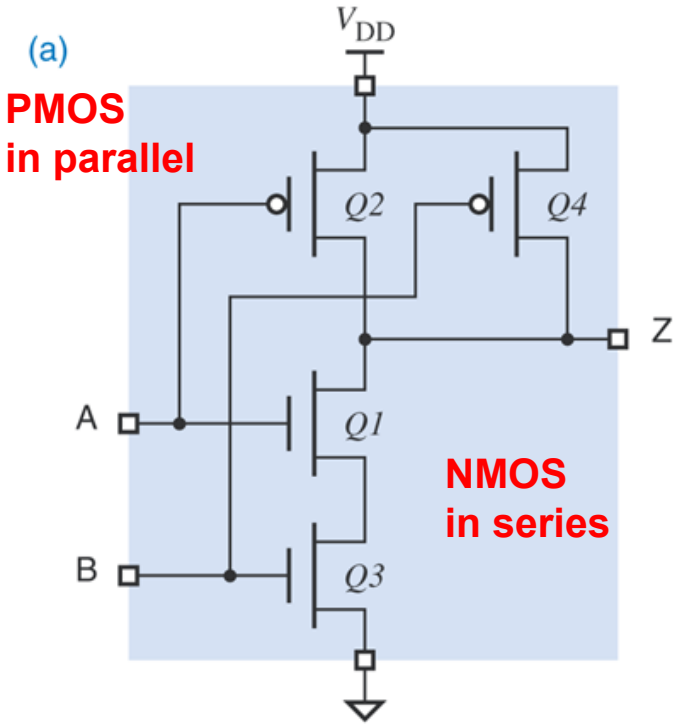


Figure 3-11

Switch model for CMOS inverter: (a) LOW input; (b) HIGH input.

CMOS 2-input NAND Gate



(b)

A	B	Q1	Q2	Q3	Q4	Z
L	L	off	on	off	on	H
L	H	off	on	on	off	H
H	L	on	off	off	on	H
H	H	on	off	on	off	L

(c)



Figure 3-13

CMOS 2-input NAND gate: (a) circuit diagram; (b) function table; (c) logic symbol.

Switch Model for CMOS 2-input NAND Gate

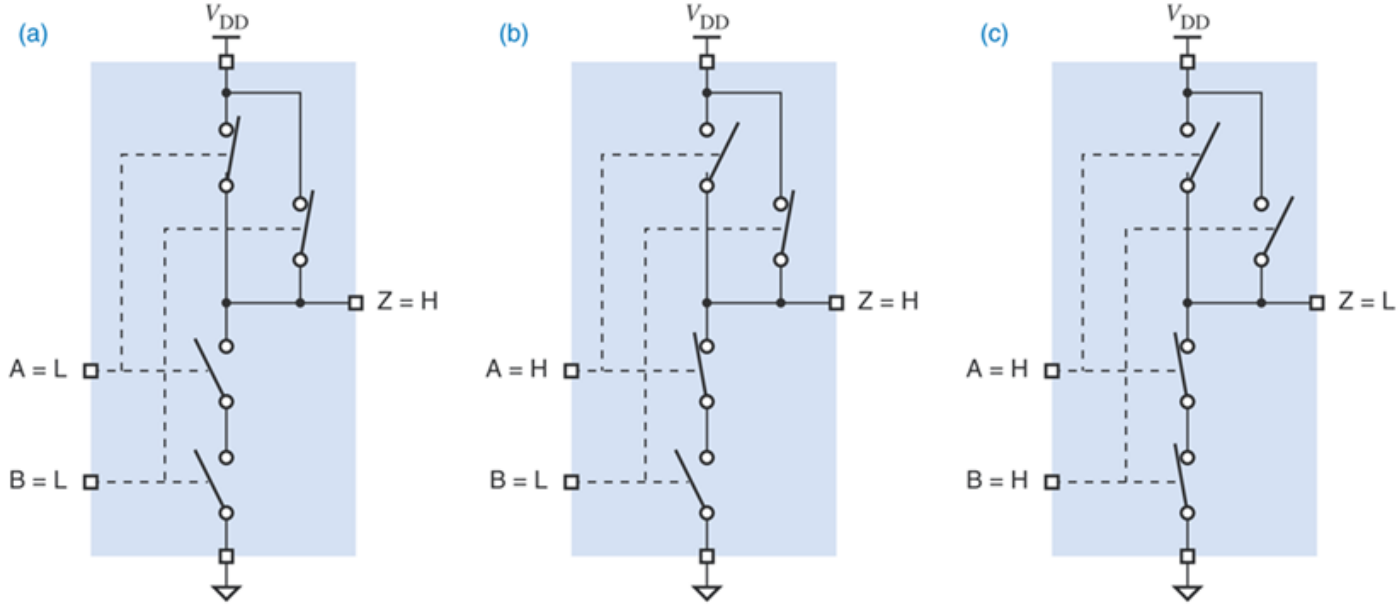
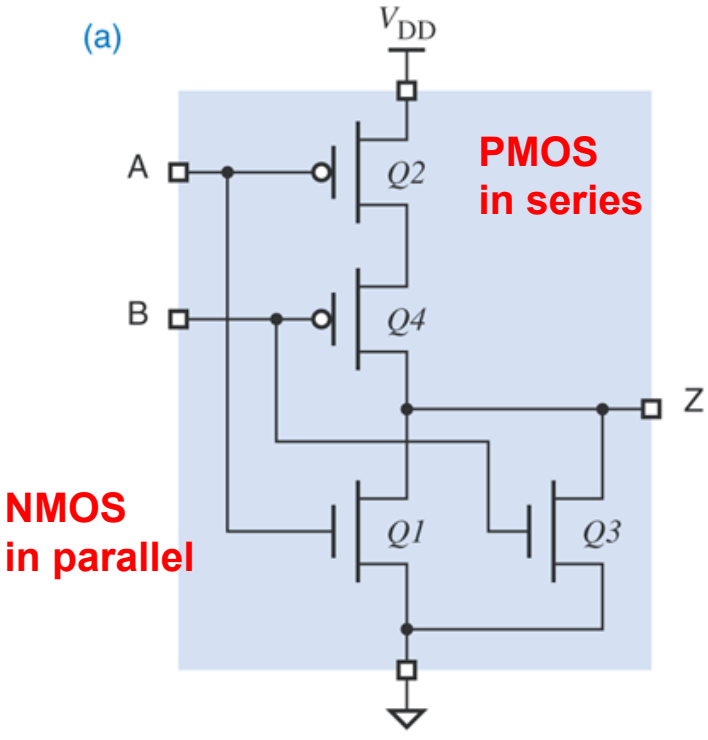


Figure 3-14 Switch model for CMOS 2-input NAND gate: (a) both inputs LOW; (b) one input HIGH; (c) both inputs HIGH.

A	B	*Q1*	*Q2*	*Q3*	*Q4*	Z
L	L	off	on	off	on	H
L	H	off	on	on	off	H
H	L	on	off	off	on	H
H	H	on	off	on	off	L

Output is either connected to power supply or ground but NOT both.
Output is H if connected to power supply.
Output is L if connected to ground.

CMOS 2-input NOR Gate



(b)

A	B	Q1	Q2	Q3	Q4	Z
L	L	off	on	off	on	H
L	H	off	on	on	off	L
H	L	on	off	off	on	L
H	H	on	off	on	off	L

(c)

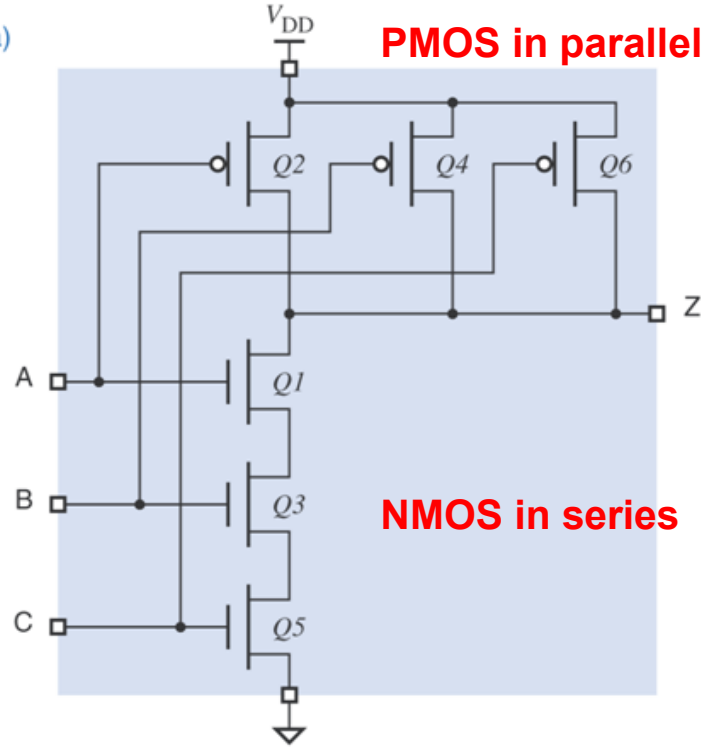


Figure 3-15

CMOS 2-input NOR gate: (a) circuit diagram; (b) function table; (c) logic symbol

CMOS 3-input NAND Gate

(a)



(b)

A	B	C	Q1	Q2	Q3	Q4	Q5	Q6	Z
L	L	L	off	on	off	on	off	on	H
L	L	H	off	on	off	on	on	off	H
L	H	L	off	on	on	off	off	on	H
L	H	H	off	on	on	off	on	off	H
H	L	L	on	off	off	on	off	on	H
H	L	H	on	off	off	on	on	off	H
H	H	L	on	off	on	off	off	on	H
H	H	H	on	off	on	off	on	off	L

(c)

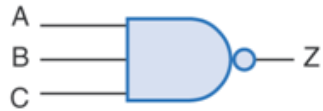


Figure 3-16

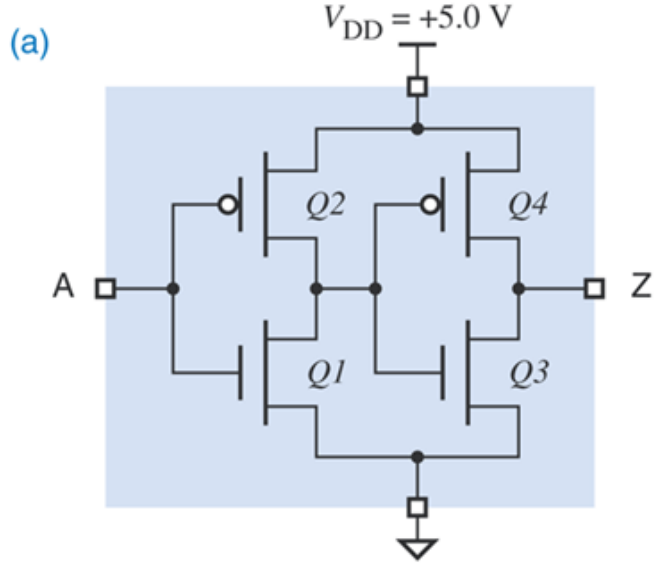
CMOS 3-input NAND gate: (a) circuit diagram; (b) function table; (c) logic symbol.

Fan-In

- The number of inputs that a gate can have in a particular logic family is called the logic family's ***fan-in***.
- An n-input gate has n series and n parallel transistors.
- Increasing the number of inputs will increase the "on" resistance of the series transistors.
- The fan-in limits of CMOS gates are typically 4 for NOR gates and 6 for NAND gates.

Non-inverting Gates

- CMOS non-inverting buffers, AND and OR gates are obtained by connecting an inverter to the output of the corresponding inverting gate.



(b)

A	Q1	Q2	Q3	Q4	Z
L	off	on	on	off	L
H	on	off	off	on	H

(c)

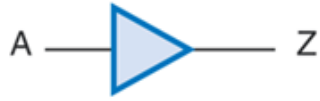
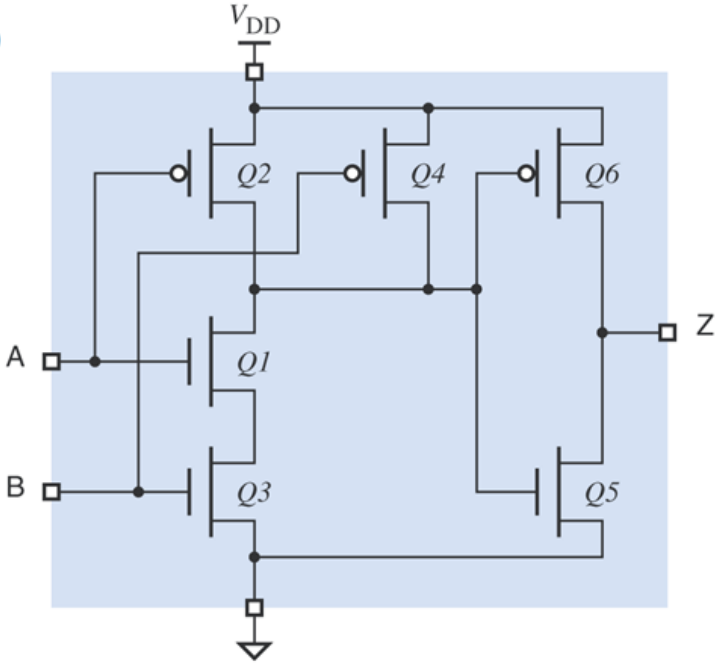


Figure 3-18

CMOS noninverting buffer: (a) circuit diagram; (b) function table; (c) logic symbol.

Non-inverting Gates: 2-input AND Gate

(a)



(b)

A	B	Q1	Q2	Q3	Q4	Q5	Q6	Z
L	L	off	on	off	on	on	off	L
L	H	off	on	on	off	on	off	L
H	L	on	off	off	on	on	off	L
H	H	on	off	on	off	off	on	H

(c)



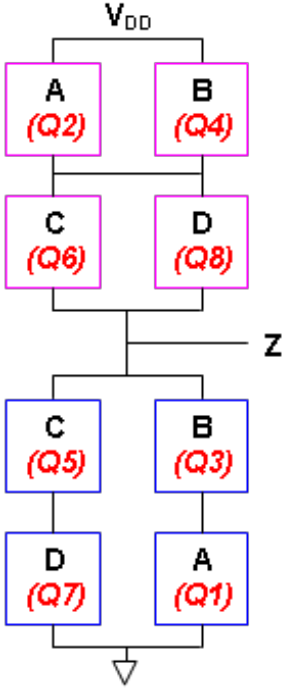
Figure 3-19

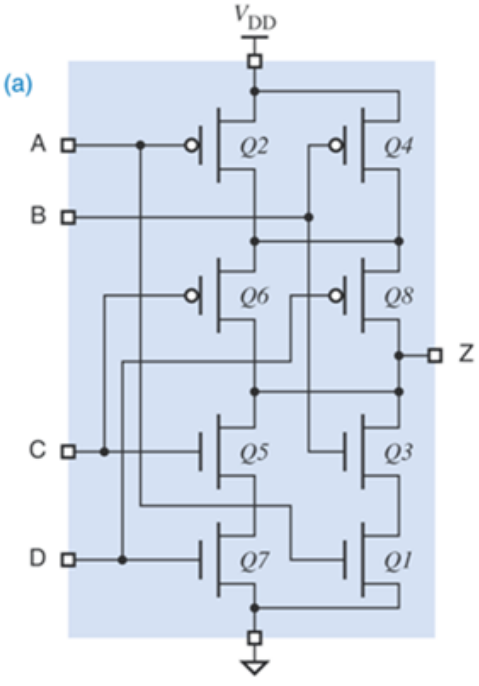
CMOS 2-input AND gate: (a) circuit diagram; (b) function table; (c) logic symbol.

CMOS AOI and OAI

- CMOS circuits can perform two levels of logic with just a single “level” of transistors, e.g., 2-input CMOS AND-OR-INVERT (AOI) gate and OR-AND-INVERT (OAI) gate.
- These gates are very popular as they can perform two levels of logic (AND-OR or OR-AND) with just one level of delay.
- The speed and other electrical characteristics of a single CMOS AOI or OAI gate are quite comparable to those of a single CMOS NAND or NOR gate.

CMOS AND-OR-INVERT Gate





(b)

A	B	C	D	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Z
L	L	L	L	off	on	off	on	off	on	off	on	H
L	L	L	H	off	on	off	on	off	on	on	off	H
L	L	H	L	off	on	off	on	on	off	off	on	H
L	L	H	H	off	on	off	on	on	off	on	off	L
L	H	L	L	off	on	on	off	off	on	off	on	H
L	H	L	H	off	on	on	off	off	on	on	off	H
L	H	H	L	off	on	on	off	on	off	off	on	H
L	H	H	H	off	on	on	off	on	off	on	off	L
H	L	L	L	on	off	off	on	off	on	off	on	H
H	L	L	H	on	off	off	on	off	on	on	off	H
H	L	H	L	on	off	off	on	on	off	off	on	H
H	L	H	H	on	off	off	on	on	off	on	off	L
H	H	L	L	on	off	on	off	off	on	off	on	L
H	H	L	H	on	off	on	off	off	on	on	off	L
H	H	H	L	on	off	on	off	on	off	off	on	L
H	H	H	H	on	off	on	off	on	off	on	off	L

CMOS AND-OR-INVERT gate: (a) circuit diagram; (b) function table.

- **Note:**
  - NMOS – (Q1, Q3) in series, (Q5, Q7) in series, then the 2 branches in parallel.
  - PMOS – (Q2, Q4) in parallel, (Q6, Q8) in parallel, then the 2 tranches in series.

CMOS AND-OR-INVERT Gate (Contd.)

- $Z = \overline{A \cdot B + C \cdot D}$

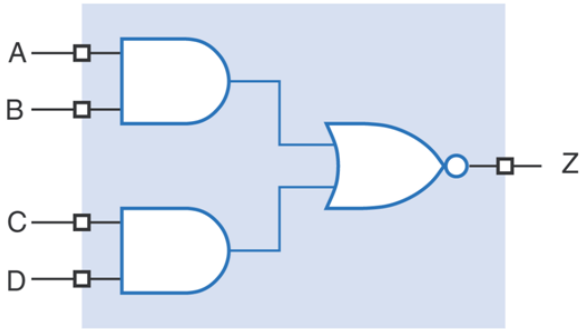
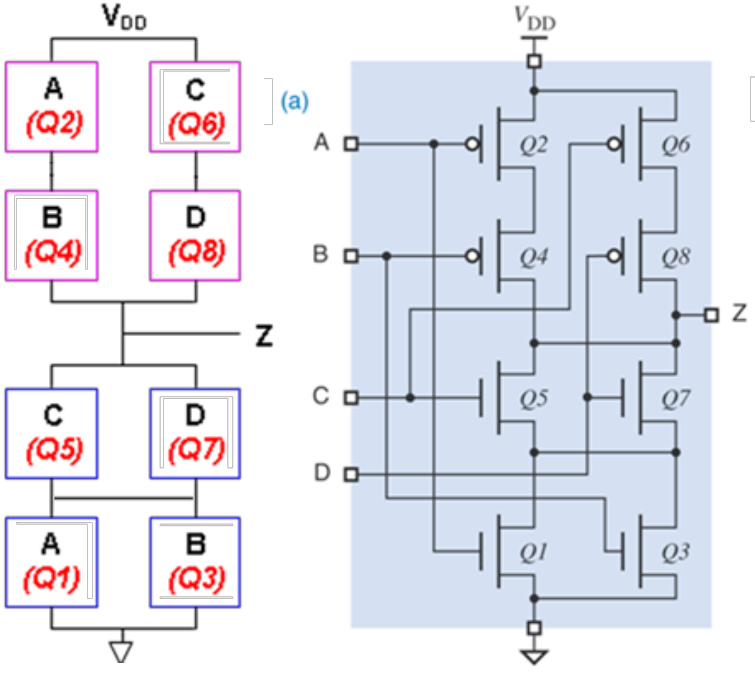


Figure 3-21
Logic diagram for CMOS AND-OR-INVERT gate.

- Transistors can be added to or removed from this circuit to obtain an AOI function with different number of ANDs or a different number of inputs per AND.

CMOS OR-AND-INVERT Gate



(b)

A	B	C	D	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Z
L	L	L	L	off	on	off	on	off	on	off	on	H
L	L	L	H	off	on	off	on	off	on	on	off	H
L	L	H	L	off	on	off	on	on	off	off	on	H
L	L	H	H	off	on	off	on	on	off	on	off	H
L	H	L	L	off	on	on	off	off	on	off	on	H
L	H	L	H	off	on	on	off	off	on	on	off	L
L	H	H	L	off	on	on	off	on	off	off	on	L
L	H	H	H	off	on	on	off	on	off	on	off	L
H	L	L	L	on	off	off	on	off	on	off	on	H
H	L	L	H	on	off	off	on	off	on	on	off	L
H	L	H	L	on	off	off	on	on	off	off	on	L
H	L	H	H	on	off	off	on	on	off	on	off	L
H	H	L	L	on	off	on	off	off	on	off	on	H
H	H	L	H	on	off	on	off	off	on	on	off	L
H	H	H	L	on	off	on	off	on	off	off	on	L
H	H	H	H	on	off	on	off	on	off	on	off	L

CMOS OR-AND-INVERT gate: (a) circuit diagram; (b) function table.

- **Note:**
  - NMOS – (Q1, Q3) in parallel, (Q5, Q7) in parallel, then the 2 branches in series.
  - PMOS – (Q2, Q4) in series, (Q6, Q8) in series, then the 2 tranches in parallel.

CMOS OR-AND-INVERT Gate (Contd.)

- $Z = \overline{(A + B) \cdot (C + D)}$

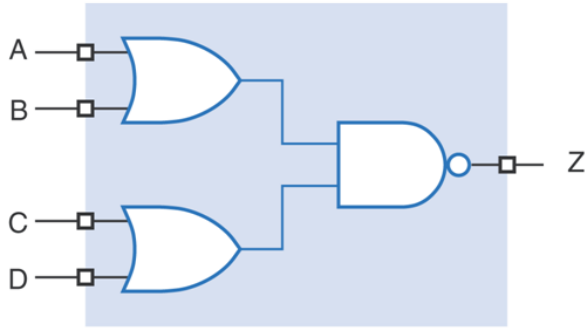
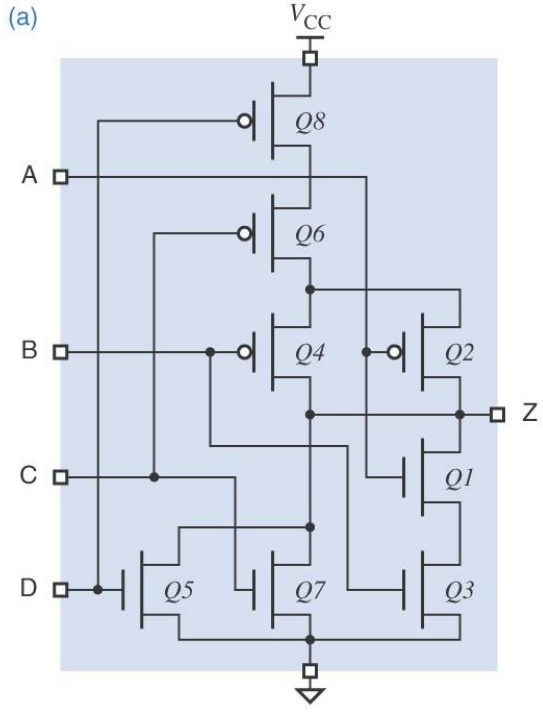


Figure 3-23
Logic diagram for CMOS OR-AND-INVERT gate.

- **Note:** Without considering the long bar (CMOS output always has its output inverted):
  - **NMOS**: Series connection → AND
    Parallel connection → OR
  - **PMOS**: Series connection → OR
    Parallel connection → AND

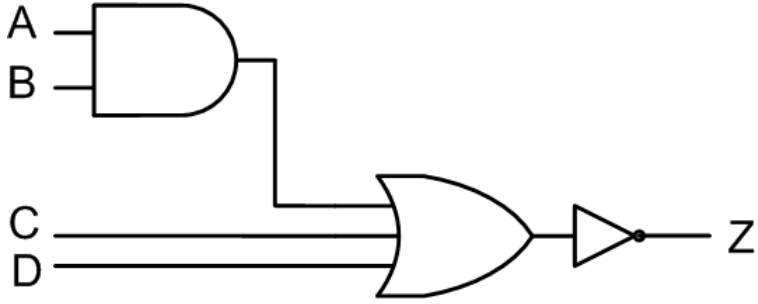
Work Example – CMOS AND-OR-INVERT

- Determine the logic function for the circuit shown below, and draw the corresponding logic diagram using AND and OR gates and inverters.



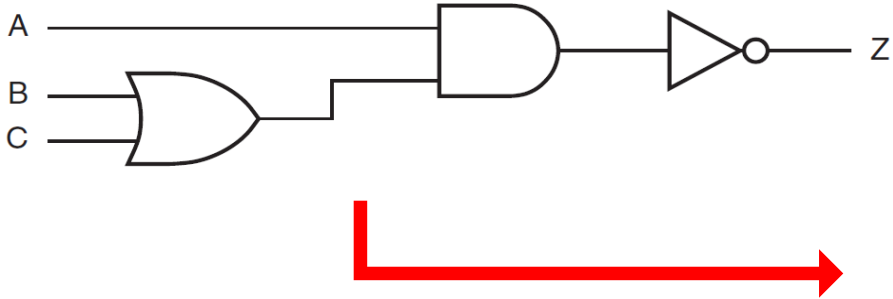
Solution:

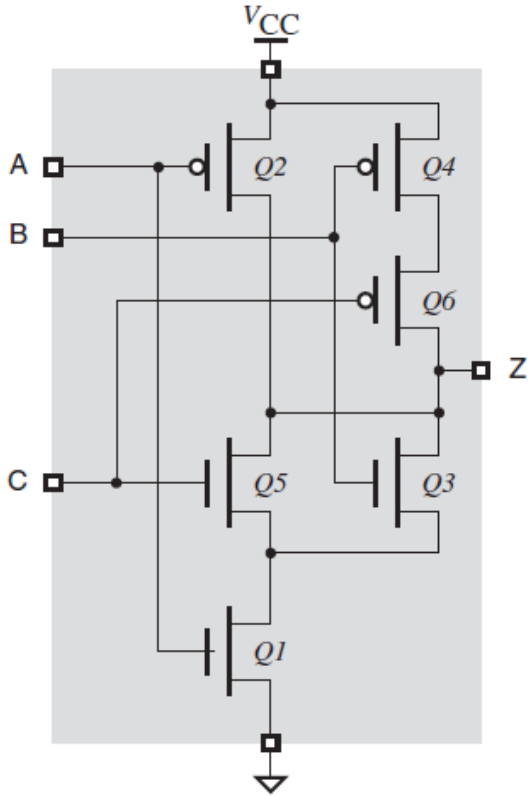
$$Z = \overline{AB + C + D}$$



Work Example on AOI & OAI

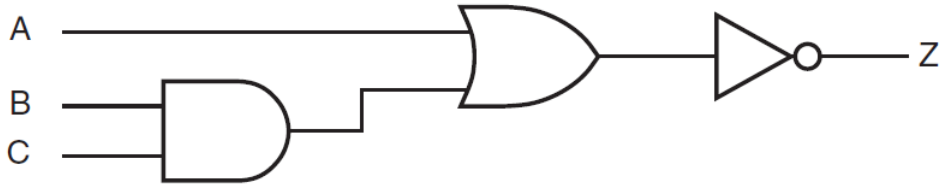
- Draw the transistor level circuit diagram for the OR-AND-NOT logic circuit shown below.





Homework

- Draw the transistor level circuit diagram for the AND-OR-NOT logic circuit shown below.



- **Note (CMOS features):**
  - Each input is connected a pair of one PMOS and one NMOS transistor.
  - If the NMOS are connected in parallel, then the corresponding PMOS are connected in series, and vice versa.
  - The output is always inverted.
  - If the output is not to be inverted, add an inverter at the ouput.

Week 2 stop

What do we know by now?

- By now, we should know:
  - Logic signals
  - Operations of logic gates — NOT, AND, OR, NAND, NOR, XOR and XNOR
  - Able to differentiate the logic families
  - Understand the switch model
  - Able to recognize the logic functions of CMOS logic circuits
  - Able to draw CMOS logic circuits for given logic functions

Topic: Digital Circuits (Continued)

- By the end of this topic, we will know:
  - Understand electrical behavior of CMOS circuits
  - CMOS steady-state electrical behavior
  - CMOS dynamic electrical behavior
  - CMOS logic families
  - Low-voltage CMOS logic

Electrical Behavior of CMOS Circuits

- It is important to understand the electrical aspect of CMOS circuit operation.
- A circuit designer must provide adequate engineering design margins — to ensure circuit works properly even under the worst conditions.
- When analyzing or designing a digital circuit, both static and dynamic behaviors must be considered.
  - ***Static behaviors*** – situations where a circuit's input and output signals are not changing.
  - ***Dynamic behaviors*** – situations where a circuit's input and output signals are changing.

Topics with static and dynamic aspects

1. Logic voltage levels
2. DC noise margins
3. Fanout
4. Speed
5. Power consumption
6. Open-drain outputs

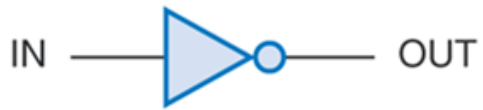
CMOS Static Electrical Behavior

- DC or static behavior of CMOS circuit is also called the steady-state behavior, because the electrical state of the inputs is not changing.

Logic Levels

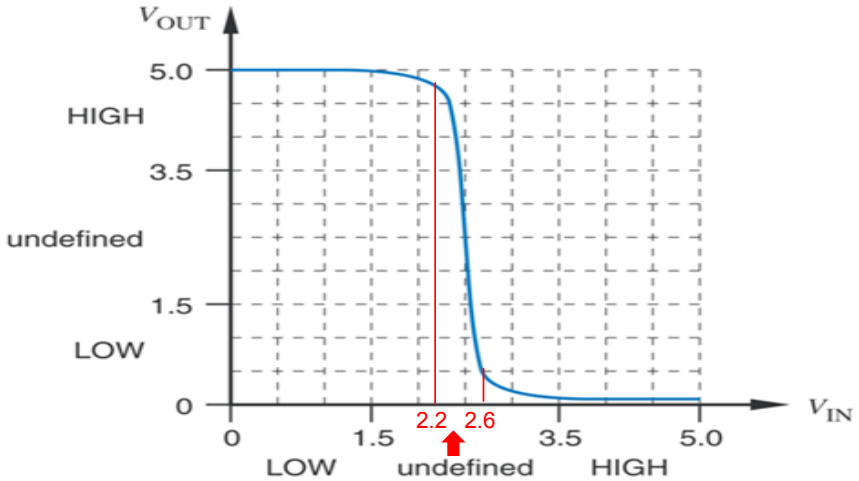
- The table defined the CMOS inverter's behavior only at two discrete input voltages
- Other input voltages may yield different output voltages.

V_{IN}	*Q1*	*Q2*	V_{OUT}
0.0 (L)	off	on	5.0 (H)
5.0 (H)	on	off	0.0 (L)



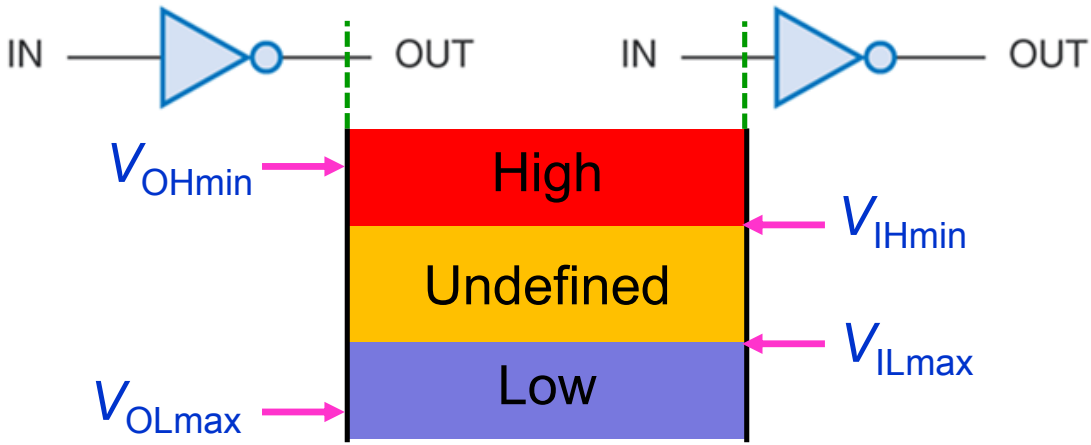
Input-output Transfer Characteristic

- The input-output transfer characteristic of an inverter can be described by a graph – Transfer Curve.
- e.g. LOW input level is any voltage under ≈ 2.2 V, and a HIGH input level is above ≈ 2.6 V.



Input / Output Voltages

- V_{OHmin} : The minimum output voltage for the HIGH state.
- V_{IHmin} : The minimum input voltage recognized as HIGH.
- V_{ILmax} : The maximum input voltage recognized as LOW.
- V_{OLmax} : The maximum output voltage for the LOW state.



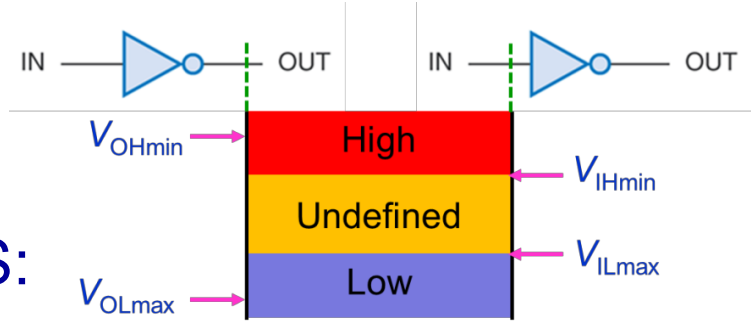
Typical Min/Max Voltages

- Input voltages are determined by switching thresholds of the two transistors, while output voltages are determined mainly by the “on” resistance of the transistors.
- All parameters are guaranteed by manufacturer over a range of temperature and output loading, as well as over a range of power-supply voltage V_{CC}, typically 5.0 V±10%.
- V_{CC} and ground are often called *power-supply rails*.
- CMOS levels are typically a function of the power-supply rails:

V_{OHmin}	V_{CC} – 0.1 V
V_{IHmin}	70% of V_{CC}
V_{ILmax}	30% of V_{CC}
V_{OLmax}	ground + 0.1 V

DC Noise Margin

- ***DC noise margin*** is a measure of how much noise it takes to corrupt a worst-case output voltage into a value that may be recognized properly by an input.



e.g., In HC-series CMOS:

Low-state DC noise margin

$= V_{ILmax} - V_{OLmax} = 1.35\ V - 0.1\ V = 1.25\ V$

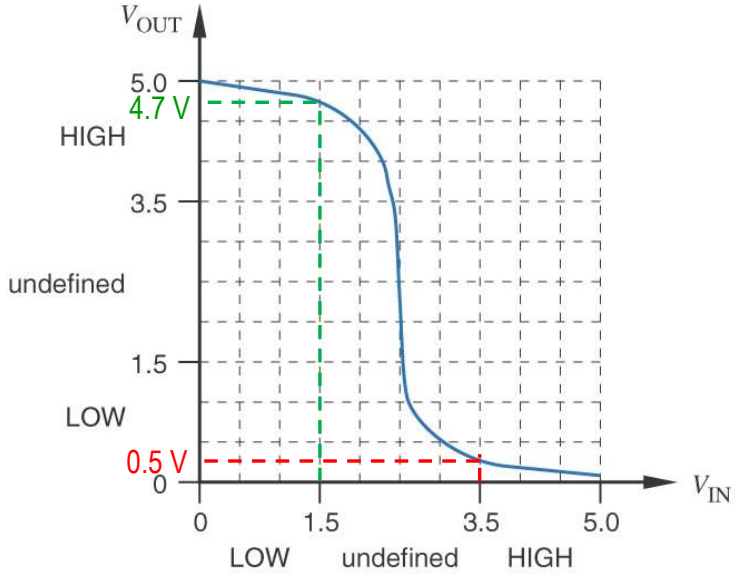
High-state DC noise margin

$= V_{OHmin} - V_{IHmin} = 4.4\ V - 3.85\ V = 0.55\ V$

- In general, CMOS outputs have excellent DC noise margins when driving other CMOS inputs.

Example on Noise Margin

- How much low-state DC noise margin is available in a CMOS inverter whose transfer characteristic under the worst-case condition that looks like the figure below?
- How much high-state DC noise margin is available? (Assume 1.5-V and 3.5-V thresholds for LOW and HIGH.)



Answer:

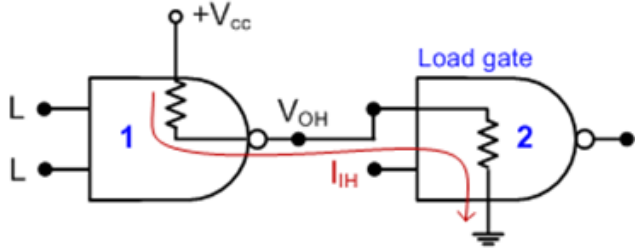
Low-state DC noise margin =
$V_{ILmax} - V_{OLmax}$ = 1.5 V − 0.5 V = 1.0 V

High-state DC noise margin =
$V_{OHmin} - V_{IHmin}$ = 4.7 V − 3.5 V = 1.2 V

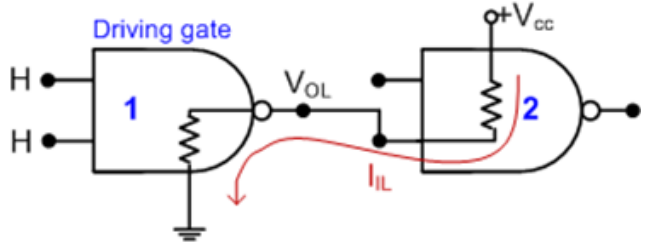
Input Currents

- CMOS gate inputs have very high impedance and consume very little current from the driving circuits.
- Input currents:
  - I_{IH} = The maximum current that flows into the input in HIGH state.
  - I_{IL} = The maximum current that flows into the input in LOW state.
- When the output of a CMOS circuit is connected to a load, the output transistor is “on” and has a nonzero resistance. If current is too large, then
  - In HIGH state, output may be lower than V_{OHmin}
  - In LOW state, output may be higher than V_{OLmax}

⇨ Wrong Logic



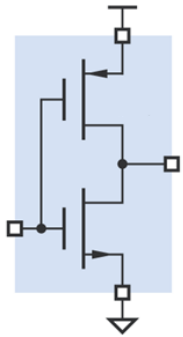
HIGH State

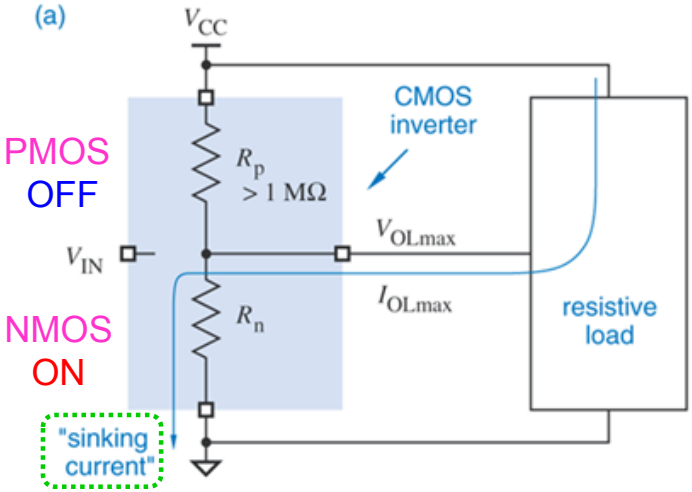


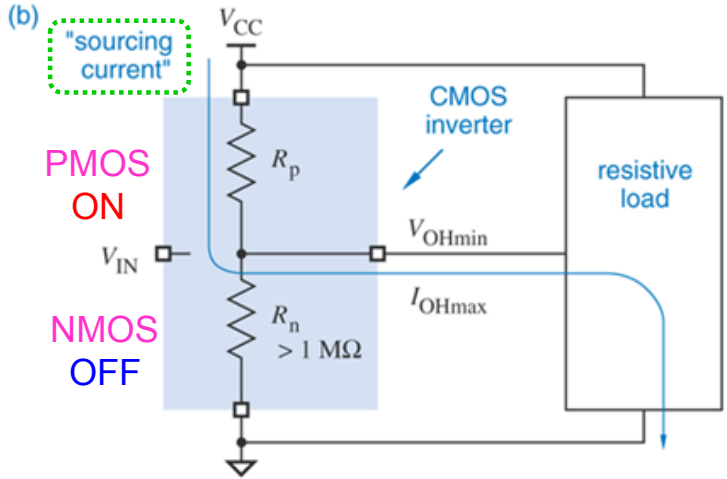
Low State

Sinking and Sourcing Currents

- (a) A device output is said to ***sink current*** when current flows from the power supply, through the load, and through the device output **to ground**.
- (b) The output is said to ***source current*** when current flows **from the power supply**, out of the device output, and through the load to ground.

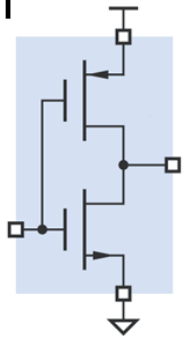


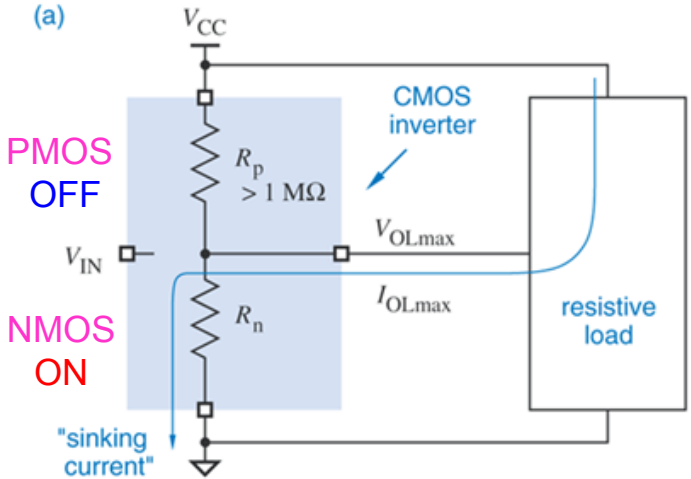


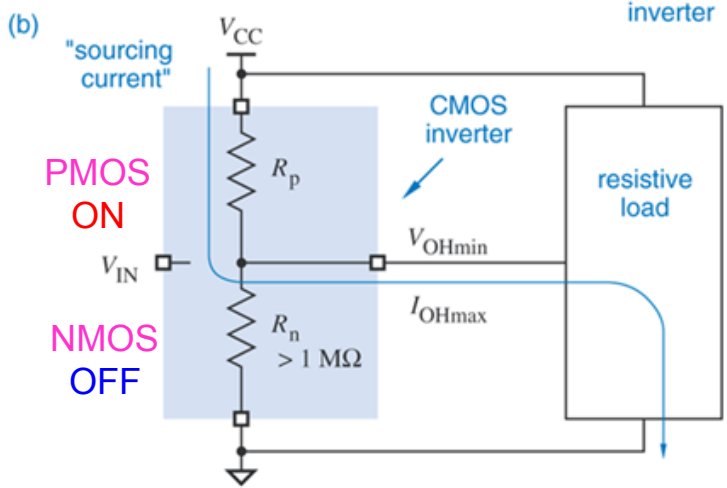


Output Currents

- (a) I_{OLmax} = The maximum current that the output can sink in the (output) LOW state while still maintaining an output voltage no greater than V_{OLmax}.
- (b) I_{OHmax} = The maximum current that the output can source in the (output) HIGH state while still maintaining an output voltage no less than V_{OHmin}.

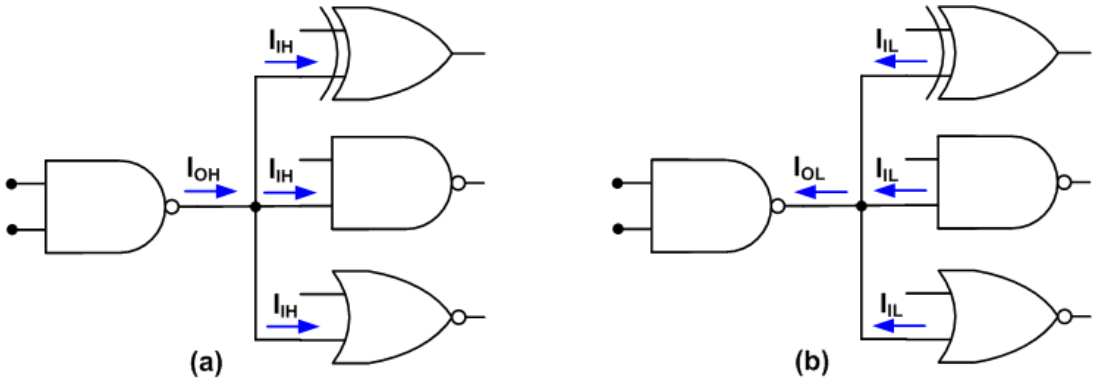






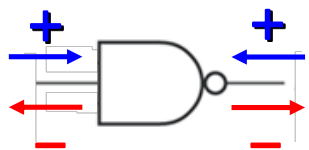
Fanout

- The fanout of a logic gate is the number of inputs that the gate can drive without exceeding its worst-case loading specifications (logic level still correct).
- The fanout is limited by the input and output currents.
- High State: fanout = I_{OHmax} / I_{IHmax}
- Low State: fanout = I_{OLmax} / I_{ILmax}
- The fanout of a gate is the minimum of the above two.
- The example below has fanout = 3.



Fanout Example

- A CMOS gate driving CMOS inputs has I_{OLmax} = 0.018 mA and I_{OHmax} = −0.022 mA.
- The maximum input current I_{Imax} in any state is ±1.1 μA.
- What is the fanout for both the HIGH and LOW state?
- **Solutions:**
  - LOW-state fanout = 0.018 mA / 1.1 μA = ~~16.36~~
  - HIGH-state fanout = 0.022 mA / 1.1 μA = 20
- Note 1: High-state and LOW-state fanouts of a gate are not necessarily equal.
- Note 2: The fanout of the above example is **16**.
- Note 3: The fanout is an **Integer** value.
- Note 4: Currents are positive if they flow into the gate, and negative if they flow out.



Loading Effects

- Loading an output beyond its rated fanout has several effects:
  - In the LOW state, the output voltage (V_{OL}) may increase beyond V_{OLmax}. ⇨ Wrong Logic
  - In the HIGH state, the output voltage (V_{OH}) may fall below V_{OHmin}. ⇨ Wrong Logic
  - Propagation delay (to discuss later) to the output may increase beyond specifications.
  - Output rise and fall times (to discuss later) may increase beyond their specifications.
  - The operating temperature of the device may increase, thereby reducing the reliability.

Unused Inputs

- Unused CMOS inputs should never be left unconnected (or floating, or open).
- Since CMOS inputs have high impedance, it takes only a small amount of circuit noise to temporarily make a floating input look HIGH, creating some nasty intermittent circuit failures.

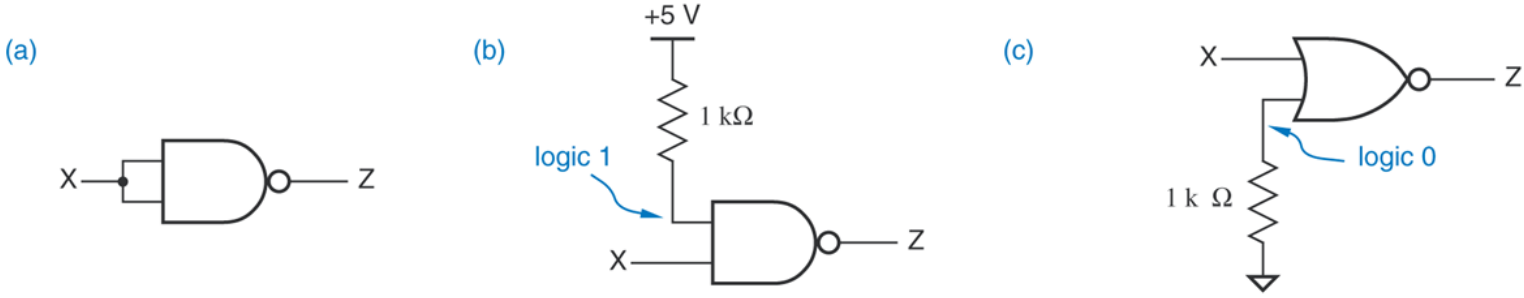


Figure 3-35

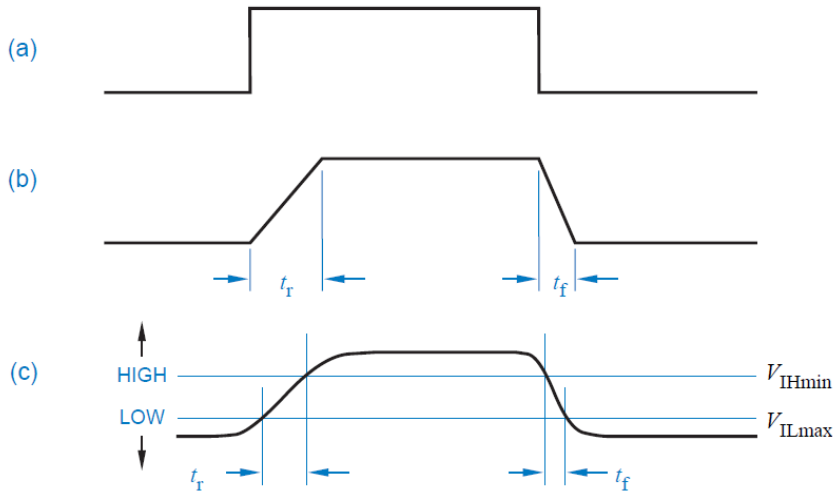
Unused inputs: (a) tied to another input; (b) NAND pulled up; (c) NOR pulled down.

CMOS Dynamic Electrical Behavior

- Both **speed** and **power consumption** of a CMOS device depend greatly on the **"AC" or dynamic characteristics** of the device and its load, i.e., what happens when the output changes between states.
- Digital designers must carefully examine the effects of output loading, and resize or redesign circuits where the load is too high.

Transition Time

- ***Transition time*** – the amount of time that the output of a logic circuit takes to change from one state to another.
- An output takes a certain time, called the ***rise time*** (**t_r**), to change from LOW to HIGH, and a possibly different time, called the ***fall time*** (**t_f**), to change from HIGH to LOW.



(a) Idea case: zero-time switching;

(b) A more realistic approximation;

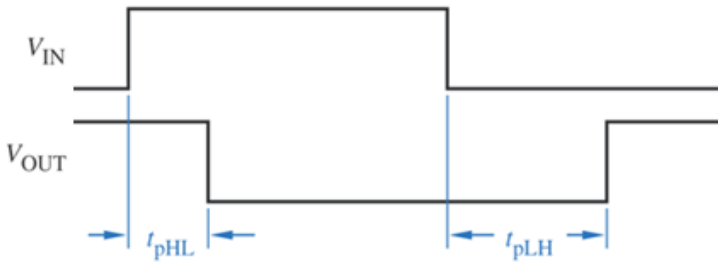
(c) Actual timing: rise and fall times shown.

Propagation Delay

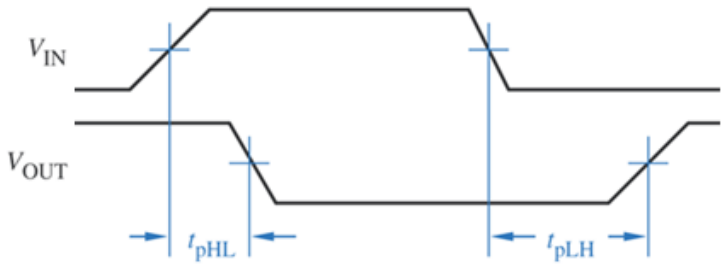
- Rise and fall times only partially describe the dynamic behavior of a logic element.
- A ***signal path*** is the electrical path from a particular input signal to a particular output signal of a logic element.
- The ***propagation delay*** t_p of a signal path is the amount of time that it takes for a change in the input signal to produce a change in the output signal.

Propagation Delays

- t_{pHL} = The time between an input change and the corresponding output change when the output is changing from HIGH to LOW.
- t_{pLH} = The time between an input change and the corresponding output change when the output is changing from LOW to HIGH.



Without
rise and fall times



Measured at
midpoints of transitions

Driving LEDs

- A LED can be turned “on” with an ordinary CMOS output.

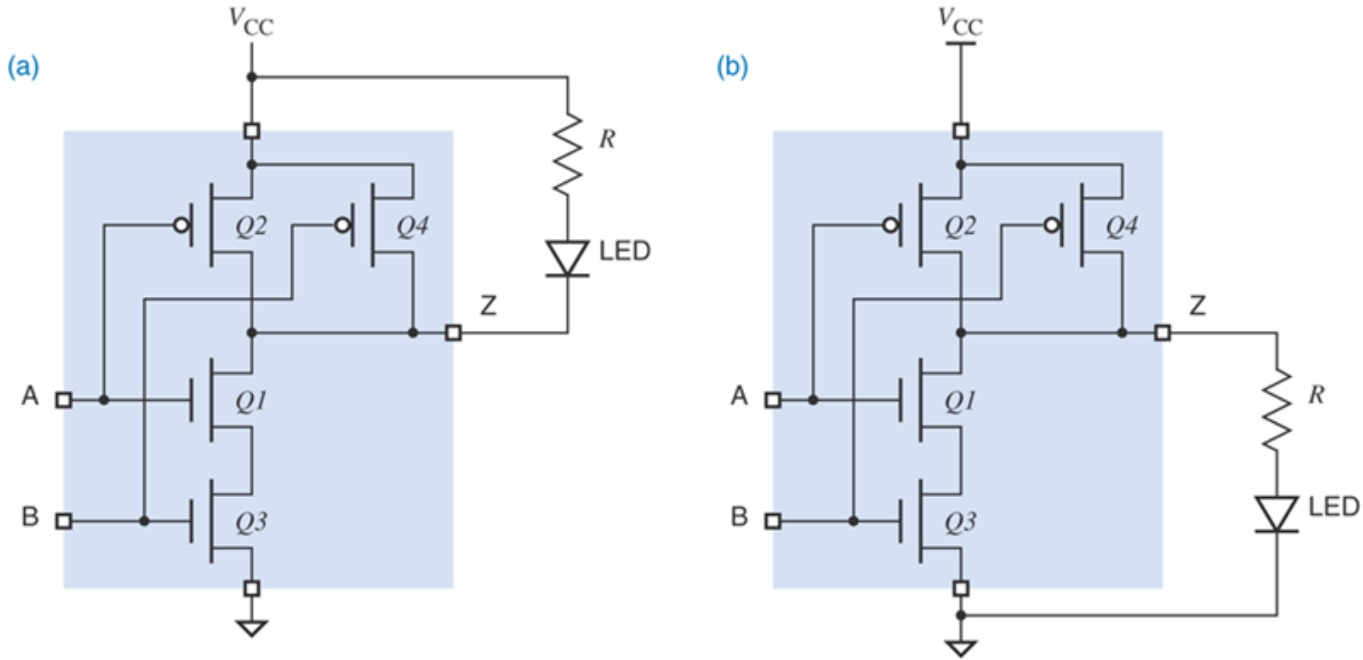
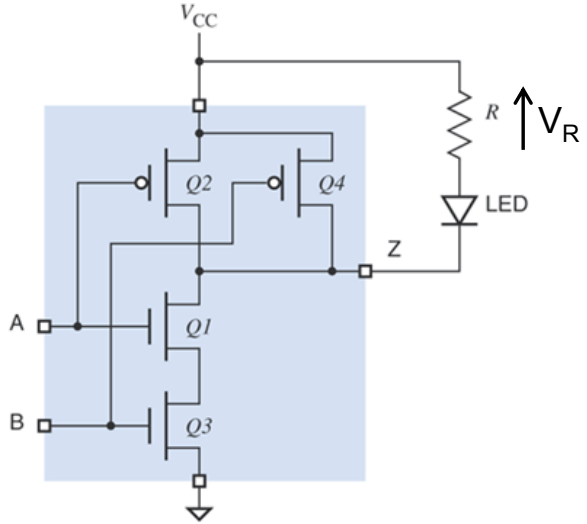


Figure 3-54 Driving an LED with an ordinary CMOS output: (a) sinking current, “on” in the LOW state; (b) sourcing current, “on” in the HIGH state.

Work Example for Driving LED

- A particular LED has a voltage drop of ~2 V in the "on" state and requires ~5 mA of current for normal brightness. Determine an appropriate value for the pull-up resistor when the LED is connected to a NAND gate, which has a V_{OLmax} of 0.37 V, as shown below with V_{CC} = 5 V.



Solution:

Voltage drop across R:

$V_R = 5 - 2 - 0.37 = 2.63$ V

Ohm's Raw: $R = V / I$

$R = 2.63 / 0.005 = 526\ \Omega$

(A single 510 Ω resistor can be used)

Wired Logic

- The wired output (z) is HIGH if and only if all of the individual gate outputs are HIGH (actually open);
- Any output going LOW will pull the wired output LOW.
- This yields an AND function.

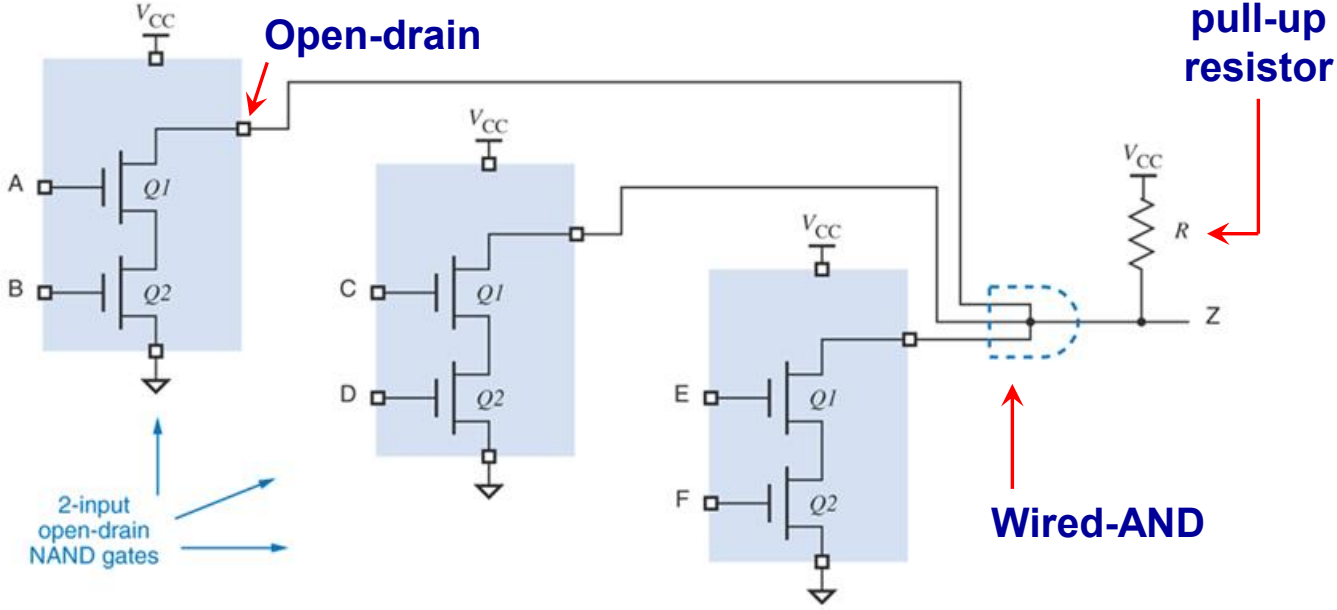


Figure 3-56 Wired-AND function on three open-drain NAND-gate outputs.

CMOS Logic Families

- All CMOS devices have part numbers of the form "74FAM*nn*", where "FAM" is an alphabetic family mnemonic and *nn* is a numeric function designator.
- Devices in different family with the same value of *nn* perform the same function.
  - E.g., 74HC30, 74HCT30, 74AC30, 74ACT30, 74AHC30, and 74AHCT30 all are 8-input NAND gates.
- The prefix "74" is simply a number used by an early popular supplier of TTL devices, Texas Instruments. The prefix "54" is used for identical parts that are specified for operation over a wide range of temperature and power-supply voltage, for use in military applications.

CMOS Logic Families: HC and HCT

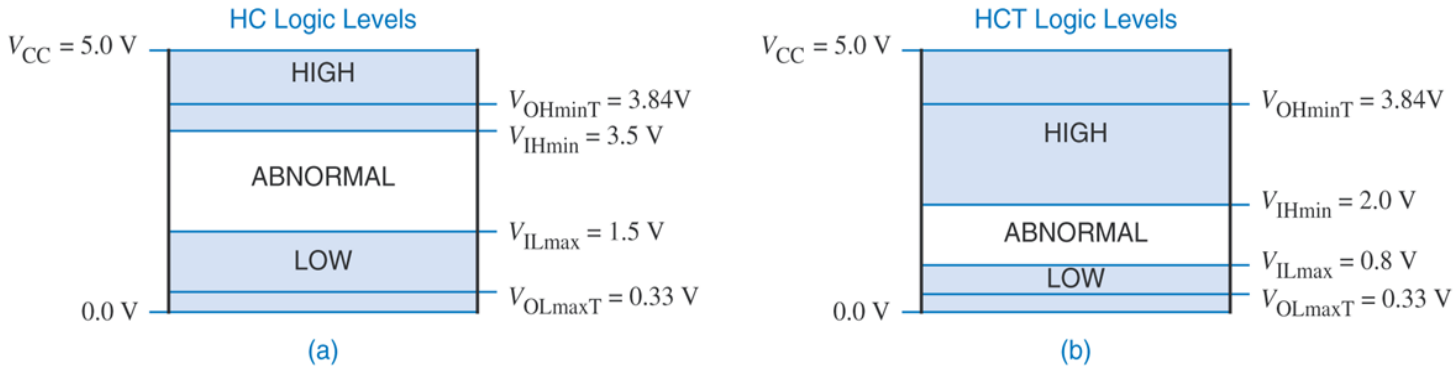
- The first two 74-series CMOS families are:
  - HC (High-speed CMOS) and
  - HCT (High-speed CMOS, TTL compatible).
- HC circuits are designed to recognize CMOS input levels, and can use any power-supply voltage 2 to 6 V.
- A higher voltage is used for higher speed, and a lower voltage for lower power dissipation.
- Note that power dissipation, $P = C_{PD} \cdot V_{CC}^2 \cdot f$

  where f = transition frequency of the output signal and

  C_{PD} = power-dissipation capacitance.

CMOS Logic Families: HC and HCT

- The HCT family uses a power-supply voltage V_{CC} of 5 V.
- HCT devices uses different input levels from HC devices.
- These levels are established in the fabrication process by making transistors with different switching thresholds, producing the different transfer characteristics.



Input and output levels for CMOS devices using a 5-V supply: (a) HC; (b) HCT.

CMOS Logic Families: AHC and AHCT

- Two of the most recent and probably most versatile CMOS families are:
  - *AHC (Advanced High-speed CMOS)* and
  - *AHCT (Advanced High-speed CMOS, TTL compatible).*
- AHC/AHCT families are 2 to 3 times faster than HC/HCT families.
- Like HC/HCT, AHC/AHCT outputs have ***symmetric output drive***. That is, an output can sink or source equal amounts of current; the current is just as “strong” in both cases.

Speed and power of CMOS families

Description	Part	Symbol	Condition	Family			
				HC	HCT	AHC	AHCT
Typical propagation delay (ns)	'00	t_{PD}		9	10	3.7	5
	'138			18	20	5.7	7.6
Quiescent power-supply current (μA)	'00	I_{CC}	$V_{in} = 0$ or V_{CC}	2.5	2.5	5.0	5.0
	'138		$V_{in} = 0$ or V_{CC}	40	40	40	40
Quiescent power dissipation (mW)	'00		$V_{in} = 0$ or V_{CC}	0.0125	0.0125	0.025	0.025
	'138		$V_{in} = 0$ or V_{CC}	0.2	0.2	0.2	0.2
Power-dissipation capacitance (pF)	'00	C_{PD}		22	15	2.4	2.6
	'138	C_{PD}		55	51	13	14
Dynamic power dissipation (mW/MHz)	'00			0.55	0.38	0.06	0.065
	'138			1.38	1.28	0.33	0.35
Total power dissipation (mW)	'00		$f = 100$ kHz	0.068	0.050	0.031	0.032
	'00		$f = 1$ MHz	0.56	0.39	0.085	0.09
	'00		$f = 10$ MHz	5.5	3.8	0.63	0.68
	'138		$f = 100$ kHz	0.338	0.328	0.23	0.24
	'138		$f = 1$ MHz	1.58	1.48	0.53	0.55
	'138		$f = 10$ MHz	14.0	13.0	3.45	3.7
Speed-power product (pJ)	'00		$f = 100$ kHz	0.61	0.50	0.11	0.16
	'00		$f = 1$ MHz	5.1	3.9	0.31	0.45
	'00		$f = 10$ MHz	50	38	2.3	3.38
	'138		$f = 100$ kHz	6.08	6.55	1.33	1.79
	'138		$f = 1$ MHz	28.4	29.5	2.99	4.2
	'138		$f = 10$ MHz	251	259	19.7	28.1

Input Specifications for CMOS Families

Description	Symbol	Condition	Family			
			HC	HCT	AHC	AHCT
Input leakage current (μA)	I_{Imax}	V_{in} = any	±1	±1	±1	±1
Maximum input capacitance (pF)	C_{INmax}		10	10	10	10
LOW-level input voltage (V)	V_{ILmax}		1.35	0.8	1.35	0.8
HIGH-level input voltage (V)	V_{IHmin}		3.85	2.0	3.85	2.0

Table 3-6

Input specifications for CMOS families with V_{CC} between 4.5 and 5.5 V.

Output Specifications for CMOS Families

Description	Symbol	Condition	Family			
			HC	HCT	AHC	AHCT
LOW-level output current (mA)	I_{OLmaxC}	CMOS load	0.02	0.02	0.05	0.05
	I_{OLmaxT}	TTL load	4.0	4.0	8.0	8.0
LOW-level output voltage (V)	V_{OLmaxC}	$I_{out} \leq I_{OLmaxC}$	0.1	0.1	0.1	0.1
	V_{OLmaxT}	$I_{out} \leq I_{OLmaxT}$	0.33	0.33	0.44	0.44
HIGH-level output current (mA)	I_{OHmaxC}	CMOS load	−0.02	−0.02	−0.05	−0.05
	I_{OHmaxT}	TTL load	−4.0	−4.0	−8.0	−8.0
HIGH-level output voltage (V)	V_{OHminC}	$\lvert I_{out}\rvert \leq \lvert I_{OHmaxC}\rvert$	4.4	4.4	4.4	4.4
	V_{OHminT}	$\lvert I_{out}\rvert \leq \lvert I_{OHmaxT}\rvert$	3.84	3.84	3.80	3.80

Table 3-7

Output specifications for CMOS families operating with V_{CC} between 4.5 and 5.5 V.

Low-Voltage CMOS Logic

- In most applications, CMOS output voltages swing from rail to rail, so the *V* in the CV^2f equation is the power-supply voltage. Cutting power-supply voltage reduces dynamic power dissipation more than proportionally.
- As transistor geometries continue to shrink, oxide insulation between a CMOS transistor's gate and its source and drain becomes even thinner; hence incapable of insulating voltages as "high" as 5 V.
- As a result, JEDEC (Joint Electron Devices Engineering Council), an IC industry standards group, selected 3.3V ± 0.3V, 2.5V ± 0.2V, 1.8V ± 0.15V, 1.5 ± 0.1V, and 1.2 ± 0.1V as the next "standard" logic voltage levels.

Low-Voltage CMOS Logic Families

- The migration to lower voltages has occurred in stages and will continue to do so.
- For discrete logic families, the trend has been to produce parts that operate and produce outputs at the lower voltage but can also tolerate inputs at the higher voltage. This approach has allowed 3.3-V CMOS families to operate with 5-V CMOS and TTL families.
- The basic CMOS series for the 3.3-V category and their designations are as follows:
  - *74LV and 74LVC — Low-voltage CMOS*
  - *74ALVCMOS — Advanced Low-voltage CMOS*

Comparison of Logic Levels

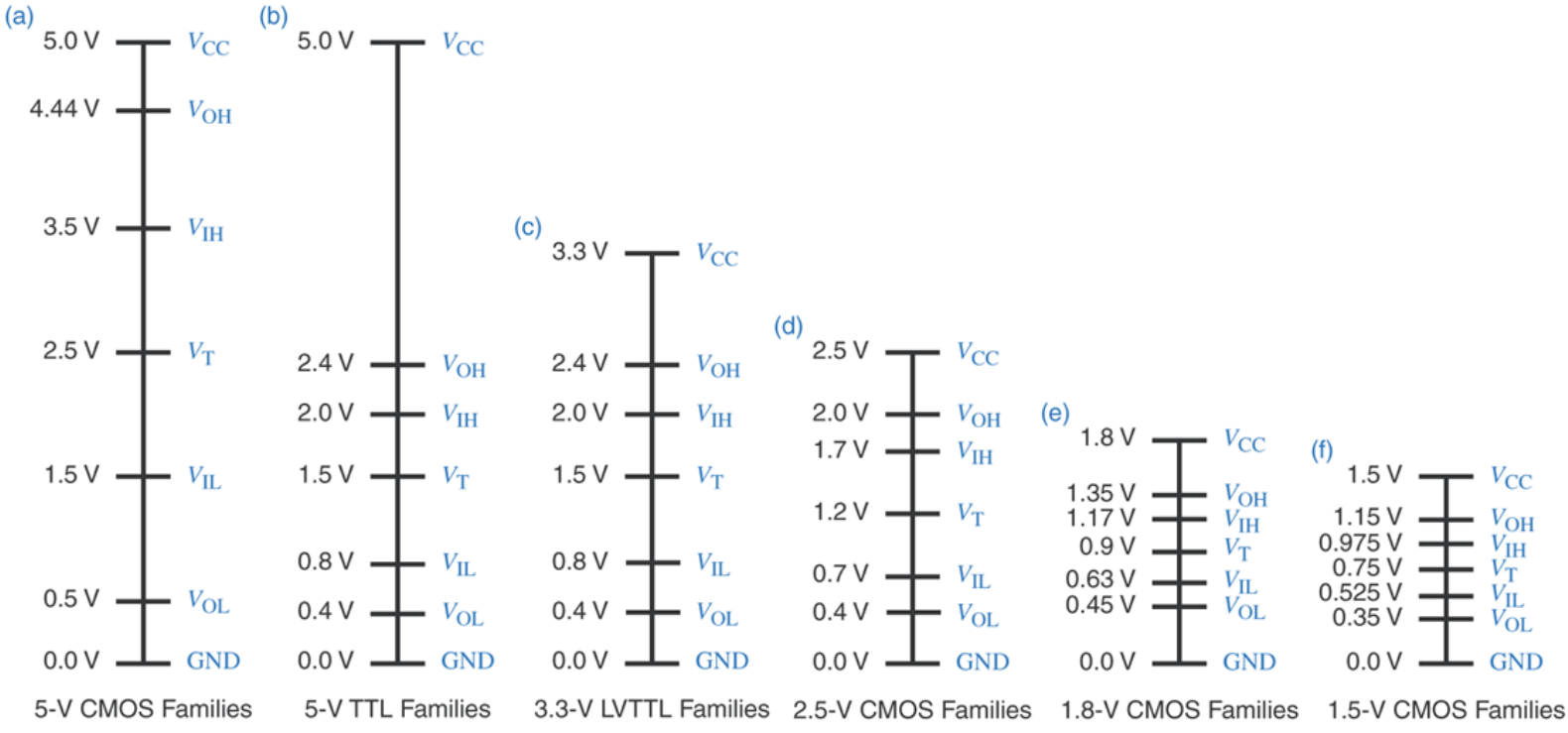


Figure 3-62

Comparison of logic levels: (a) 5-V CMOS; (b) 5-V TTL, including 5-V TTL-compatible CMOS; (c) 3.3-V LVTTL; (d) 2.5-V CMOS; (e) 1.8-V CMOS; (f) 1.5-V CMOS.

Transistor-Transistor Logic

- The most commonly used bipolar logic family is transistor-transistor logic (TTL). One advantage of TTL is that it is not sensitive to electrostatic discharge whereas CMOS is.
- There are many different TTL families, with a range of speed, power consumption and other characteristics. All of which operate from a 5-V dc supply.
  - *74* — *standard TTL (no letter)*
  - *74S* — *Schottky TTL*
  - *74AS* — *Advanced Schottky TTL*
  - *74LS* — *Low-power Schottky TTL*
  - *74ALS* — *Advanced Low-power Schottky TTL*
  - *74F* — *Fast TTL*
- TTL use the same logic levels as TTL-compatible CMOS:
  - LOW : 0 – 0.8 volts
  - HIGH : 2.0 – 5.0 volts

Typical Data Sheet for 74LS00

RECOMMENDED OPERATING CONDITIONS

Parameter	Description	SN54LS00 Min.	SN54LS00 Nom.	SN54LS00 Max.	SN74LS00 Min.	SN74LS00 Nom.	SN74LS00 Max.	Unit
V_{CC}	Supply voltage	4.5	5.0	5.5	4.75	5.0	5.25	V
V_{IH}	High-level input voltage	2.0			2.0			V
V_{IL}	Low-level input voltage			0.7			0.8	V
I_{OH}	High-level output current			−0.4			−0.4	mA
I_{OL}	Low-level output current			4			8	mA
T_A	Operating free-air temperature	−55		125	0		70	°C

ELECTRICAL CHARACTERISTICS OVER RECOMMENDED FREE-AIR TEMPERATURE RANGE

Parameter	Test Conditions[1]	SN54LS00 Min.	SN54LS00 Typ.[2]	SN54LS00 Max.	SN74LS00 Min.	SN74LS00 Typ.[2]	SN74LS00 Max.	Unit
V_{IK}	V_{CC} = Min., I_N = −18 mA			−1.5			−1.5	V
V_{OH}	V_{CC} = Min., V_{IL} = Max., I_{OH} = −0.4 mA	2.5	3.4		2.7	3.4		V
V_{OL}	V_{CC} = Min., V_{IH} = 2.0 V, I_{OL} = 4 mA		0.25	0.4		0.25	0.4	V
	V_{CC} = Min., V_{IH} = 2.0 V, I_{OL} = 8 mA					0.35	0.5	V
I_I	V_{CC} = Max., V_I = 7.0 V			0.1			0.1	mA
I_{IH}	V_{CC} = Max., V_I = 2.7 V			20			20	μA
I_{IL}	V_{CC} = Max., V_I = 0.4 V			−0.4			−0.4	mA
I_{OS}[3]	V_{CC} = Max.	−20		−100	−20		−100	mA
I_{CCH}	V_{CC} = Max., V_I = 0 V		0.8	1.6		0.8	1.6	mA
I_{CCL}	V_{CC} = Max., V_I = 4.5 V		2.4	4.4		2.4	4.4	mA

SWITCHING CHARACTERISTICS, V_{CC} = 5.0 V, T_A = 25° C

Parameter	From (Input)	To (Output)	Test Conditions	Min.	Typ.	Max.	Unit
t_{PLH}	A or B	Y	R_L = 2 kΩ, C_L = 15 pF		9	15	ns
t_{PHL}					10	15	ns

NOTES:

1. *For conditions shown as Max. or Min., use appropriate value specified under Recommended Operating Conditions.*
2. *All typical values are at V_{CC} = 5.0 V, T_A = 25 °C.*
3. *Not more than one output should be shorted at a time; duration of short-circuit should not exceed one second.*

Work Example on Noise Margin

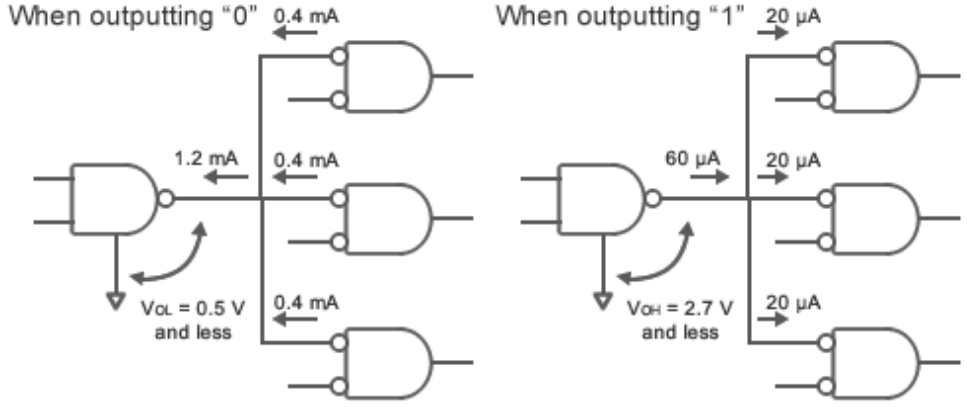
- Using the typical data sheet for the 74LS00 on last slide, determine the worst-case LOW-state and HIGH-state DC noise margins of the 74LS00.

Solutions:

- For I_{OL} = 8 mA, the worst-case LOW-state noise margin is: $V_{ILmax} - V_{OLmax} = 0.8 - 0.5 = 0.3$ V.
- For I_{OL} = 4 mA, the worst-case LOW-state noise margin is: $V_{ILmax} - V_{OLmax} = 0.8 - 0.4 = 0.4$ V.
- The worst-case HIGH-state noise margin in any case is: $V_{OHmin} - V_{IHmin} = 2.7 - 2.0 = 0.7$ V.
- For the 54LS00, the worst-case LOW-state noise margin is 0.3 V and the worst-case HIGH-state noise margin is 0.5 V.

Fanout

- Fanout is a measure of the number of gate inputs that are connected to (and driven by) a single gate output.



$$\text{TTL Fan-out} = \frac{\text{L output current } (I_{OL})}{\text{L input current } (I_{IL})} , \frac{\text{H output current } (I_{OH})}{\text{H input current } (I_{IH})}$$

- Fanout = I_{OLmax} / I_{ILmax} (Low) or I_{OHmax} / I_{IHmax} (High).
- **The overall fanout is the lower of the two values.**

Work Example on Fanout

- Compute the fanout for each of the following cases of a TTL-compatible CMOS output driving multiple inputs of TTL gates. Indicate how much “excess” driving capability is available in the LOW or HIGH state for each case.
- (a) 74HCT driving 74LS (b) 74HCT driving 74S
- (c) 74VHCT driving 74AS (d) 74VHCT driving 74LS

TTL-compatible CMOS Logic Family	$I_{OHmax} = I_{OLmax}$
74HCT	4 mA
74VHCT	8 mA

TTL Logic Family	I_{ILmax}	I_{IHmax}
74S (Schottky TTL)	2 mA	50 μA
74LS (Low-power Schottky)	0.4 mA	20 μA
74AS (Advanced Schottky TTL)	2 mA	20 μA

Solution for Work Example on Fanout

Case	LOW-state Ratio	LOW-state Fanout	HIGH-state Ratio	HIGH-state Fanout	Overall Fanout	Excess State	Excess Drive
74HCT driving 74LS	$\frac{4\text{mA}}{0.4\text{mA}}$	10	$\frac{4000\ \mu\text{A}}{20\ \mu\text{A}}$	200	10	HIGH	3800 µA
74HCT driving 74S	$\frac{4\text{mA}}{2\text{mA}}$	2	$\frac{4000\ \mu\text{A}}{50\ \mu\text{A}}$	80	2	HIGH	3900 µA
74VHCT driving 74AS	$\frac{8\text{mA}}{2\text{mA}}$	4	$\frac{8000\ \mu\text{A}}{20\ \mu\text{A}}$	400	4	HIGH	7920 µA
74VHCT driving 74LS	$\frac{8\text{mA}}{0.4\text{mA}}$	20	$\frac{8000\ \mu\text{A}}{20\ \mu\text{A}}$	400	20	HIGH	7600 µA

What do we know by now?

- By now, we should know:
  - Static and dynamic electrical behaviors of CMOS logic circuit
  - Logic levels and noise margins
  - Fan-out and fan-in
  - Transition time and propagation delays
  - Wired logic
  - Different CMOS logic families

Topic: Combinational Logic Principles

- By the end of this topic, we will know:
  - What Boolean algebra is
  - Axioms of Switching algebra
  - Switching algebra theorems
  - How to manipulate Boolean functions using Boolean laws
  - What Minterms and Maxterms are
  - How to derive the logic function from truth table
  - How to express a logic function in truth table, minterm expression or maxterm expression
  - Logic circuit manipulation

Boolean Algebra

- In 1854, George Boole (English,1815–1864) introduced a systematic manipulation of logic — algebraic system, called ***Boolean algebra***
- Using this system, one can formulate propositions that are true or false
- In 1938, Bell Labs researcher Claude E. Shannon adapted Boolean algebra to analyze and describe the behavior of circuits built from *relays*.
- In Shannon's switching algebra, the condition of a relay contact, open or closed, is represented by a variable X that can have two possible values, 0 or 1.

Switching Algebra

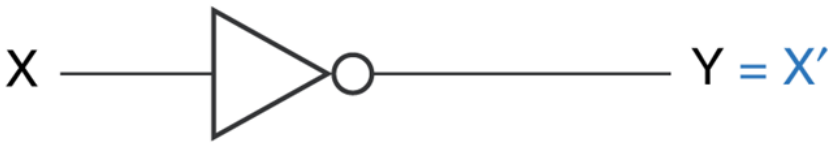
- In switching algebra, symbolic variable, such as X, is used to represent the condition of a logic signal.
- A logic signal is one of two possible conditions — low or high, off or on, etc.
- ***Positive-logic convention*** associate the value "0" with a LOW voltage and "1" with a HIGH voltage, denoted as **X(H)**.
- ***Negative-logic convention*** makes the opposite association: 0 = HIGH and 1 = LOW, denoted as **X(L)**.

Axioms of Switching Algebra

- The Axioms of a mathematical system are a minimal set of basic definitions that we assume to be true, from which all other information about the system can be derived.

(A1)	X = 0 if X ≠ 1	(A1')	X = 1 if X ≠ 0
(A2)	If X = 0 then X' = 1	(A2')	If X = 1 then X' = 0

- The above axioms state that a variable X can take on only one of two values.
- In inverter, the output signal level is the opposite (or complement) of its input signal level. A prime (') or overbar ($\bar{}$) is used to denote the value of the output signal.



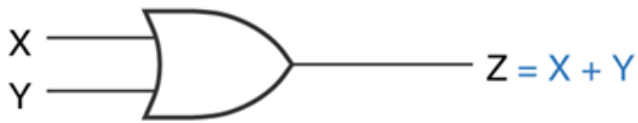
Axioms of Switching Algebra for AND & OR

- Axioms for the formal definitions of AND and OR operations:

(A3)	$0 \cdot 0 = 0$	(A3')	$1 + 1 = 1$
(A4)	$1 \cdot 1 = 1$	(A4')	$0 + 0 = 0$
(A5)	$0 \cdot 1 = 1 \cdot 0 = 0$	(A5')	$1 + 0 = 0 + 1 = 1$

- AND operation is sometimes called ***logic multiplication*** and is symbolized algebraically by a *multiplication dot* (·).
- OR operation is sometimes called ***logic addition*** and is symbolized algebraically by a plus sign (+).





Single-Variable Theorems

- The table below lists switching-algebra theorems involving a single variable X.

(T1)	$X + 0 = X$	(T1′)	$X \cdot 1 = X$	(Identities)
(T2)	$X + 1 = 1$	(T2′)	$X \cdot 0 = 0$	(Null elements)
(T3)	$X + X = X$	(T3′)	$X \cdot X = X$	(Idempotency)
(T4)	$(X')' = X$			(Involution)
(T5)	$X + X' = 1$	(T5′)	$X \cdot X' = 0$	(Complements)

- These theorems can be proven using *perfect induction*, i.e., proof by showing that it is true for all possible cases.
- E.g., in T1, there is an identity element with respect to:
  (a) **+**, designated by **0** : **x + 0 = 0 + x = x**
  (b) •, designated by **1** : **x • 1 = 1 • x = x**

Two- and Three-Variable Theorems

- Switching Algebra theorems with two or three variables are listed below.

(T6)	$X + Y = Y + X$	(T6′)	$X \cdot Y = Y \cdot X$	(Commutativity)
(T7)	$(X + Y) + Z = X + (Y + Z)$	(T7′)	$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$	(Associativity)
(T8)	$X \cdot Y + X \cdot Z = X \cdot (Y + Z)$	(T8′)	$(X + Y) \cdot (X + Z) = X + Y \cdot Z$	(Distributivity)
(T9)	$X + X \cdot Y = X$	(T9′)	$X \cdot (X + Y) = X$	(Covering)
(T10)	$X \cdot Y + X \cdot Y' = X$	(T10′)	$(X + Y) \cdot (X + Y') = X$	(Combining)
(T11)	$X \cdot Y + X' \cdot Z + Y \cdot Z = X \cdot Y + X' \cdot Z$			(Consensus)
(T11′)	$(X + Y) \cdot (X' + Z) \cdot (Y + Z) = (X + Y) \cdot (X' + Z)$			

- Each theorem can easily be proven by perfect induction (a.k.a. proof by exhaustion or proof by cases)
- These are discussed one by one next.

Boolean Algebra Postulates

(T6) Commutative with respect to + : $X + Y = Y + X$

(T6') Commutative with respect to • : $X \bullet Y = Y \bullet X$

(T7) Associative law : $(X + Y) + Z = X + (Y + Z)$

(T7') Associative law : $(X \bullet Y) \bullet Z = X \bullet (Y \bullet Z)$

(T8) • is distributive over + : $X \bullet (Y + Z) = X \bullet Y + X \bullet Z$

(T8') + is distributive over • : $X + (Y \bullet Z) = (X + Y) \bullet (X + Z)$

Covering theorem (a.k.a. Absorption law):

(T9) $X + X \bullet Y = X$

(X covers X • Y or X • Y is absorbed into X)

(T9') $X \bullet (X + Y) = X$ (X covers X + Y)

(X covers X + Y or X + Y is absorbed into X)

Boolean Algebra Postulates

Combining theorem:

(T10) $X \bullet Y + X \bullet Y' = X$

(T10') $(X + Y) \bullet (X + Y') = X$

Consensus theorem:

(T11) $X \bullet Y + X' \bullet Z + Y \bullet Z = X \bullet Y + X' \bullet Z$

(The $Y \bullet Z$ term is called the consensus of the two terms $X \bullet Y$ and $X' \bullet Z$)

(T11') $(X + Y) \bullet (X' + Z) \bullet (Y + Z) = (X + Y) \bullet (X' + Z)$

DeMorgan theorem:

(T12) $(X + Y)' = X' \bullet Y'$

(T12') $(X \bullet Y)' = X' + Y'$

Absorption Law

x	y	x•y	x+x•y	x+y	x•(x+y)
0	0	0	0	0	0
0	1	0	0	1	0
1	0	0	1	1	1
1	1	1	1	1	1



$$x + x \bullet y = x$$

$$x \bullet (x + y) = x$$

Consensus theorem

x	y	z	x•y	x'•z	y•z	x•y+x'•z+y•z	x•y+x'•z
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	1
0	1	0	0	0	0	0	0
0	1	1	0	1	1	1	1
1	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	1	0	1	1	1

DeMorgan Theorem

x	y	x+y	(x+y)'	x'	y'	x'•y'	x•y	(x•y)'	x'+y'
0	0	0	1	1	1	1	0	1	1
0	1	1	0	1	0	0	0	1	1
1	0	1	0	0	1	0	0	1	1
1	1	1	0	0	0	0	1	0	0

$$\overline{(x + y)} = \overline{x} \bullet \overline{y} \qquad \overline{(x \bullet y)} = \overline{x} + \overline{y}$$

DeMorgan Theorem – "Break the Bar & Change the Sign"

$$\overline{x + y} = \overline{x} \bullet \overline{y} \qquad \overline{x \bullet y} = \overline{x} + \overline{y}$$

Equivalent Circuits for NAND Gate

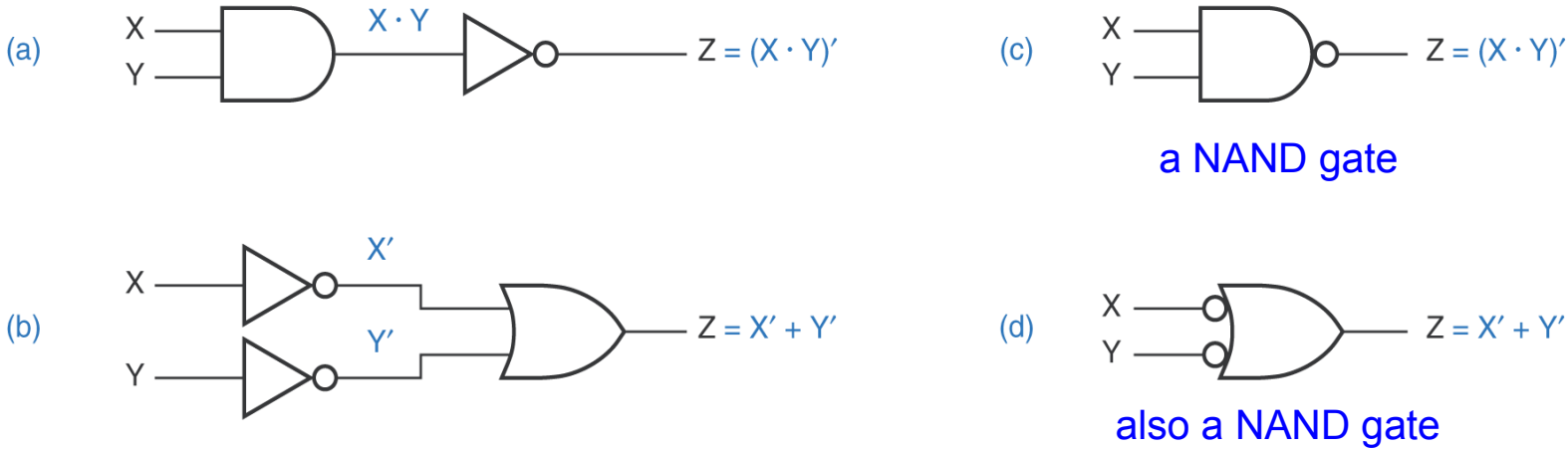


Figure 4-3

Equivalent circuits according to DeMorgan's theorem T13: (a) AND-NOT; (b) NOT-OR; (c) logic symbol for a NAND gate; (d) equivalent symbol for a NAND gate.

Equivalent Circuits for NOR Gate

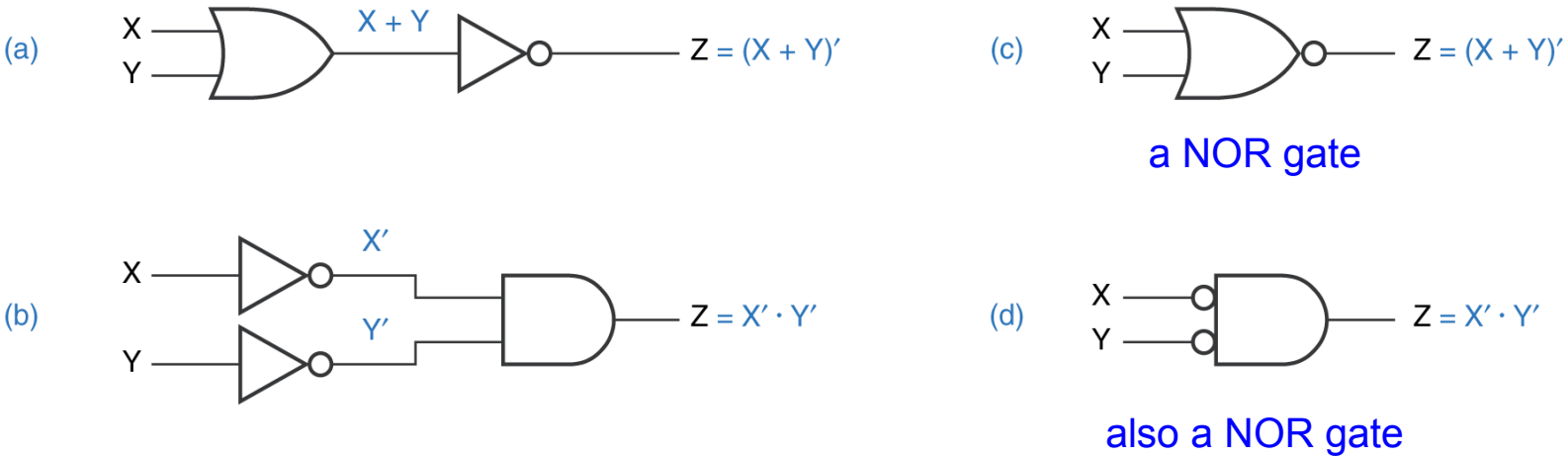


Figure 4-4

Equivalent circuits according to DeMorgan's theorem T13' :(a) OR-NOT; (b) NOT-AND; (c) logic symbol for a NOR gate; (d) equivalent symbol for a NOR gate.

Operations with XOR and XNOR

- XOR



a	b	y
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

- $a \oplus b = a'b + ab'$
- $a \oplus 0 = a$
- $a \oplus 1 = a'$
- $a \oplus a = 0$
- $a \oplus a' = 1$

- XNOR



a	b	y
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**1**

- $\overline{a \oplus b} = a'b' + ab$
- $\overline{a \oplus 0} = a'$
- $\overline{a \oplus 1} = a$
- $\overline{a \oplus a} = 1$
- $\overline{a \oplus a'} = 0$

Boolean Function

- A Boolean function is an algebraic expression consists of binary variables, the constants 0 & 1, and the Boolean operators.
- For a set of given values of the variables, the function is evaluated to either 0 or 1.
- e.g. f = x • y + x • z'
- The Boolean function f has 3 binary variables x, y and z.
- The function is 1 if x and y are both 1 or if x is 1 and z is 0. Otherwise, f = 0.

Operator Precedence

- The operator precedence is:
  1. **Parentheses**
  2. **NOT**
  3. **AND**
  4. **OR**
- e.g. **f = x • y + x • z'**
- Precedence: **z', x • y, x • z', x • y + x • z'**
- e.g. **f = (a +b) • (c+d')**
- Precedence: **a+b, d', c+d', (a +b) • (c+d')**
- The parentheses precedence are the same as in normal algebra

Boolean Function Truth Table

- Boolean function can be represented by Truth Table as well
- If the function has n variables, its truth table will have 2^n rows
- e.g. **f = x • y + x • z'**
- f has 3 variables
- 2^3 combinations
- f is 1 when the expression evaluated to 1. Otherwise 0.

x	y	z	f
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**
0	1	1	**0**
1	0	0	**1**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

2-Variable Function From Truth Table

- Boolean function can be derived directly from Truth Table
- The expression has all the '1' rows OR together
- e.g. f = x'y + xy
- f equals to 1 when x'y is 1 or xy is 1. i.e. xy = 01 or 11
- Also, f' = x'y' + xy'
- $\Rightarrow$ f = (x'y' + xy')' = (x + y)(x' + y)

x	y	f
0	0	0
0	1	**1**
1	0	0
1	1	**1**

3-Variable Function From Truth Table

- f = xy'z' + xyz' + xyz

  or

- f = (x + y + z)(x + y + z')
  (x + y' + z)(x + y' + z')
  (x' + y + z')

- **Exercise:**

  Show that the above two equations are equal.

x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Standard Representation of Logic Functions

- A literal is a variable in a product or sum term.
- x·y' is a 2-literal product term.
- x'·y·z has three literals.
- x' + x·y' + x'·y·z is a sum-of-products (SOP) expression with three product terms. The three product terms have one, two and three literals, respectively.
- x'·(x+y')·(x'+y+z) is an expression of product-of-sums (POS). The three sum terms have one, two and three literals.
- A normal term is a product or sum term in which no variable appears more than once. E.g., w·x·y'.
- Example of non-normal term: w·x·x·y'.

Minterm

- In a Boolean function, a binary variable (x) may appear either in its normal form (x) or in its complement form (x')
- Consider two binary variables x and y and an AND operation, there are four and only four possible combinations: x'•y', x'•y, x•y' and x•y
- Each of the four product terms is called a ***minterm*** or ***standard product***.

Minterm

- By definition, a ***minterm*** is a product which consists of all the variables in the normal or complement form but **NOT BOTH**.
- Minterm examples for a function with two variables x and y:

  x•y' is a minterm but **x' is NOT a minterm**
- Minterm examples for a function with three variables x, y and z:

  x'yz' is a minterm but **xy' is NOT a minterm**

Maxterm

- Consider two binary variables x and y and an OR operation, there are four and only four possible combinations: x'+y', x'+y, x+y', x+y.
- Each of the four sum terms is called a ***maxterm*** or ***standard sum***.
- By definition, a ***maxterm*** is a sum in which each variable appears once and only once either in its normal form or its complement form but **NOT BOTH**.

Minterms and Maxterms for 3 Variables

Variable			Minterm		Maxterm	
x	y	z	Term	Designation	Term	Designation
0	0	0	x'y'z'	m_0	x+y+z	M_0
0	0	1	x'y'z	m_1	x+y+z'	M_1
0	1	0	x'yz'	m_2	x+y'+z	M_2
0	1	1	x'yz	m_3	x+y'+z'	M_3
1	0	0	xy'z'	m_4	x'+y+z	M_4
1	0	1	xy'z	m_5	x'+y+z'	M_5
1	1	0	xyz'	m_6	x'+y'+z	M_6
1	1	1	xyz	m_7	x'+y'+z'	M_7

Minterm Boolean Expression

- Boolean functions can be expressed with minterms, e.g.
- $f_1(x,y,z) = m_1 + m_4 + m_6$
  $= x'y'z + xy'z' + xyz'$
  $= \Sigma m(1, 4, 6) = \Sigma(1, 4, 6)$
  - small 'm' for minterm
  - Σ for sum (i.e. OR)
- $f_2(x,y,z) = m_2 + m_4 + m_6 + m_7$
  $= \Sigma m(2, 4, 6, 7)$
  $= \Sigma(2, 4, 6, 7)$

x	y	z	f_1	f_2
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**0**	**1**
0	1	1	**0**	**0**
1	0	0	**1**	**1**
1	0	1	**0**	**0**
1	1	0	**1**	**1**
1	1	1	**0**	**1**

Maxterm Boolean Expression

- Boolean functions can also be expressed with maxterms, e.g.
- $f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz$
- $f_1 = (x'y'z' + x'yz' + x'yz + xy'z + xyz)'$

  $= (x+y+z)\cdot(x+y'+z)\cdot(x+y'+z')\cdot(x'+y+z')\cdot(x'+y'+z')$

  $= M_0 \bullet M_2 \bullet M_3 \bullet M_5 \bullet M_7$

  $= \Pi M(0, 2, 3, 5, 7)$
  - capital 'M' for maxterm
  - Π for product (i.e. AND)
- $f_2 = M_0 \bullet M_1 \bullet M_3 \bullet M_5$

  $= \Pi M(0, 1, 3, 5)$

x	y	z	f_1	f_2	f_1'
0	0	0	0	0	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	0	0	1
1	1	0	1	1	0
1	1	1	0	1	1

Canonical Sum and Canonical Product

- The ***canonical sum*** of a logic function is a sum of the minterms corresponding to truth-table rows (input combinations) for which the function produces a '**1**' output.
- e.g. $F(X,Y,Z) = \Sigma m(0, 3, 4, 6, 7)$
  $= X'Y'Z' + X'YZ + XY'Z' + XYZ' + XYZ$
- The ***canonical product*** of a logic function is a product of the maxterms corresponding to input combinations for which the function produces a '**0**' output.
  e.g. $F(X,Y,Z) = \Pi M(1, 2, 5)$
  $= (X + Y + Z')(X + Y' + Z)(X' + Y + Z')$

X	Y	Z	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Express Boolean Functions in Minterms

- If product terms in a Boolean function are not minterms, they can be converted to minterms.
- e.g. f(a,b,c) = a’ + bc’ + ab’c
- Function f has three variables, therefore, each minterm must have all the three variables.
- Neither **a’** nor **bc’** are minterms.
- They can be converted to minterms.
- **ab’c** is a minterm.

Conversion to Minterms

- e.g. f(a,b,c) = a’ + bc’ + ab’c
- To convert a’ to a minterm, the 2 variables (b, c) must be added, without changing its functionality
- Since a’ = a’•1 & 1 = b+b’, a’ = a’(b+b’) = a’b + a’b’
- Similarly, a’b = a’b(c+c’) = a’bc + a’bc’

  and a’b’ = a’b’(c+c’) = a’b’c + a’b’c’
- bc’ = bc’(a+a’) = abc’ + a’bc’
- f = a’bc + a’bc’ + a’b’c + a’b’c’ + abc’ + a’bc’ + ab’c

  = a’bc + a’bc’ + a’b’c + a’b’c’+ abc’ + ab’c

Express Boolean Functions in Maxterms

- Using the Distribution Law:

  x + y·z = (x + y)·(x + z), a Boolean function can be converted to an expression in product of maxterms. e.g.,

- f(a, b, c) = a' + bc'

  = (a' + b)(a' + c') {not maxterms}

  = (a' + b + cc')(a' + c' + bb') {cc' = 0}

  = (a'+b+c)(a'+b+c')(a'+c'+b)(a'+c'+b')

  = (a'+b+c)(a'+b+c')(a'+c'+b')

Boolean Function Manipulation

- Boolean functions can be manipulated with Boolean algebra.
- Manipulation can transform logic expressions, but still keep the same logic functionality.
- Manipulation can reduce the complexity, hence, easier to be implemented in hardware, i.e. fewer logic gates.

Boolean Function Manipulation Example

- $f = x \cdot y' + x \cdot y \cdot z + x' \cdot z$

  $= x \cdot (y' + y \cdot z) + x' \cdot z$ {common factor}

  $= x\,[(y' + y) \cdot (y' + z)] + x' \cdot z$ {Distribution law}

  $= x\,(y' + z) + x' \cdot z$ {$y' + y = 1$}

  $= x \cdot y' + x \cdot z + x' \cdot z$ {Distribution law}

  $= x \cdot y' + (x + x') \cdot z$ {common factor}

  $= x \cdot y' + z$ {$x + x' = 1$}

More Examples

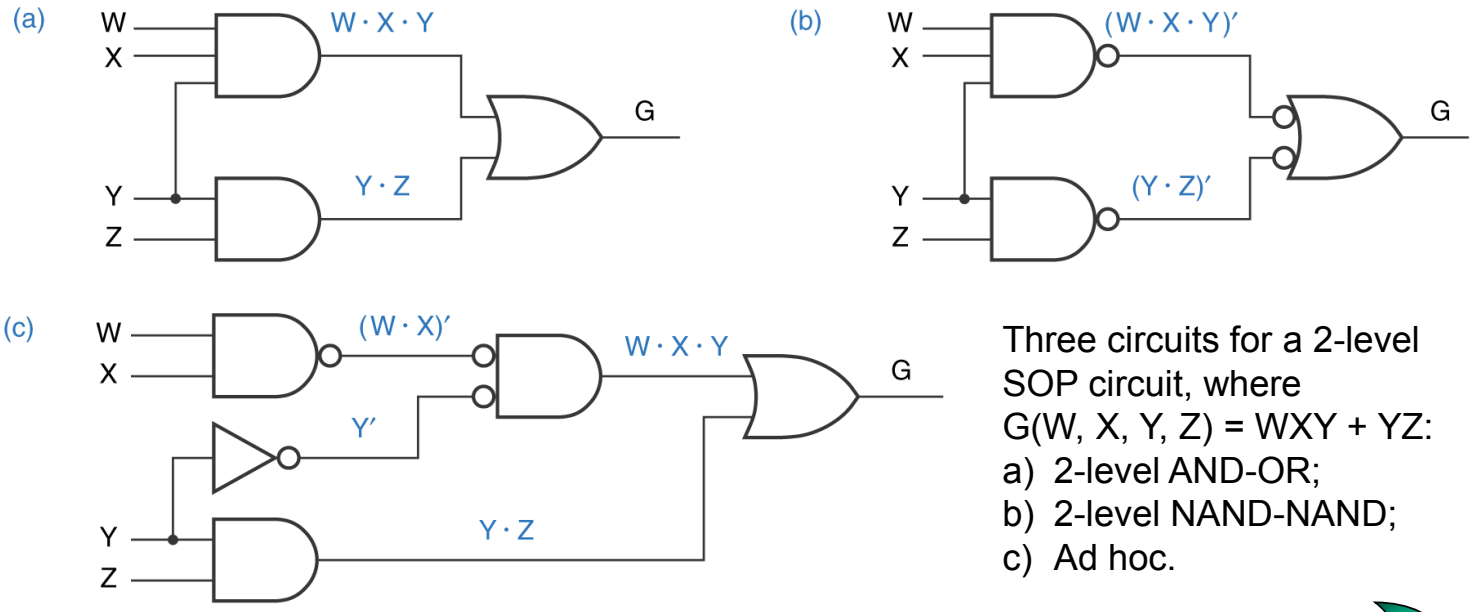
- Simplify f_1= abc + a'b + abc' & f_2 = (a+b)'(a'+b') to the minimum literals

- f_1 = abc + a'b + abc'
  = ab(c+c') + a'b
  = ab + a'b = (a+a')b = b

- f_2 = (a+b)'(a'+b')
  = a'b'(a'+b') {DeMorgan}
  = a'b'a' + a'b'b'
  = a'b' + a'b' = a'b'

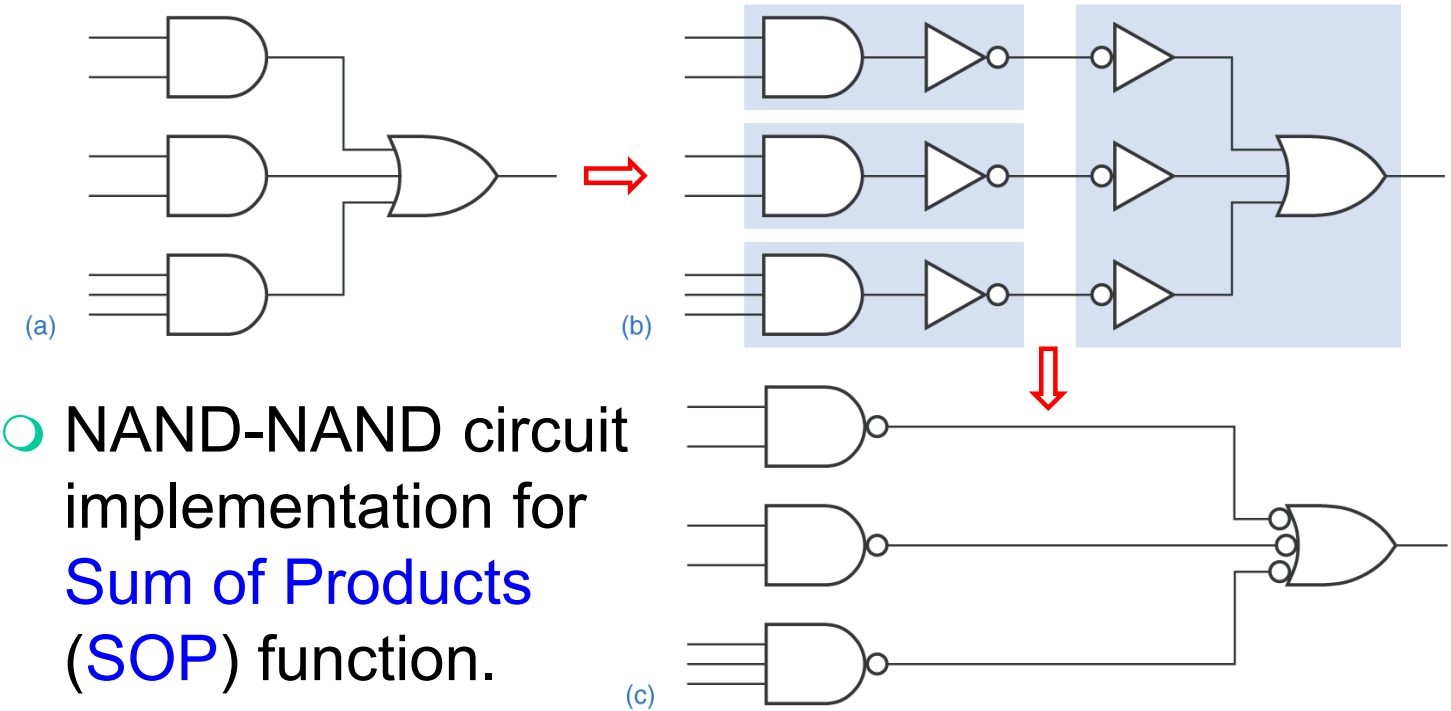
Circuit Manipulations

- The design method that we have described so far use AND, OR, and NOT gates.
- We might like to use **NAND** and **NOR gates** too — they **are faster than AND** and **OR gates**.



Three circuits for a 2-level SOP circuit, where G(W, X, Y, Z) = WXY + YZ:
a) 2-level AND-OR;
b) 2-level NAND-NAND;
c) Ad hoc.

SOP Using NAND Gates



- NAND-NAND circuit implementation for Sum of Products (SOP) function.
- 2-level NAND circuit.

Figure 4-21

Alternative sum-of-products realizations: (a) AND-OR; (b) AND-OR with extra inverter pairs; (c) NAND-NAND.

POS Using NOR Gates

- NOR-NOR circuit implementation for Product of Sums (POS) function.
- 2-level NOR circuit.

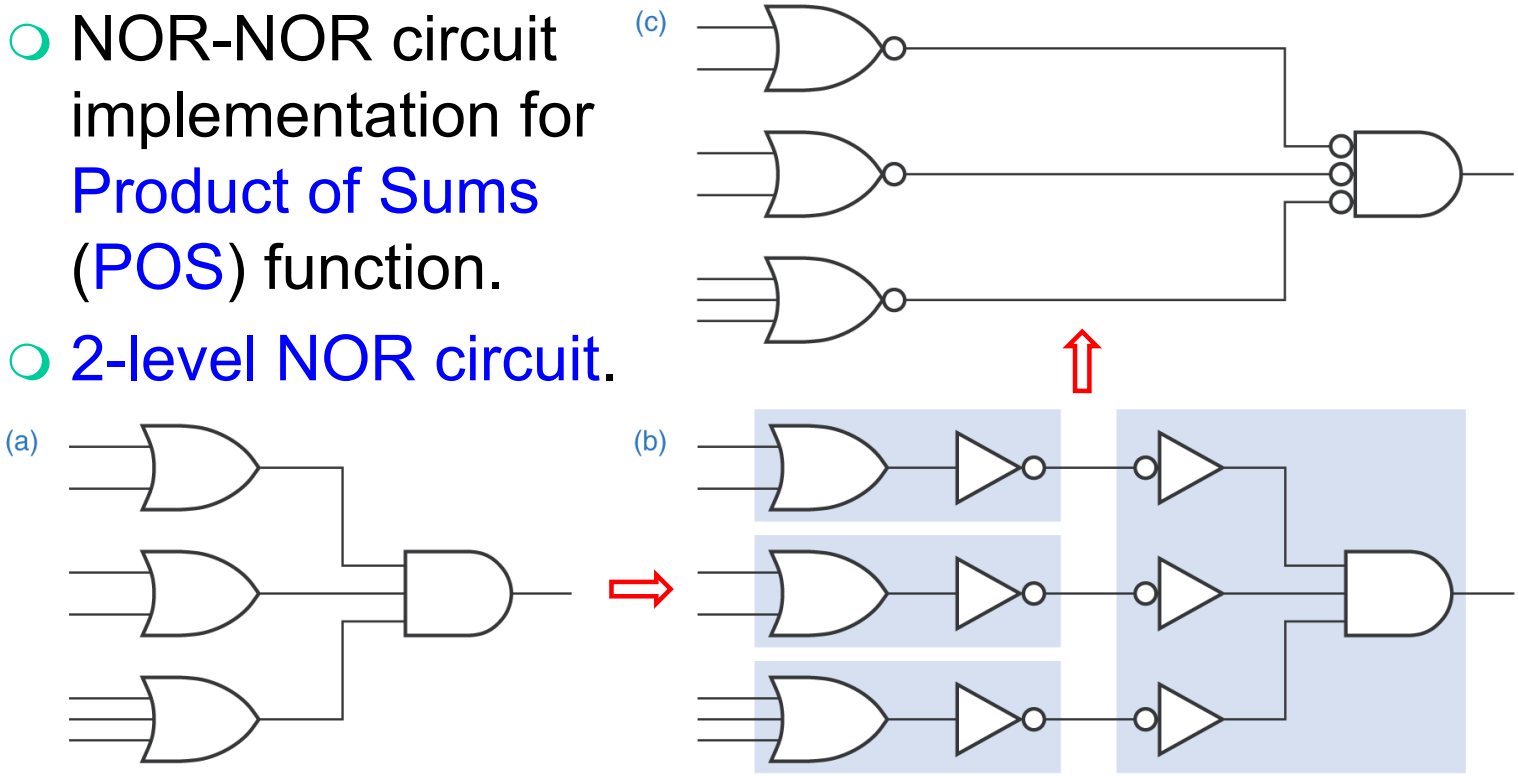


Figure 4-23

Realizations of a product-of-sums expression: (a) OR-AND; (b) OR-AND with extra inverter pairs; (c) NOR-NOR.

What do we know by now?

- By now, we should know:
  - What Boolean algebra is
  - Switching Algebra Postulates and Theorems
  - How to manipulate Boolean functions using Boolean laws
  - Truth table
  - Literals, Minterms and Maxterms
  - Sum-of-products and product-of-sum
  - How to derive the logic function from truth table
  - How to express a logic function in truth table, minterm expression or maxterm expression
  - Logic circuit manipulation

Week 4 End

Karnaugh Maps

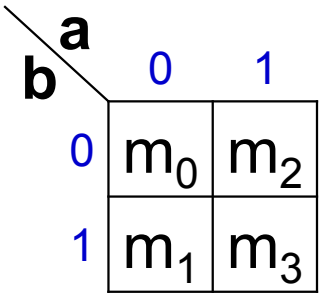
- By the end of this topic, we will know:
  - How to create Karnaugh maps (K-Map) from logic expressions or truth tables for logic functions with 2 to 4 variables
  - How to perform logic minimization with K-Map and derive minimized logic expressions from K-Map
  - What implicants, prime/essential prime Implicants are
  - How to derive sum-of-products/product-of-sums expressions from K-Map
  - What odd and even functions are
  - How to use DON'T-CARE Conditions in K-Map

Introduction to Karnaugh Map

- Karnaugh Map, invented by Maurice Karnaugh of Bell Labs in 1953, also known as K-map, is a diagrammatic method for logic minimization.
- K-map is usually more efficient than Boolean algebra.
- K-map is a diagram made up of squares.
- Each square represents exactly one minterm of the logic function.

2-variable K-map

- For a 2-variable function, there are 4 minterms. Therefore, the K-map for a 2-variable function has 4 squares:

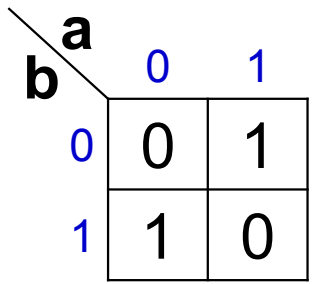


m_0 = a'b' (00)
m_1 = a'b (01)
m_2 = ab' (10)
m_3 = ab (11)

- Each square has a value either 0 or 1 depending on the value of the function at that square.

Construction of 2-variable K-map

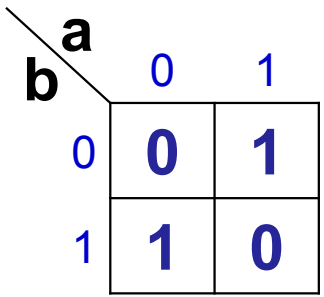
- To construct a K-map for a 2-variable function, a logic 1 is entered into the square where the corresponding minterm exists. A logic 0 is entered otherwise (or the square is left blank).
- e.g. **f = a'b + ab'**
- f is true when ab = 01 or 10
- f = Σm(1, 2)



Construction of K-map from Truth Table

- A K-map can be created directly from a truth table
- Each square of the K-map corresponds to 1 row of the truth table
- A logic 1 is entered when the function is 1
- A logic 0 is entered when the function is 0
- e.g.

a	b	f
0	0	0
0	1	1
1	0	1
1	1	0



3-variable K-map

- A 3-variable logic function has 8 minterms and its truth table has 8 rows
- Hence, a 3-variable K-map has 8 squares

c \ ab	00	01	11	10
0	m_0	m_2	m_6	m_4
1	m_1	m_3	m_7	m_5

- Note that the binary sequence of ab follows **Gray code** order, i.e. 00, 01, 11, 10.
- This is to enable logic minimization as can be seen in the next slide.

Logic Minimization with K-Map

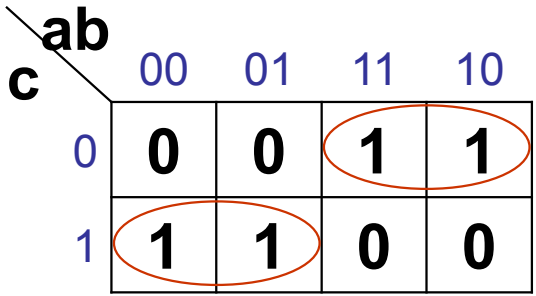
- Consider a logic function with m_2 and m_6, i.e. $f = \Sigma m(2, 6)$
- m_2 is a'bc' and m_6 is abc'
- $f = m_2 + m_6 = a'bc' + abc' = bc'(a' + a) = bc'$
- The 2 minterms have a *common factor* bc'
- In the K-map, if we group these 2 adjacent minterms, we can reduce 1 variable
- Gray code must be used to ensure adjacent squares have only 1 variable changed value (to have a common factor, bc' in this case)

c \ ab	00	01	11	10
0	0	1	1	0
1	0	0	0	0

Logic Minimization for 3-variable K-map

- e.g.

a	b	c	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

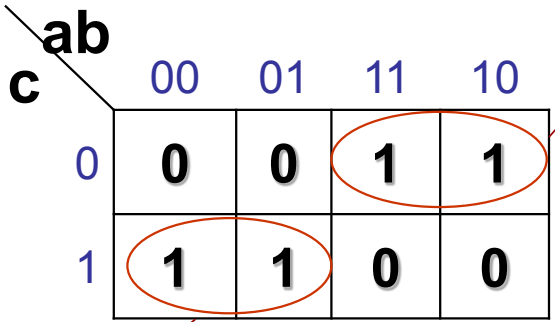


From the truth table:

$$f = m_1 + m_3 + m_4 + m_6$$
$$= a'b'c + a'bc + ab'c' + abc'$$
$$= a'c(b'+b) + ac'(b'+b)$$
$$= a'c + ac'$$

- From the map, we can group 2 adjacent logic 1s
- Each group has a common factor and gives one simplified product term

3-variable K-map for Logic Simplification



ab'c' + abc' = ac'(b'+b) = ac'

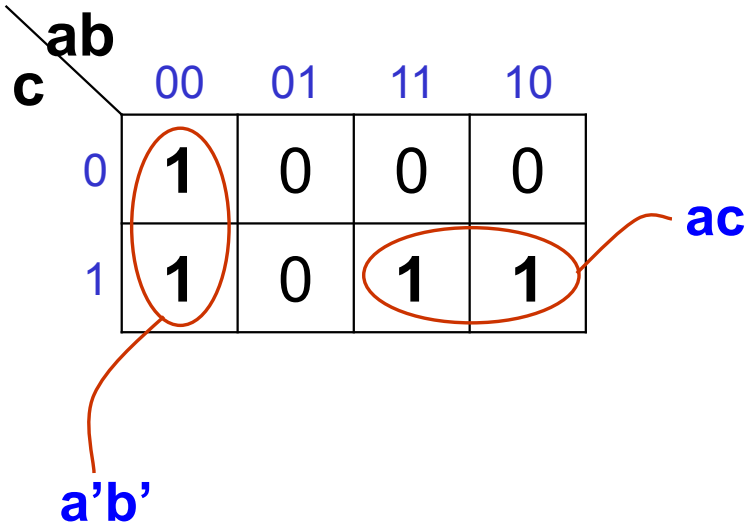
For these 2 squares,
a = 1 & **c** = 0,
b changes from 1 to 0,
so **b** is eliminated.

a'b'c + a'bc = a'c(b'+b) = a'c

For these 2 squares, **a** = 0 and **c** = 1,
b changes from 0 to 1, so **b** is eliminated

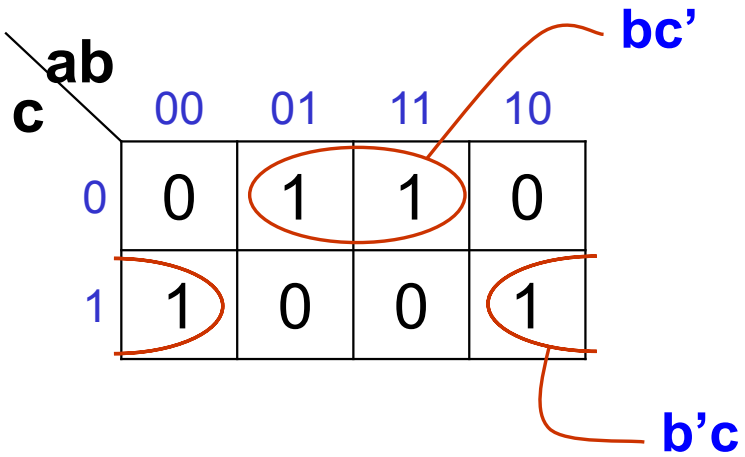
∴ f = a'c + ac'

3-variable K-map Example 1



$$\therefore f = a'b' + ac$$

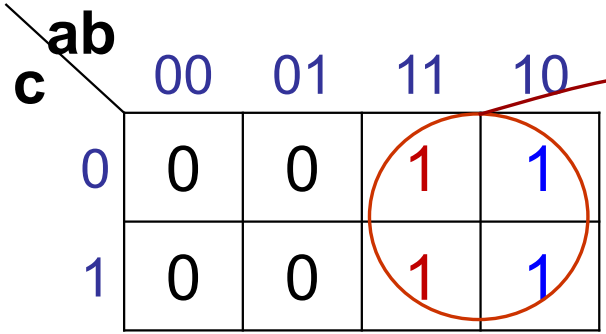
3-variable K-map Example 2



$\therefore f = b'c + bc'$

- The first column and the last column are considered as adjacent to each other as only 1 variable changes its value

3-variable K-map Example 3



abc' + abc + ab'c' + ab'c
= ab + ab' = a

a = 1 in all 4 squares.
Both **b** & **c** change values.

- 4 adjacent squares can be grouped
- In K-map, the **number of adjacent squares that can be grouped is 2^n**, where **n** = 0, 1, 2, …
- Hence, we can group 1, 2, 4, 8 squares
- But not 3, 5, 6, 7 squares

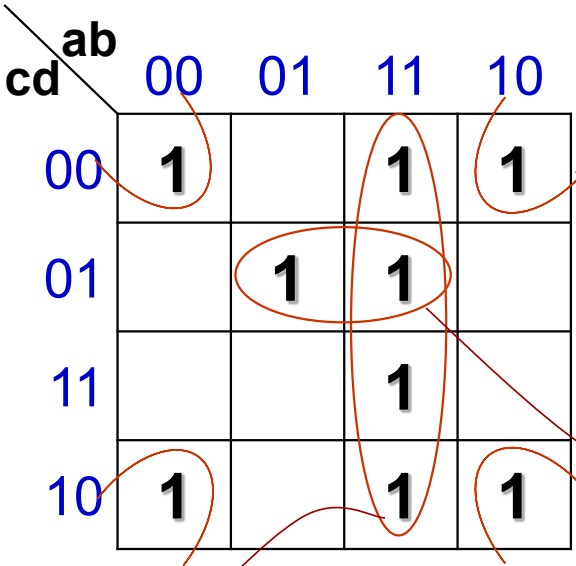
4-variable K-map

- A **4-variable** K-map has **16 squares**

cd \ ab	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

- Note that **the order is in Gray Code** and the **minterms are not in sequence**

4-variable K-map Example



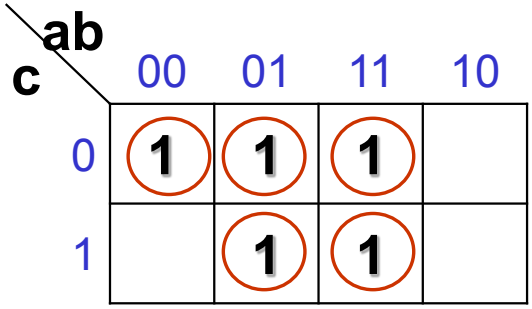
a'b'c'd' + a'b'cd'+ ab'c'd' + ab'cd'
= a'b'd'(c'+c) + ab'd'(c'+c)
= a'b'd' + ab'd'
= b'd'(a'+a) = **b'd'**

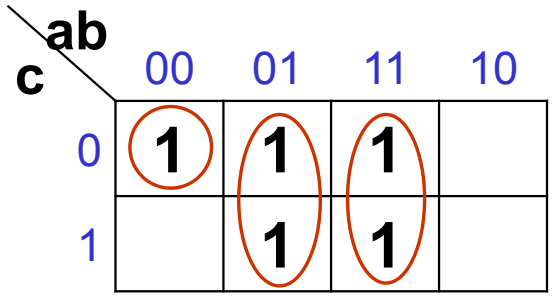
a'bc'd + abc'd
= bc'd(a'+a) = **bc'd**

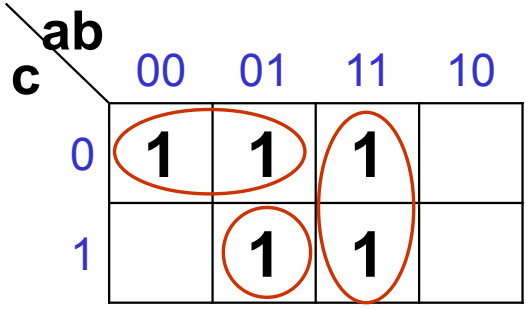
abc'd' + abc'd + abcd + abcd'
= abc'(d'+d) + abc(d'+d)
= abc' + abc = ab(c'+c) = **ab**

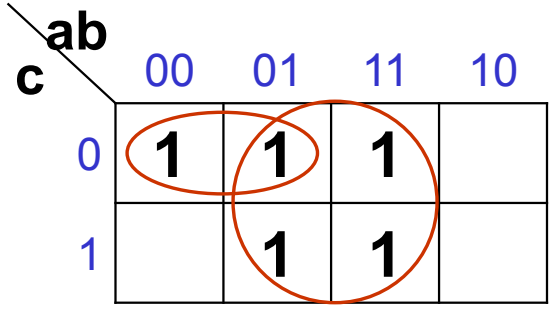
K-map Covering

- In K-map, a function is said to have been **covered if all logic-1 squares are grouped**
- We can cover a K-map with **groups in many different ways**, e.g.



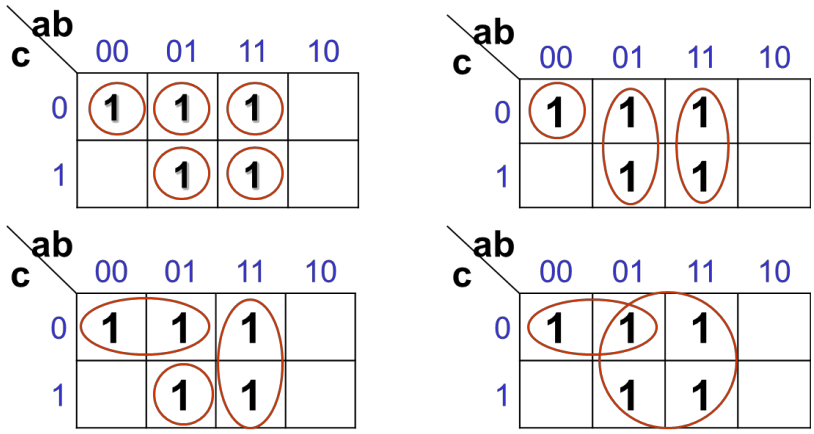






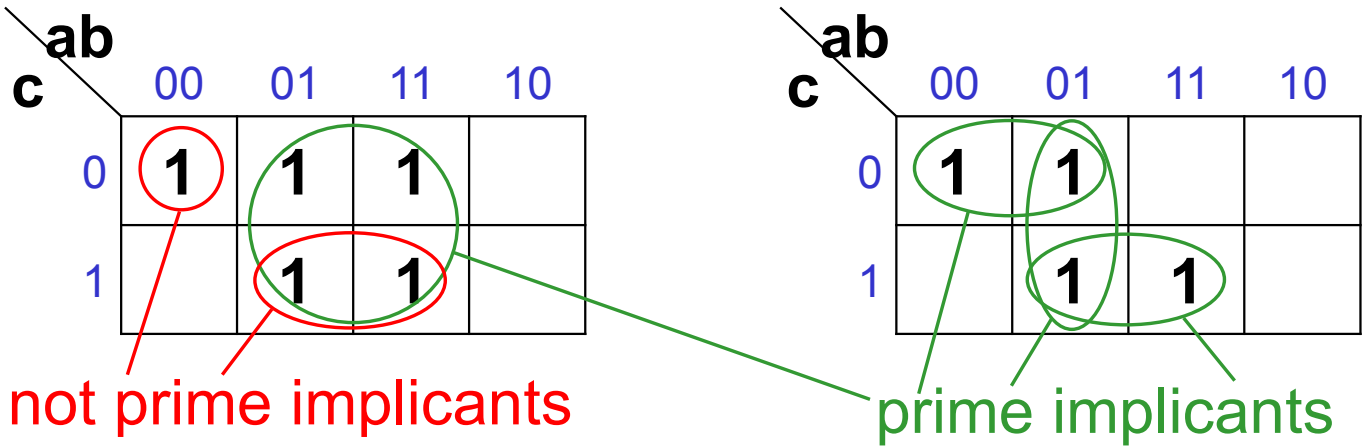
Implicants

- In the previous example, the same K-map is covered in 4 different ways
- Each group gives a product term and is called an *implicant* of the function
- Each implicant is a product term of the function
- Hence, a function can be expressed in many ways

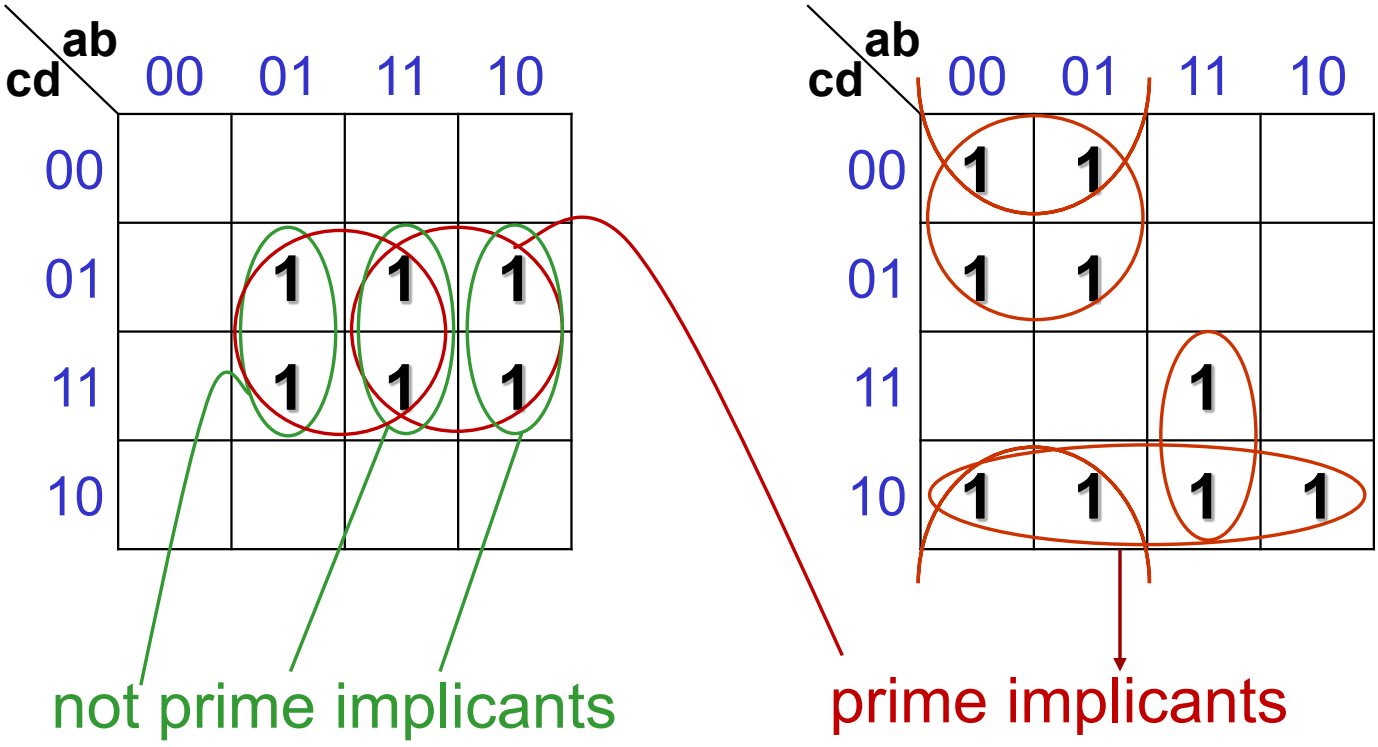


Prime Implicants

- A Prime Implicant is an implicant that cannot be combined with other implicants.
- In K-map, a prime implicant is a group that covers the maximum possible number of adjacent squares. e.g.



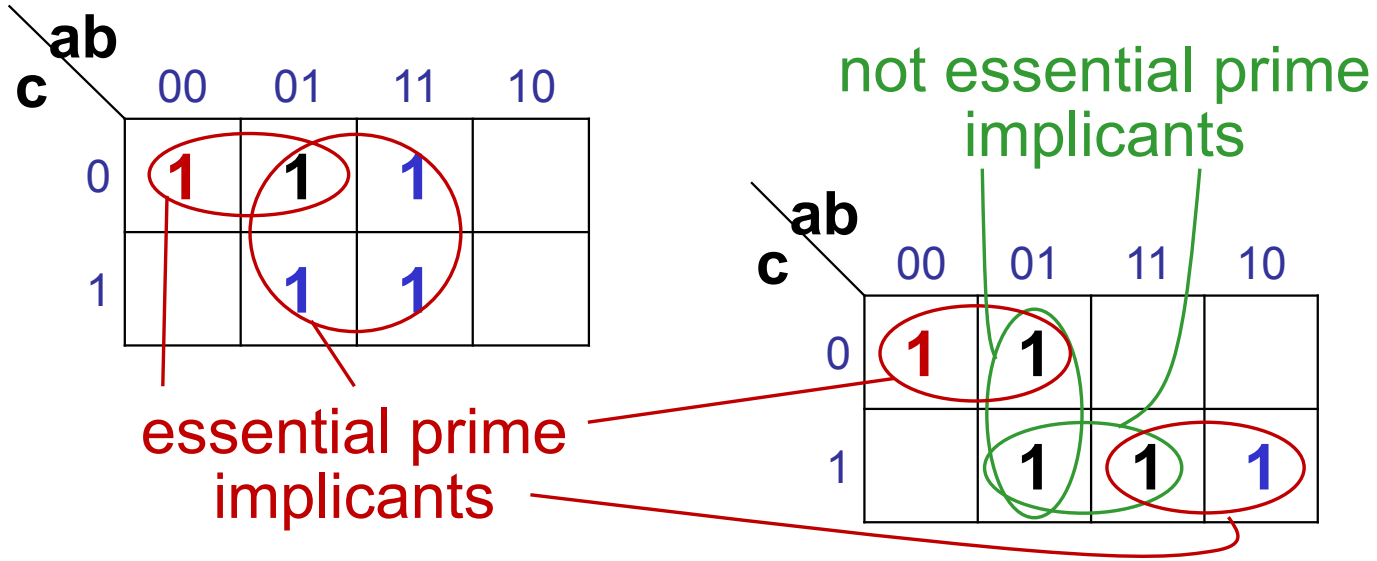
Prime Implicant Examples



- Noted that a prime implicant may have 1, 2, 4, 8 or even 16 minterms in the group.

Essential Prime Implicant

- An Essential Prime Implicant is a prime implicant that covers a minterm which is not covered by any other prime implicants. e.g.



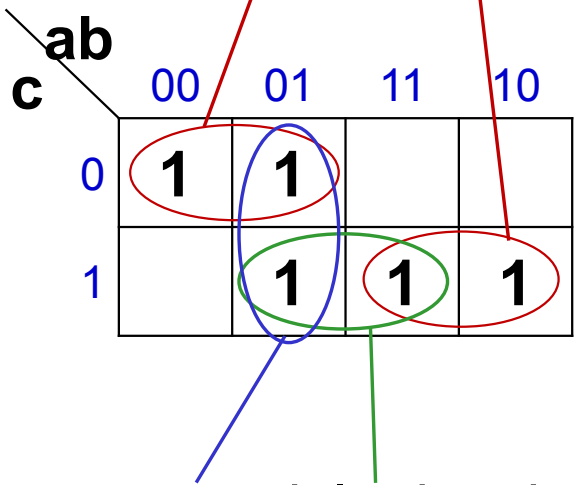
Minimized Logic Functions

- A minimized logic function is a logic function consisting of **ALL** essential prime implicants, plus other prime implicants that may be needed to cover any remaining minterms which are not covered by the essential prime implicants

- Sometimes, there may be different combinations of prime implicants to give different but equivalent minimized logic functions

Minimized Logic Function Example

essential prime implicants must be present



f = a'c' + ac + a'b

or

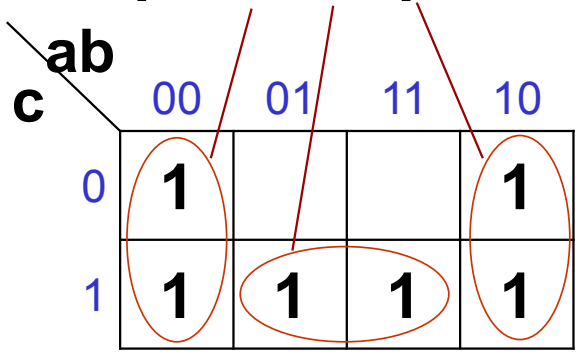
f = a'c' + ac + bc

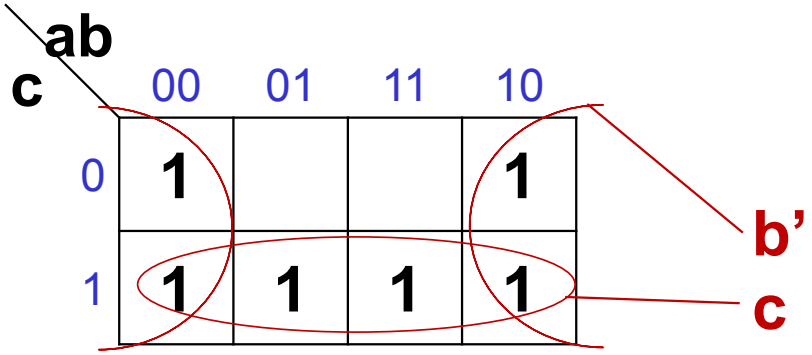
not essential prime implicants,
only 1 of them is needed to
cover the function completely

K-map Example for 3-variable Function

a	b	c	f
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

wrong, they are not prime implicants



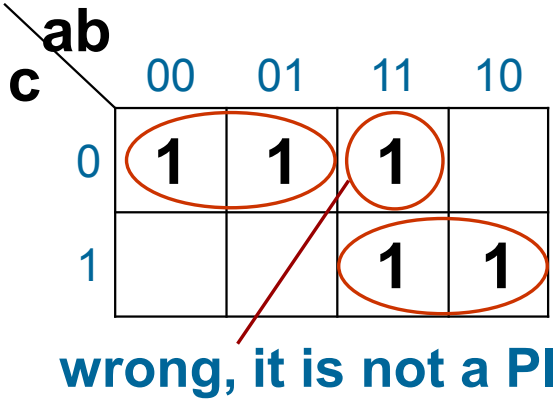


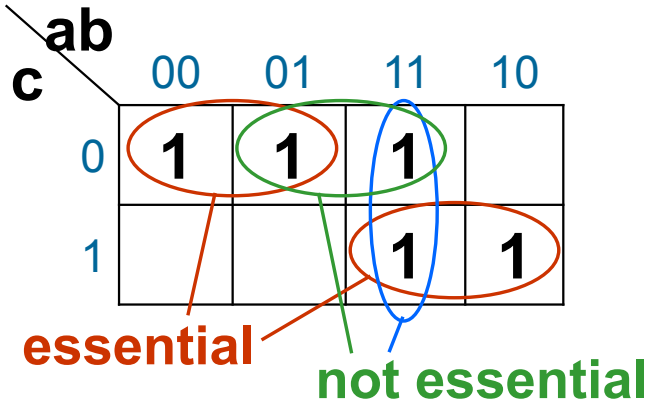
essential prime implicants

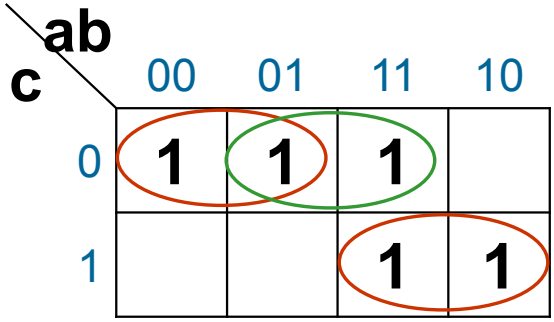
$\Rightarrow f = b' + c$

K-map Example 2 for 3-variable Function

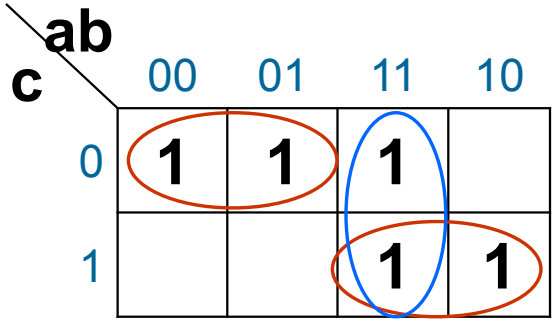
- e.g. f(a,b,c) = Σm(0, 2, 5, 6, 7)





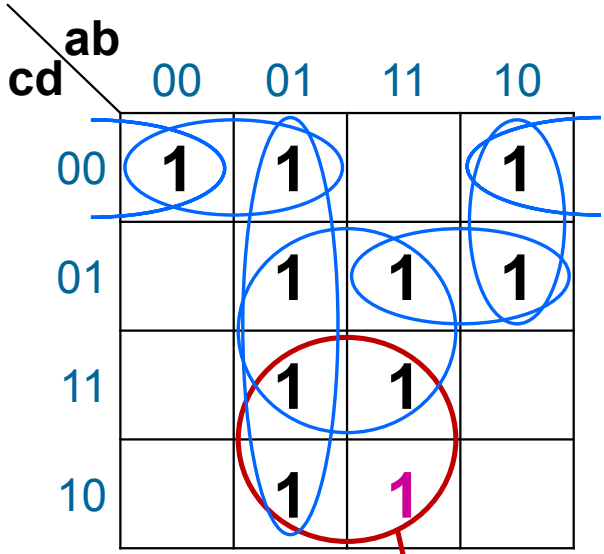


f = a'c' + bc' + ac



f = a'c' + ab + ac

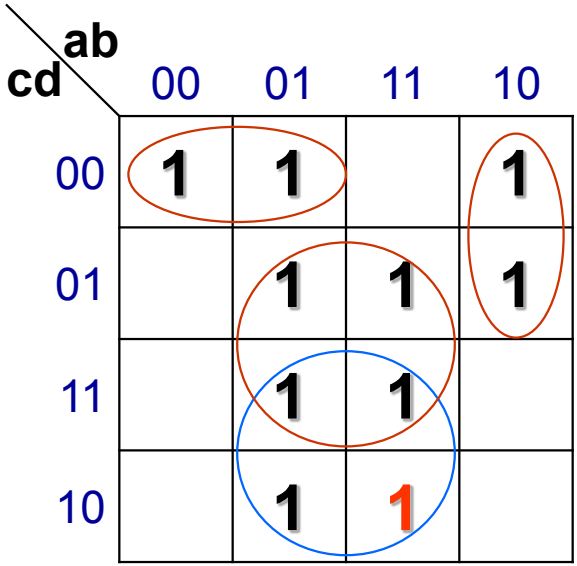
K-map Example for 4-variable Function

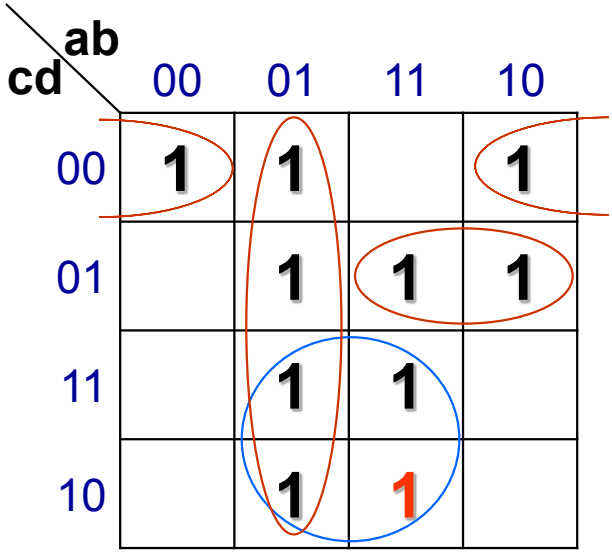


essential prime implicant

all the others are not

K-map Example for 4-variable Function



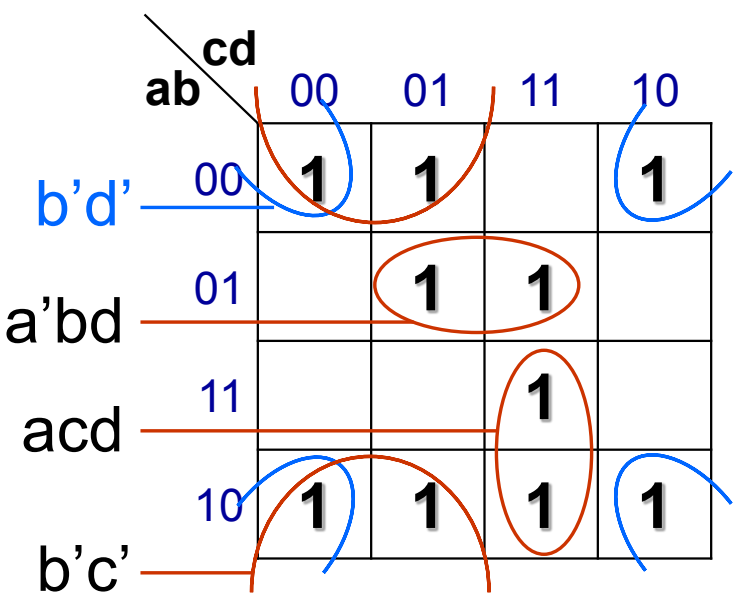


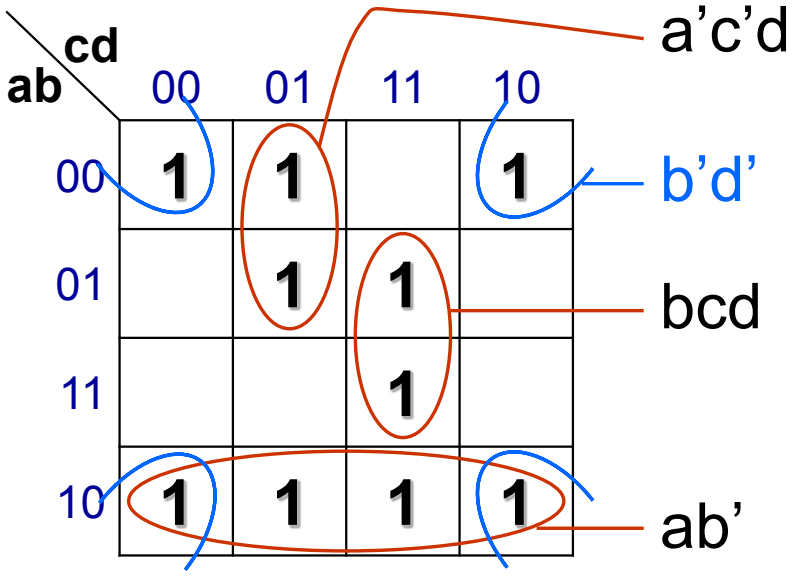
f = a'c'd' + bd + bc + ab'c'

f = b'c'd' + a'b + bc + ac'd

Both require a minimum of 4 prime implicants and contain the essential prime implicant. Both functions are minimized. They are equivalent.

K-map Example 2 for 4-variable Function





f = a'bd + b'd' + acd + b'c' f = a'c'd + b'd' + bcd + ab'

Rules to Cover K-map

1. Cover all logic 1 squares with minimum number of groups
2. Group only adjacent 1, 2, 4, 8 or 16 squares
3. Make a group as large as possible (To get the Prime Implicant)

K-map for Product of Sums

- Covering logic-1 squares in K-map gives us logic functions in sum-of-products form.
- We can also cover logic-0 squares, which will give us logic functions in product-of-sums form.

e.g.

Map for f

c \ ab	00	01	11	10
0	1	0	1	1
1	1	0	0	1

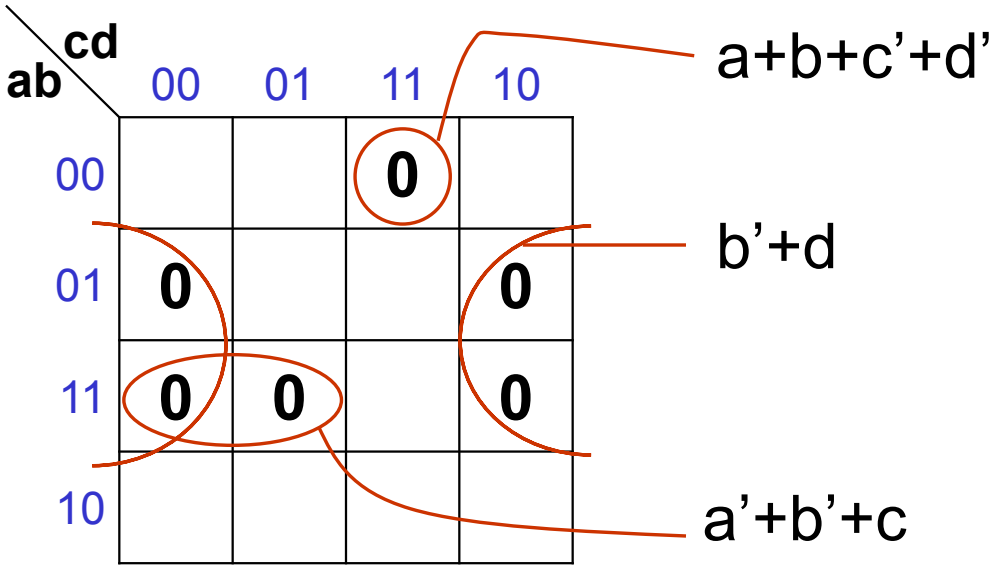
f = (a+b'+c)(a+b'+c')
(a+b'+c')(a'+b'+c')
= (a+b')(b'+c')

Map for f'

c \ ab	00	01	11	10
0	0	1	0	0
1	0	1	1	0

f' = a'b + bc
⇒ f = (a+b')(b'+c')

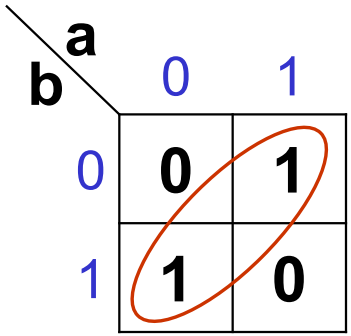
K-map for Product of Sums Example



3 groups give:

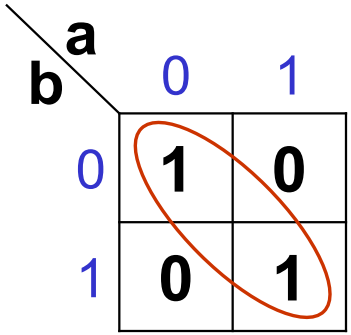
f = (a+b+c'+d')(b'+d)(a'+b'+c)

2-variable K-map for XOR and XNOR



$ab' + a'b = a \oplus b$

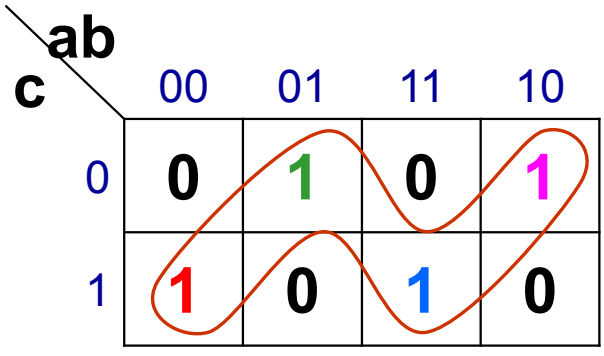
(m_1, m_2)



$a'b' + ab = (a \oplus b)'$

(m_0, m_3)

3-variable K-map for XOR and XNOR

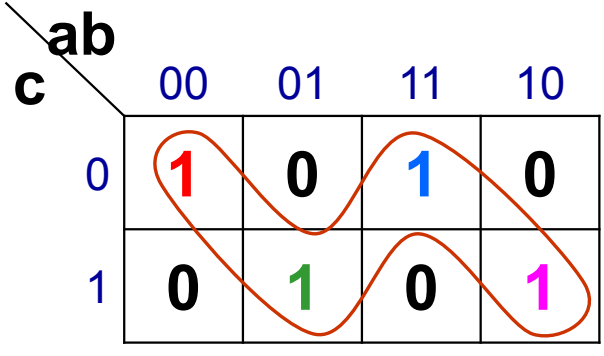


(m_1, m_7, m_2, m_4)

$a'b'c + abc + a'bc' + ab'c'$

$= (a \oplus b)'c + (a \oplus b)c'$

$= a \oplus b \oplus c$



(m_0, m_3, m_6, m_5)

$a'b'c' + a'bc + abc' + ab'c$

$= a'(b \oplus c)' + a(b \oplus c)$

$= (a \oplus b \oplus c)'$

Odd and Even Function

- For a 3-variable function f = a ⊕ b ⊕ c, f = 1 if only 1 variable is logic 1 or if 3 variables are logic 1
- So XOR is also called an ODD function because 1 and 3 are odd numbers
- In general, for an n-variable odd function, the function is 1 if there are odd number of variables having logic 1
- XNOR is called an EVEN function.

DON'T-CARE Conditions

- In logic function, sometimes we do not have the specification for all the combinations
- We might define a logic function to be 1 for some combinations and 0 for some others but the rest is not defined
- We do not care about the logic value of the function for these undefined combinations
- We call these combinations DON'T-CARE conditions
- DON'T-CARE conditions are usually denoted by 'x', or 'X' or 'd'

Truth Table with DON'T-CARE Conditions

a	b	c	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	x
1	0	1	1
1	1	0	x
1	1	1	0

- f has unknown (or don't care) values for the combinations:
  - abc = 100
  - abc = 110
- Usually expressed as:

  $f(a,b,c) = \Sigma m(1, 2, 5) + d(4, 6)$

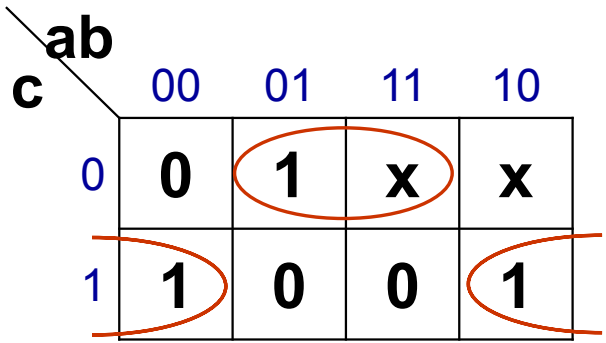
K-map with DON'T-CARE Conditions

- When constructing a K-map for a logic function with don't-care conditions, we enter 'x' into the squares where the function is undefined
- e.g. **f(a,b,c) = Σm(1, 2, 5) + d(4, 6)**

c \ ab	00	01	11	10
0	0	1	x	x
1	1	0	0	1

Logic Minimization with DON'T-CAREs

- When a K-map contains don't-care conditions, we can treat the don't-cares as either 1 or 0.
- We make use of x = 1 when we can make the group larger and x = 0 otherwise.
- e.g.



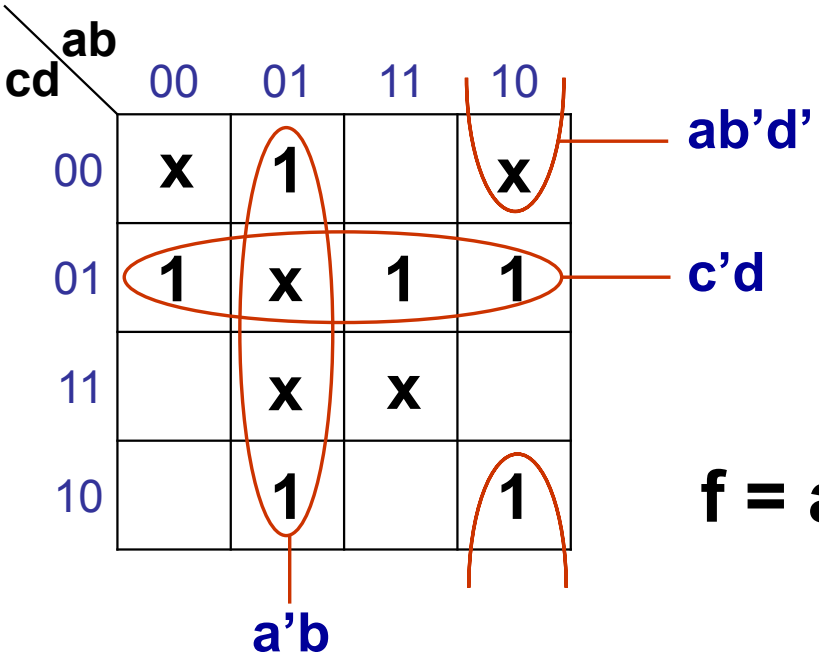
x in 100 is 0

x in 110 is 1

f = bc' + b'c

- We don't group x when it is treated as 0.

K-map with DON'T-CARE Example



$f = a'b + c'd + ab'd'$

- **Rule of Thumb:**

  Don't care about the DON'T-CAREs first

Solving Logic Problem

- Let a_1a_0 and b_1b_0 be two 2-bit binary numbers.
- Derive a truth table for a logic function representing that these two numbers are equal.

a_1	a_0	b_1	b_0	f_E
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	

a_1	a_0	b_1	b_0	f_E
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

Other Logic Minimization Techniques

- **Variable-Entered Map (VEM):**

  David J. Comer,
  Digital Logic and State Machine Design
  3rd Edition, Oxford University Press Inc, USA,1994

- **Quine-McCluskey Method:**

  Charles H. Roth, Jr. and Larry L. Kinney
  Fundamentals of Logic Design
  6th Edition, Cengage Learning, 2010

What do we know by now?

- By now, we should know:
  - How to create Karnaugh maps (K-Map) from logic expressions or truth tables for logic functions with 2 to 4 variables
  - How to do logic minimization with K-Map and derive minimized logic expressions from K-Map
  - What implicants, prime/essential prime implicants are
  - How to derive sum-of-products/product-of-sums expressions from K-Map
  - What odd and even function are
  - How to use DON'T-CARE conditions in K-Map

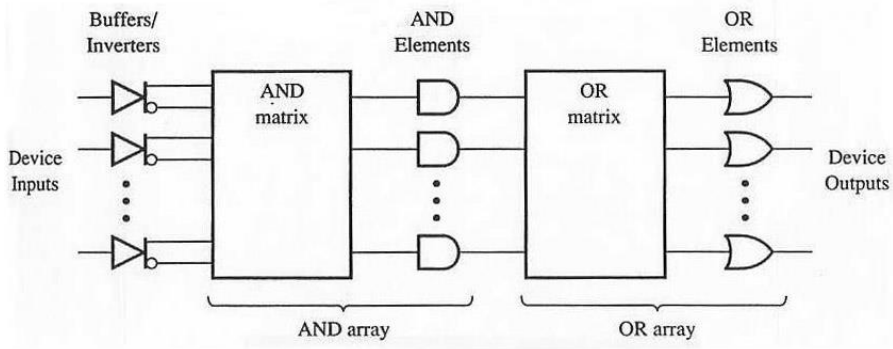
Topic: Programmable Logic Devices

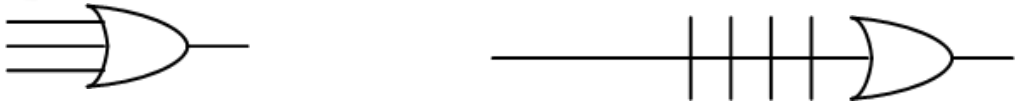
- By the end of this topic, we will know:
  - What Programmable Array Logic (**PAL**) and Programmable Logic Array (**PLA**) are
  - How to recognize their structures
  - How to use them to implement logic functions

Programmable Logic Devices (PLDs)

- PLDs consist of AND elements and OR elements
- Interconnections are programmable
- Sum of Products (SOP) of logic functions can be implemented
- Three common PLDs:
  - Programmable Array Logic (PAL)
  - Programmable Logic Array (PLA)
  - Programmable Read Only Memory (PROM)
- The basic differences are whether the AND & OR arrays are FIXED or PROGRAMMABLE
- Buffers / Inverters at the inputs to provide uncomplemented/complemented signals

Basic structure of PAL and PLA



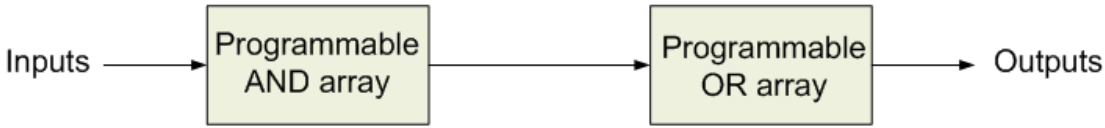


Conventional Symbol

Array Logic Symbol



(a) Programmable array logic (PAL)



(b) Programmable logic array (PLA)

Programmable Array Logic (PAL)

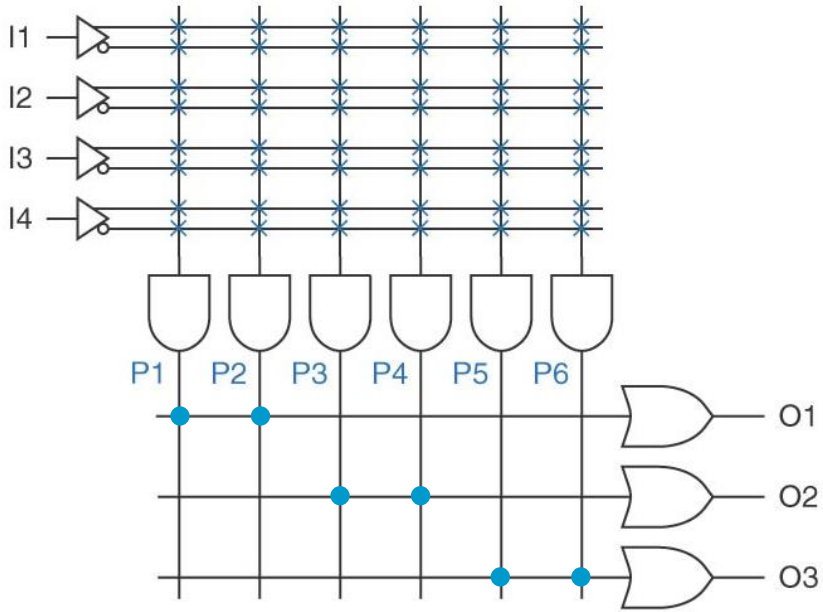
- It has a programmable AND array & a fixed OR array
- The AND plane has 2n programmable connections where n is the number of inputs
- The number of product terms for each OR gate output is fixed during the manufacturing process and cannot be changed by the user
- Different types of PAL have different number of product terms and different number of outputs

A Simple 4-Input 3-Output PAL

- AND plane is programmable: can have any product term
- OR plane is fixed: Number of product terms is fixed
- In this example, each OR gate has 2 inputs: Each output can have only 2 product terms

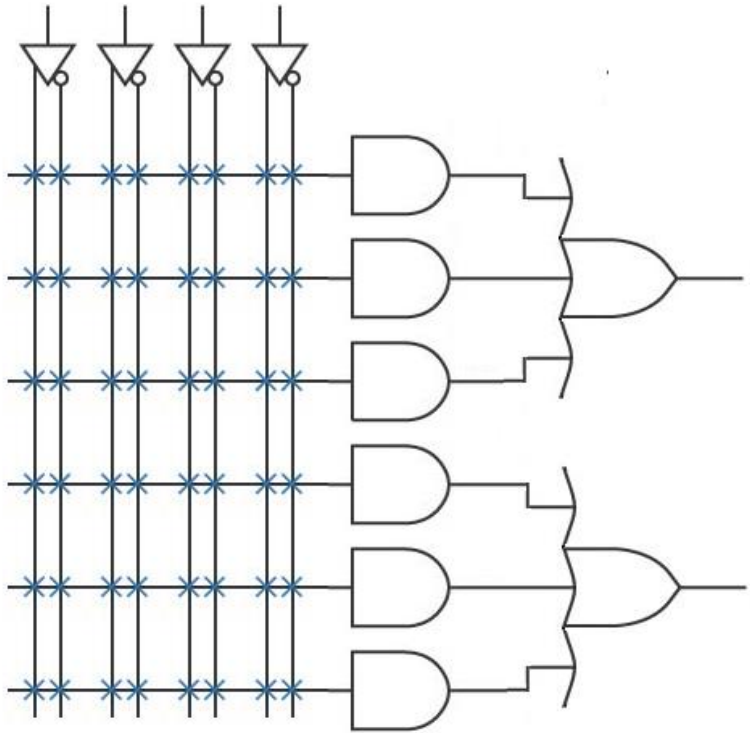
● = Fixed connection

x = Fuse-programmable connection
Blown if not needed



A Different PAL Representation

- 4-input 2-output PAL
- Each OR gate has 3 product terms



Implementing Logic Functions With PAL

- An example of 4 functions:

$F1 = \sum m(1,3,4,5,6,7)$

$F1 = A + C$

A \ BC	00	01	11	10
0	0	1	1	0
1	1	1	1	1

$F2 = \sum m(0,1,4,5,6)$

$F2 = B' + AC'$

A \ BC	00	01	11	10
0	1	1	0	0
1	1	1	0	1

$F3 = \sum m(1,2,5)$

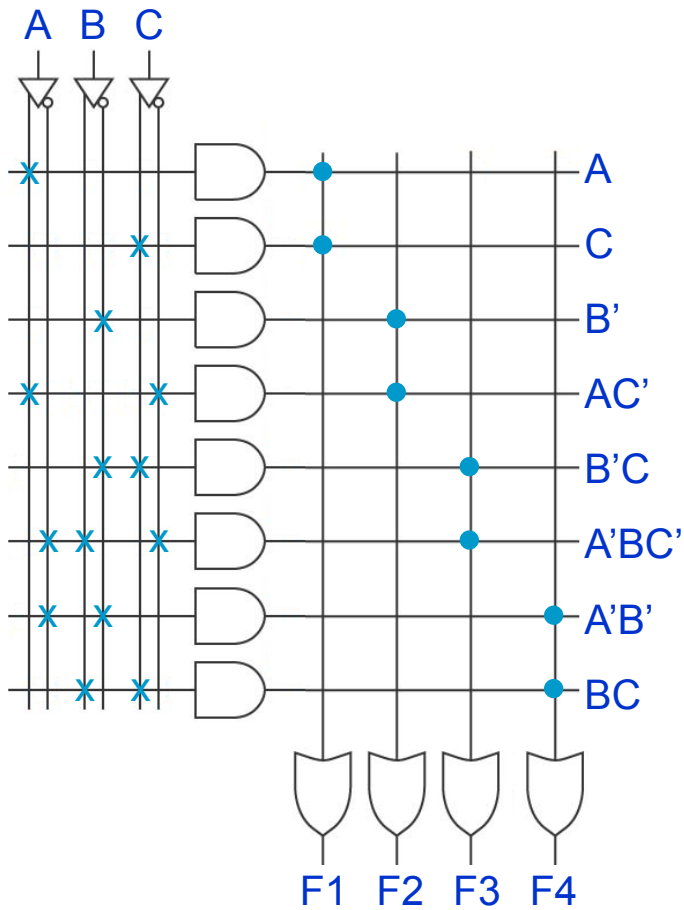
$F3 = B'C + A'BC'$

A \ BC	00	01	11	10
0	0	1	0	1
1	0	1	0	0

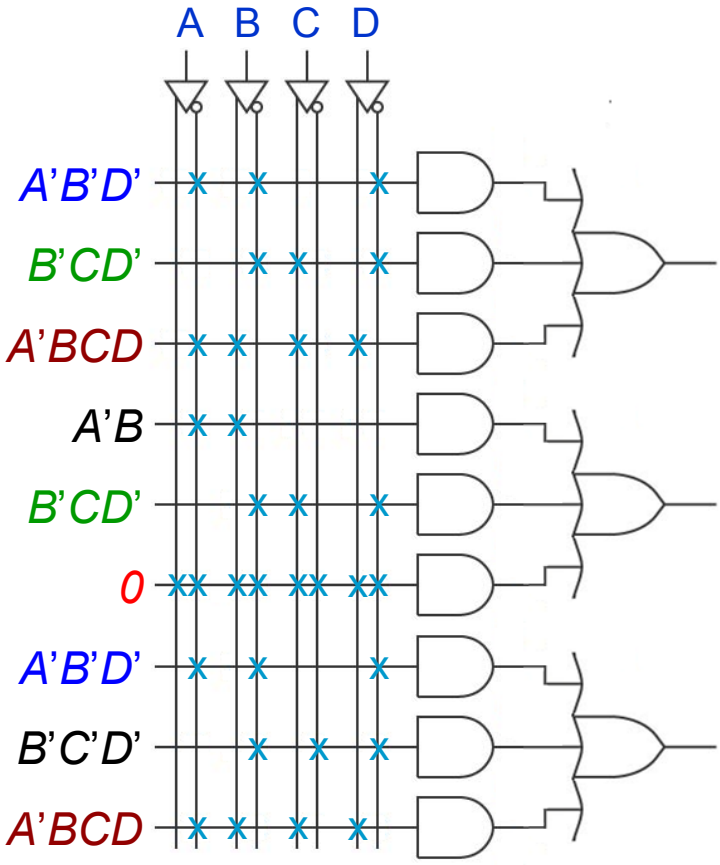
$F4 = \sum m(0,1,3,7)$

$F4 = A'B' + BC$

A \ BC	00	01	11	10
0	1	1	1	0
1	0	0	1	0



Implementing Logic Functions With PAL



$F1 = A'B'D' + B'CD' + A'BCD$

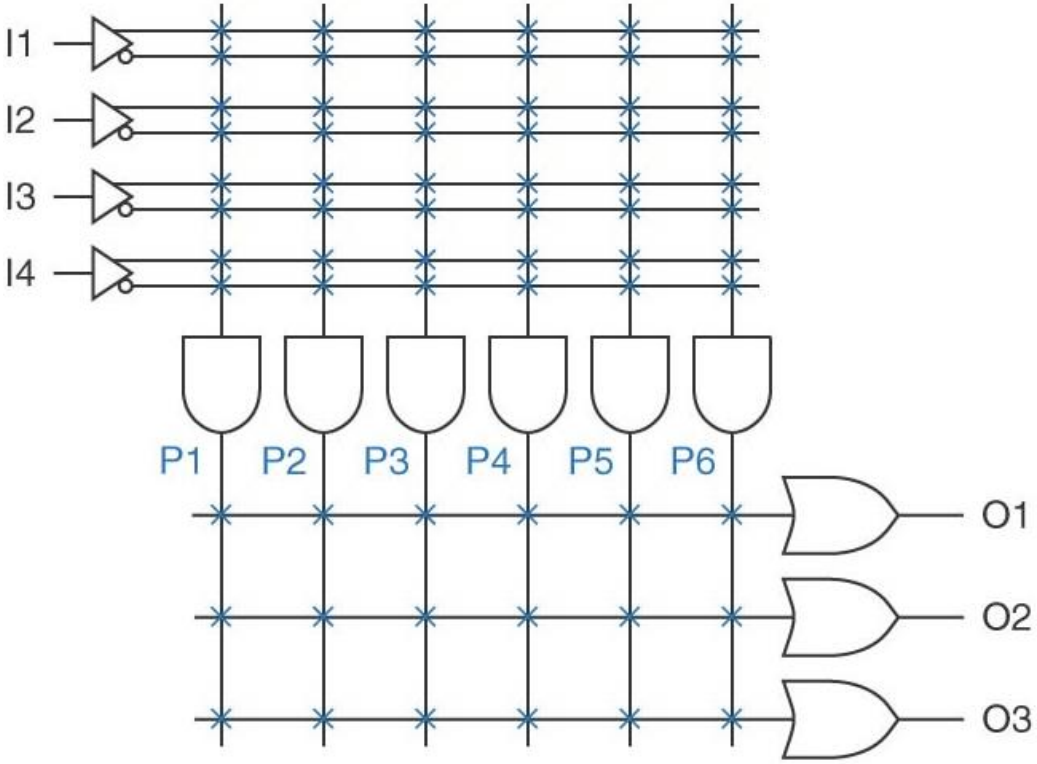
$F2 = A'B + B'CD'$

$F3 = A'B'D' + B'C'D' + A'BCD$

Programmable Logic Array (PLA)

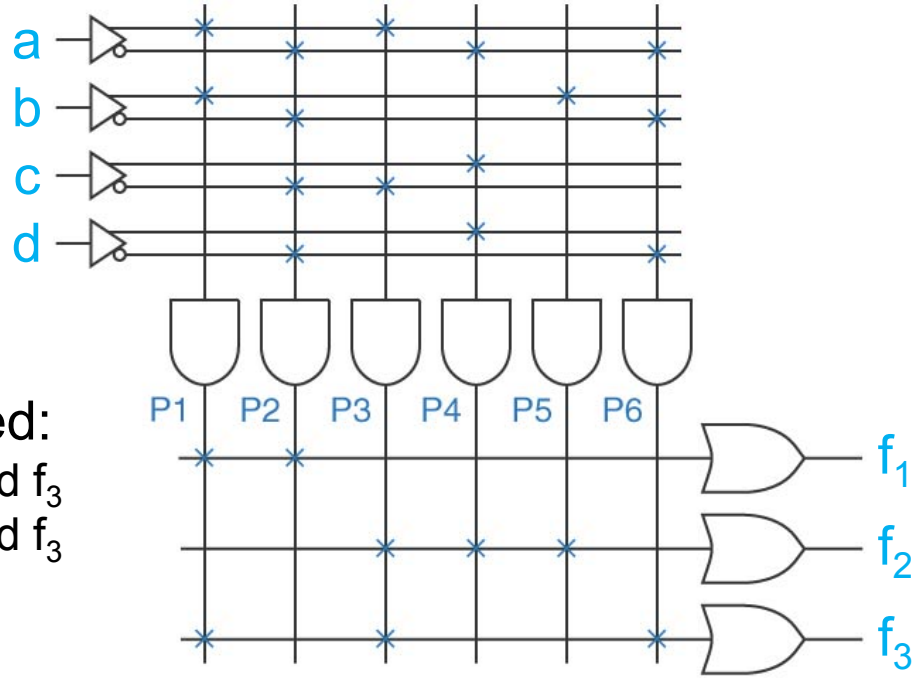
- Both the **AND** array & the **OR** array are programmable
- The **AND** plane has ***2n*** programmable connections where ***n*** is the number of inputs
- The number of product terms for each OR gate output is programmable by the user
- Product terms can be shared by PLA outputs, but not by PAL outputs due to their fixed OR array

A Simple 4-Input 3-Output PLA



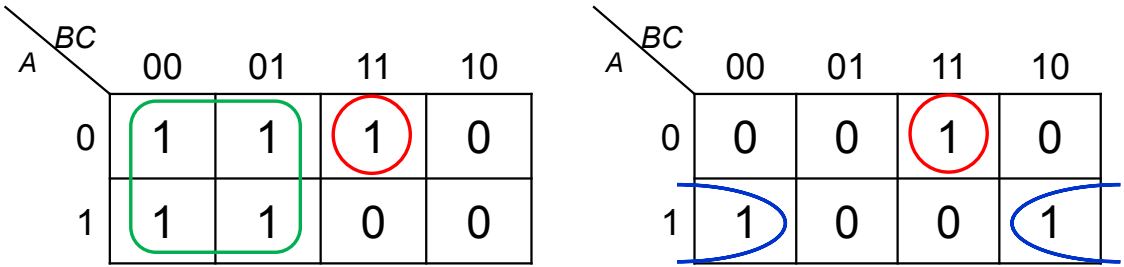
Implementing Logic Functions With PLA

- The PLA below can be used to implement any 3 logic functions, each with up to 4 variables (inputs).
- The functions implemented here are:
- f_1 = P1 + P2
  = $ab + \bar{a}\bar{b}\bar{c}\bar{d}$
- f_2 = P3 + P4 + P5
  = $a\bar{c} + \bar{a}cd + b$
- f_3 = P1 + P3 + P6
  = $ab + a\bar{c} + \bar{a}\bar{b}\bar{d}$
- Product terms shared:
  - P1 is shared by f_1 and f_3
  - P3 is shared by f_2 and f_3



A Trick for PLA implementation

- How to simplify the following two logic functions for PLA implementation:



- For PLA implementation, the important is to minimize the total number of distinct product terms of all logic functions, not to minimize the complexity of each product term, because the total number of AND gates in a PLA is fixed.

Advantages Of Using PLDs

- **Reduce PCB size**: a PLD can replace many SSI / MSI packages, hence reduce PCB size and cost.
- **Easier to assemble**: it takes less time and effort to assemble fewer components, hence reduce cost too.
- **Shorten design time**: PLD implementation is much faster than ASICs or full custom ICs.
- **Allow design changes**: reprogrammable PLDs allow the logic functions to be changed easily.
- **Improve reliability**: fewer components mean greater reliability.

What do we know by now?

- **By now, we should know:**
  - What Programmable Array Logic (**PAL**) and Programmable Logic Array (**PLA**) are
  - How to recognize their structures
  - How to use them to implement logic functions

Topic: Combinational Logic Modules

- By the end of this topic, we will know:
  - What Decoder and Encoder, Multiplexer and Demultiplexer, Comparator, Adder are
  - How to read the function table of these modules
  - How to derive the logic functions of these modules
  - How to use these modules

Combinational Logic Modules

- Modules to be discussed:
  - Decoder and Encoder
  - Multiplexer and Demulitplexer
  - Comparator
  - Adder
- Three questions are addressed for each module:
  - What are they?
  - What is the logic?
  - How to use them?

Binary Decoder

- Binary Decoder has n inputs and 2^n outputs
- Also called n-to-2^n decoder
- Inputs have all the 2^n combinations
- Only 1 of the 2^n outputs is activated for each of the corresponding input combinations
- e.g. 3-to-8 decoder
  - 3 inputs
  - 8 outputs

3-to-8 Decoder

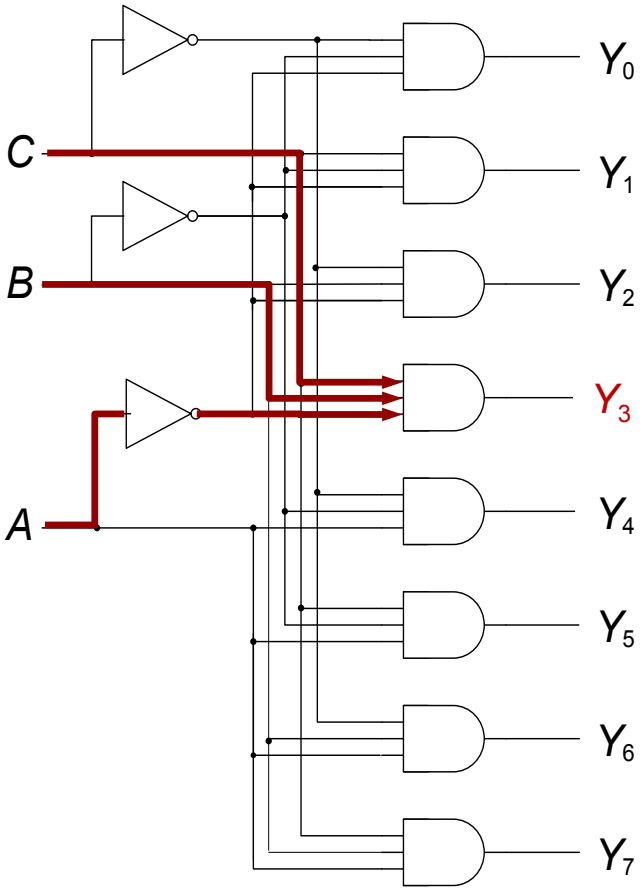
- Function Table

Inputs			Outputs							
A	B	C	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Logic Function

$Y_0 = A'B'C' = \Sigma m(0)$
$Y_1 = A'B'C = \Sigma m(1)$
$Y_2 = A'BC' = \Sigma m(2)$
$Y_3 = A'BC = \Sigma m(3)$
$Y_4 = AB'C' = \Sigma m(4)$
$Y_5 = AB'C = \Sigma m(5)$
$Y_6 = ABC' = \Sigma m(6)$
$Y_7 = ABC = \Sigma m(7)$

3-to-8 Decoder Logic Circuit



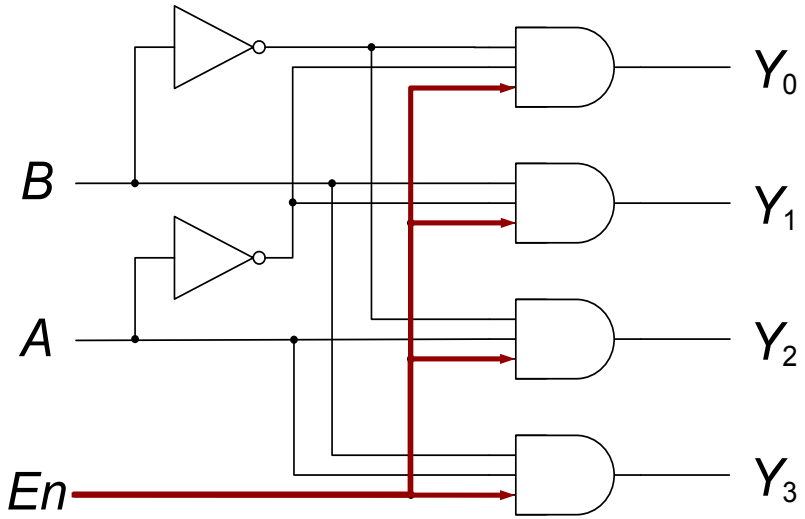
Logic Function

$$Y_0 = A'B'C'$$
$$Y_1 = A'B'C$$
$$Y_2 = A'BC'$$
$$Y_3 = A'BC$$
$$Y_4 = AB'C'$$
$$Y_5 = AB'C$$
$$Y_6 = ABC'$$
$$Y_7 = ABC$$

2-to-4 Decoder with Output Enable

- Function Table

Inputs			Outputs			
En	*A*	*B*	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



$Y_0 = EnA'B'$

$Y_1 = EnA'B$

$Y_2 = EnAB'$

$Y_3 = EnAB$

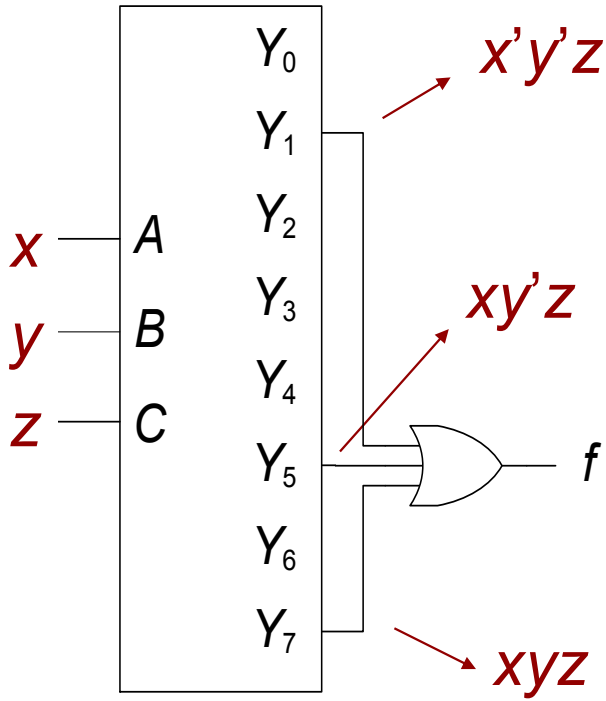
Implement Logic Function with Decoder

- Use a 3-to-8 decoder to implement:

$f = x'y'z + xy'z + xyz$

$(m_1 + m_5 + m_7)$

Inputs			Outputs							
A	B	C	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



OR the minterms

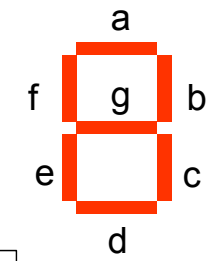
3-to-4 Decoder

- It is NOT a binary decoder
- Not all input combinations are specified
- Not just single output activated
- Functions depend on user applications, e.g.

Inputs			Outputs			
X	Y	Z	Q_3	Q_2	Q_1	Q_0
0	0	1	0	1	1	1
0	1	0	1	0	1	1
0	1	1	0	0	1	1
1	0	1	0	1	0	0
1	1	0	1	0	0	0

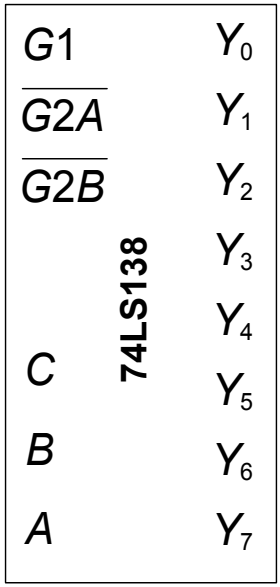
MSI Decoders

- Some MSI (Medium Scale Integration) Decoders:
  - 2-to-4, 3-to-8, 4-to-16 Decoders
  - BCD-to-Decimal Decoder
  - BCD-to-Seven-Segment Decoder



- e.g. Low Power Schottky TTL:
  74LS138 3-to-8 Decoder
  $G1$, $\overline{G2A}$ and $\overline{G2B}$
  are enable pins

Logic Symbol



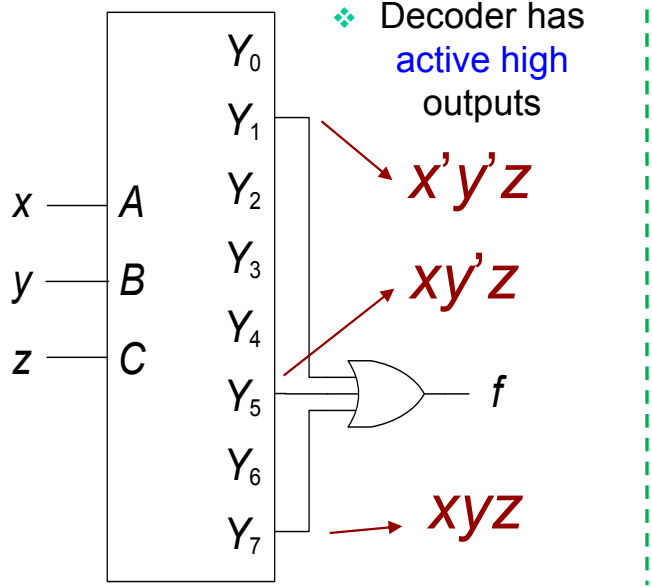
74LS138 3-to-8 Decoder

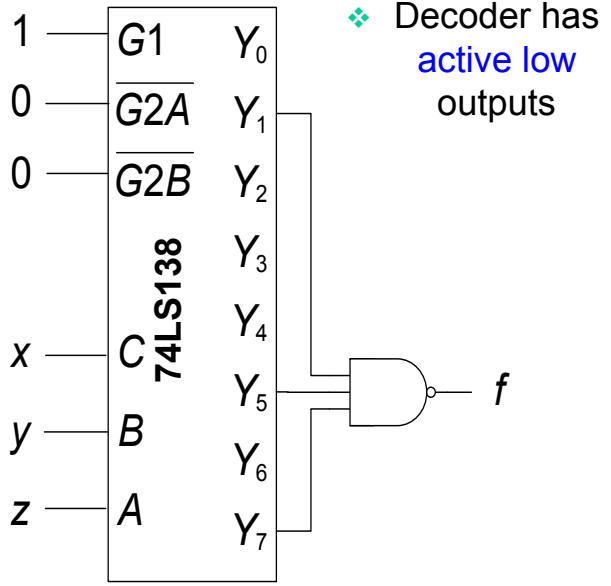
Inputs						Outputs							
$G1$	$\overline{G2A}$	$\overline{G2B}$	C	B	A	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

- Outputs are **active low**, i.e. **0** indicates the output is active.

Implement Logic Function with 74LS138

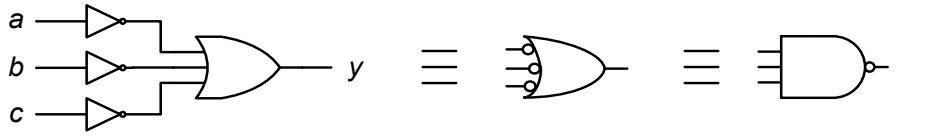
- Use a 3-to-8 decoder to implement:
  $f = x'y'z + xy'z + xyz = \Sigma m(1, 5, 7)$





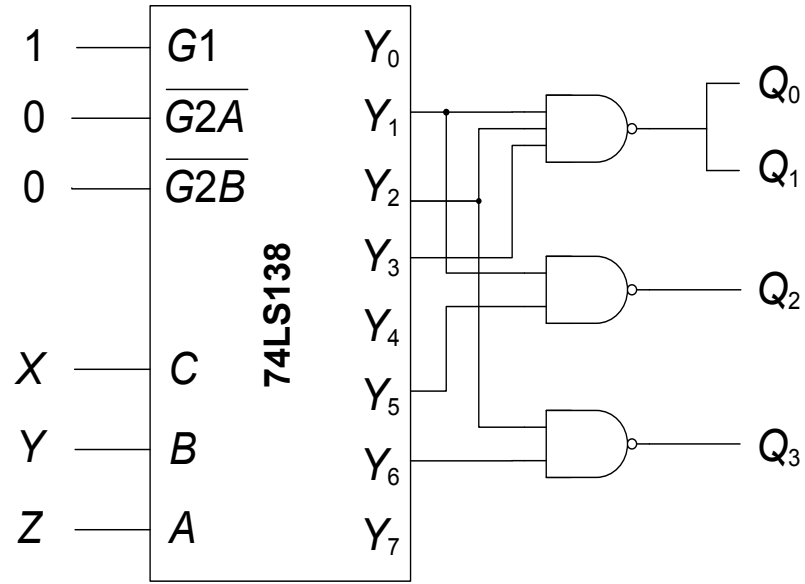
DeMorgan:

$y = \bar{a} + \bar{b} + \bar{c} = \overline{a \cdot b \cdot c}$



Implement 3-to-4 Decoder with 74LS138

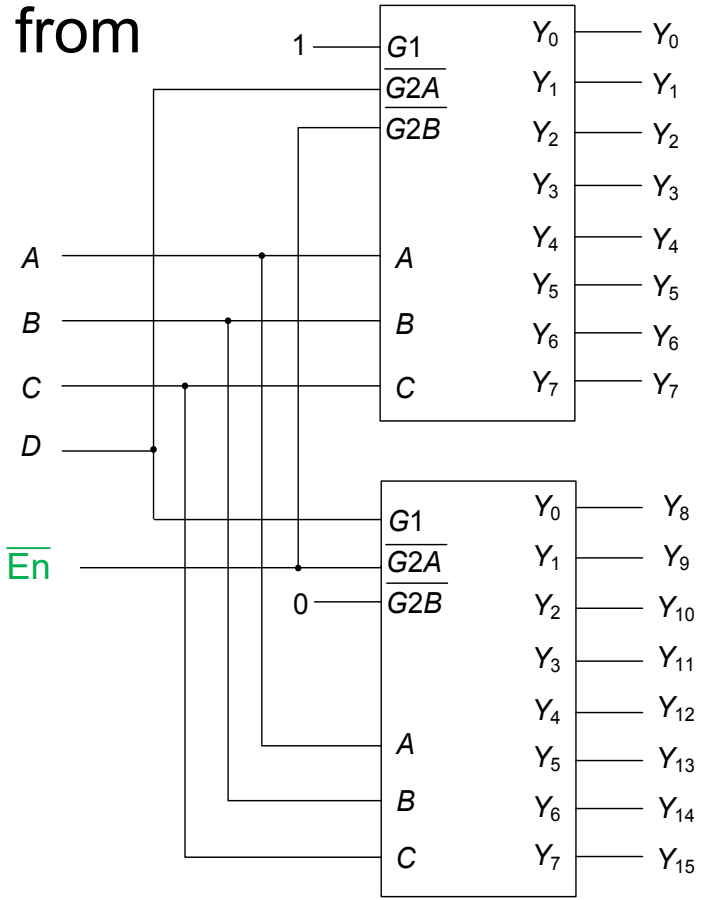
Inputs			Outputs			
X	Y	Z	Q_3	Q_2	Q_1	Q_0
0	0	1	0	1	1	1
0	1	0	1	0	1	1
0	1	1	0	0	1	1
1	0	1	0	1	0	0
1	1	0	1	0	0	0



- $Q_3 = m_2 + m_6$
- $Q_2 = m_1 + m_5$
- $Q_1 = Q_0 = m_1 + m_2 + m_3$

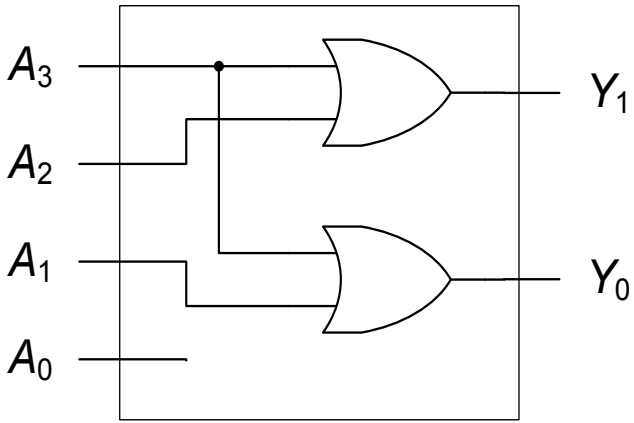
4-to-16 Decoder

- Build a 4-to-16 Decoder from two 74LS138 decoders
- Inputs: D, C, B, A
- Outputs: Y0 ~ Y15
- When D = 0, top decoder is enabled
- When D = 1, bottom decoder is enabled
- $\overline{En}$ is the overall enable
  - $\overline{En}$ = 0 : enable
  - $\overline{En}$ = 1 : disable



Binary Encoders

- Perform reverse operation of decoder
- 2^n inputs and n outputs (2^n-to-n)
- Can be built from OR gates
- e.g. 4-to-2 Encoder



Inputs				Outputs	
A_3	A_2	A_1	A_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

- Problem: when more than 1 inputs are activated.

Priority Encoder

- The binary encoder on last slide has a problem when there are more than 1 inputs activated
- Solution: Use Priority Encoder instead
- Assign priority to inputs
- e.g. 4-to-2 Priority Encoder
- A3 has the highest priority
- A0 has the lowest priority
- Encoding is according to priority

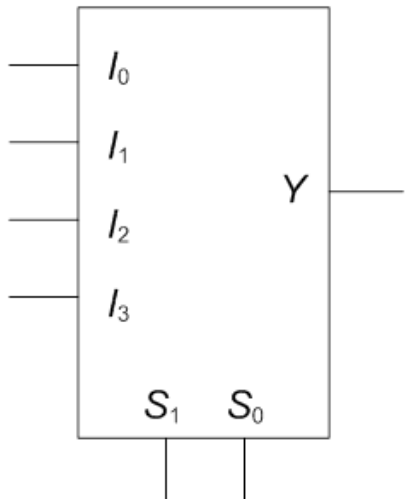
Inputs				Outputs	
A_3	A_2	A_1	A_0	Y_1	Y_0
1	x	x	x	1	1
0	1	x	x	1	0
0	0	1	x	0	1
0	0	0	1	0	0

Multiplexers (MUX)

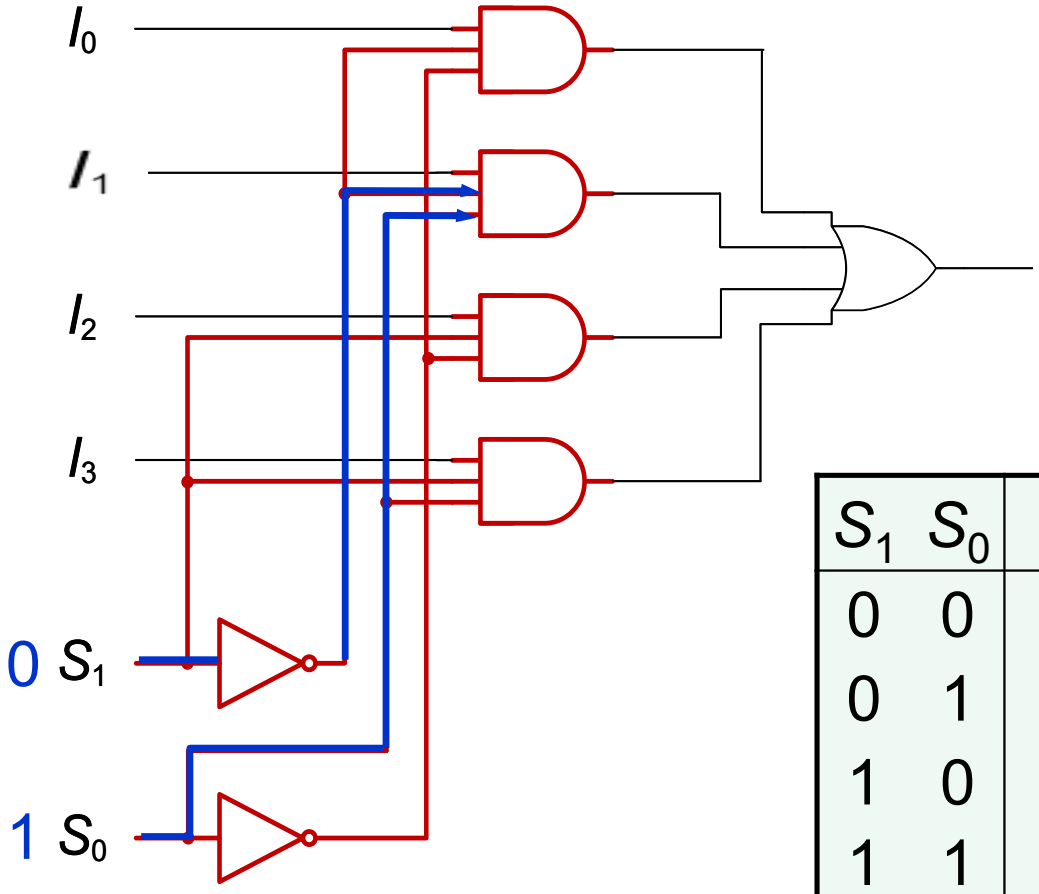
- Many input lines and 1 output line
- Select the input to go to the output
- If 2^n inputs, n select signals are needed
- Also called Data Selector
- e.g. 4-to-1 MUX

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Logic Symbol



4-to-1 MUX

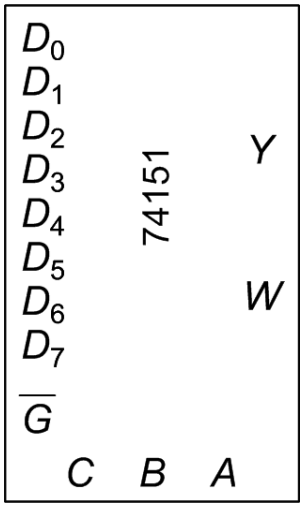


$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

MSI MUX

- 74150: 16-to-1
- 74153: Dual 4-to-1
- 74157: Quad 2-to-1
- 74151: 8-to-1



Inputs				Outputs	
Select			Strobe		
C	B	A	$\overline{G}$	Y	W
x	x	x	1	0	1
0	0	0	0	D_0	$\overline{D}_0$
0	0	1	0	D_1	$\overline{D}_1$
0	1	0	0	D_2	$\overline{D}_2$
0	1	1	0	D_3	$\overline{D}_3$
1	0	0	0	D_4	$\overline{D}_4$
1	0	1	0	D_5	$\overline{D}_5$
1	1	0	0	D_6	$\overline{D}_6$
1	1	1	0	D_7	$\overline{D}_7$

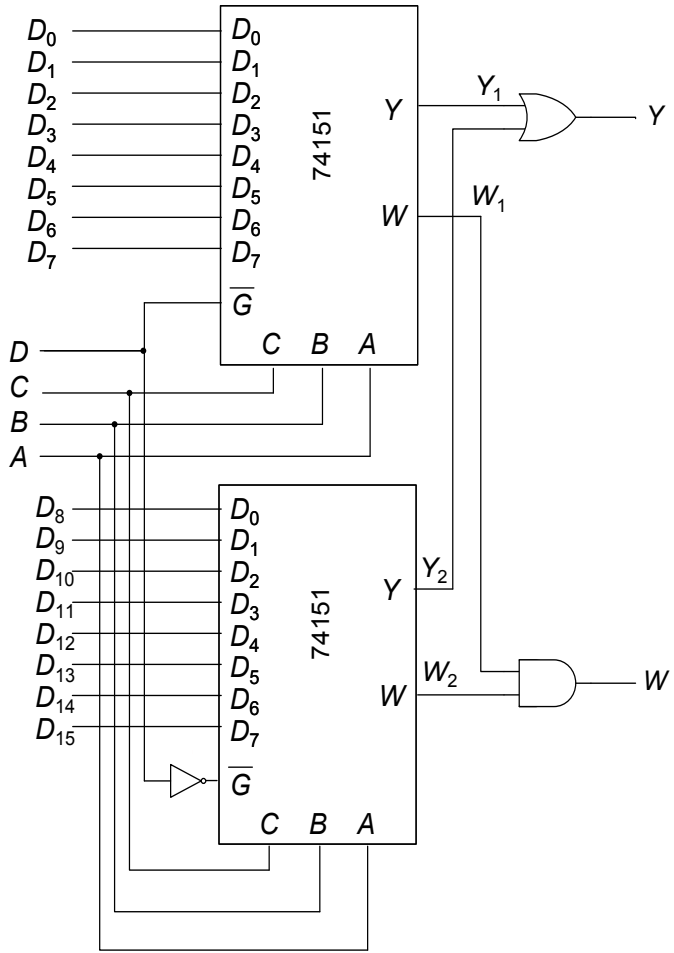
16-to-1 MUX

- Built with two 74151
- D = 0 :
  - Top MUX enabled
  - Bottom MUX disabled
- D = 1 :
  - Bottom MUX enabled
  - Top MUX disabled
- $W = \overline{Y}$

  $= \overline{Y_1 + Y_2}$

  $= \overline{\overline{W_1} + \overline{W_2}}$

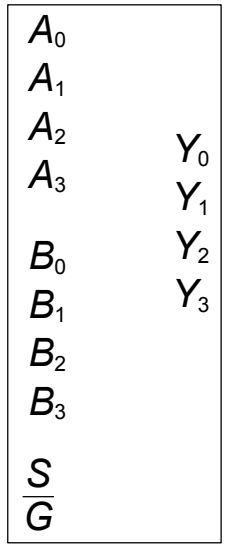
  $= W_1 \cdot W_2$



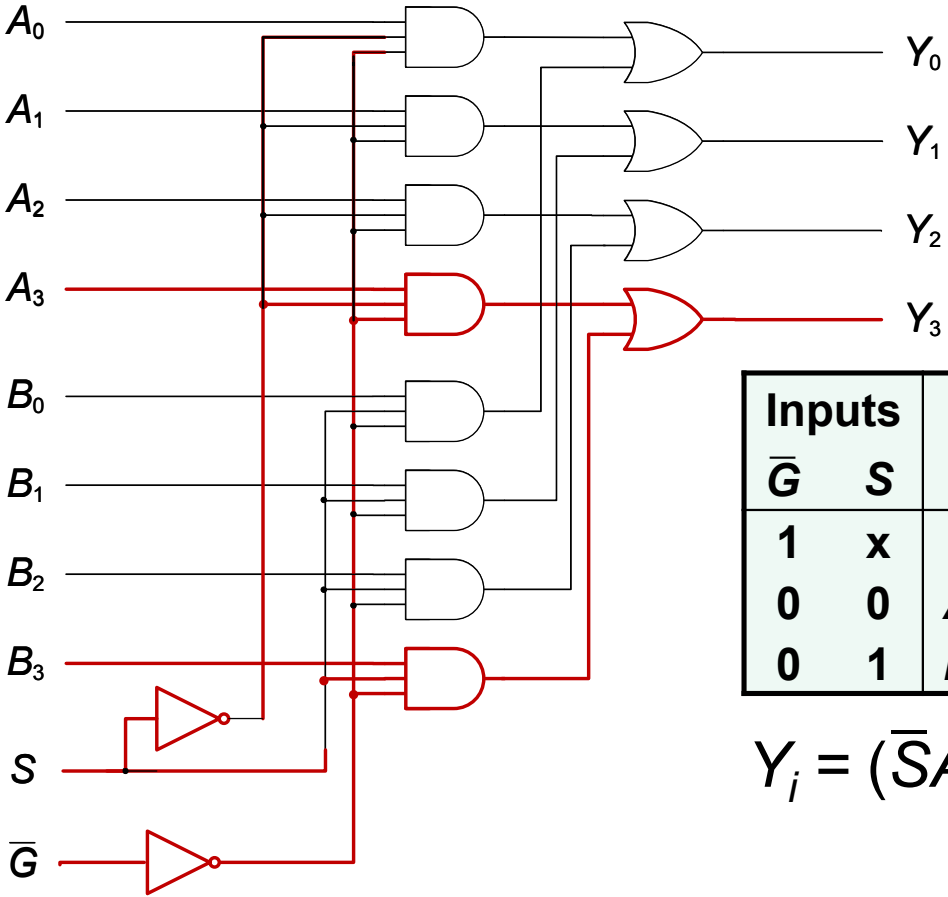
Quadruple 2-to-1 MUX

- It is 2-to-1 MUX with 4 bits for each input
- There is 1 output of 4 bits
- There is 1 single-bit select signal
- When 1 input is selected, the whole group of 4 bits goes to the output

Inputs Strobe G	Inputs Select S	Outputs Y_0	Y_1	Y_2	Y_3
1	x	0	0	0	0
0	0	A_0	A_1	A_2	A_3
0	1	B_0	B_1	B_2	B_3



Quad 2-to-1 MUX Logic Circuit



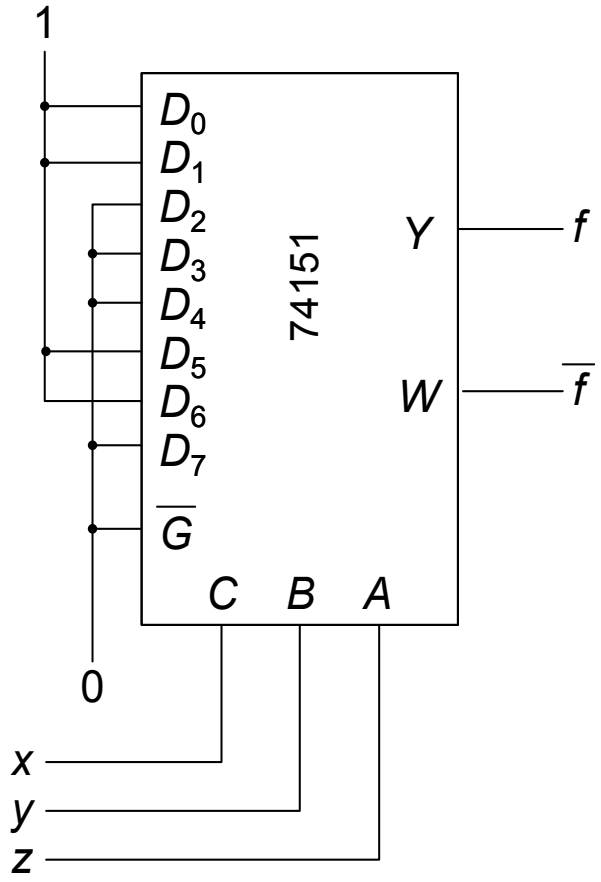
Inputs		Outputs			
$\overline{G}$	S	Y_0	Y_1	Y_2	Y_3
1	x	0	0	0	0
0	0	A_0	A_1	A_2	A_3
0	1	B_0	B_1	B_2	B_3

$$Y_i = (\overline{S}A_i + SB_i)\overline{\overline{G}}$$

MUX As Function Generator

- Logic function can be implemented with MUX
- Direct from truth table
- e.g.

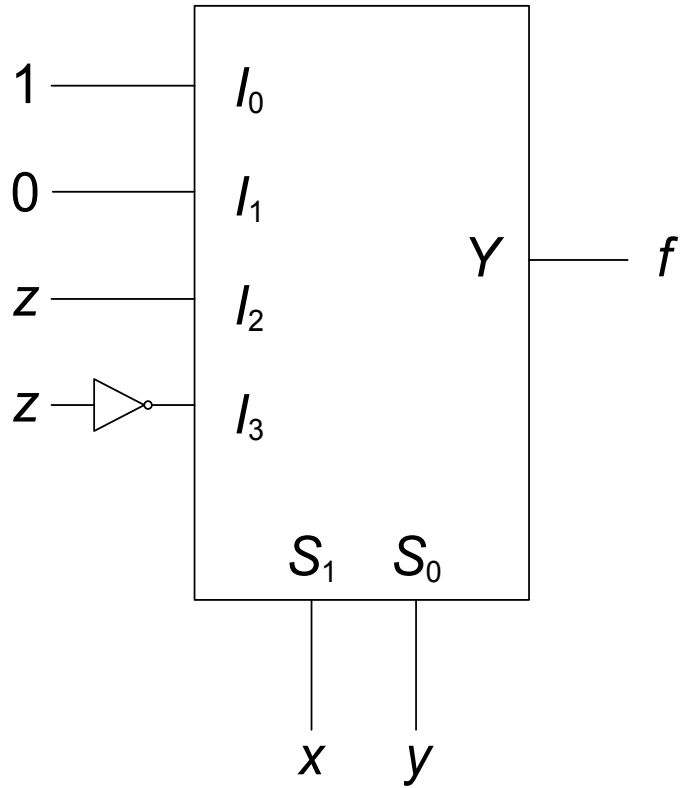
x	y	z	f
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



Implement Logic Function with MUX

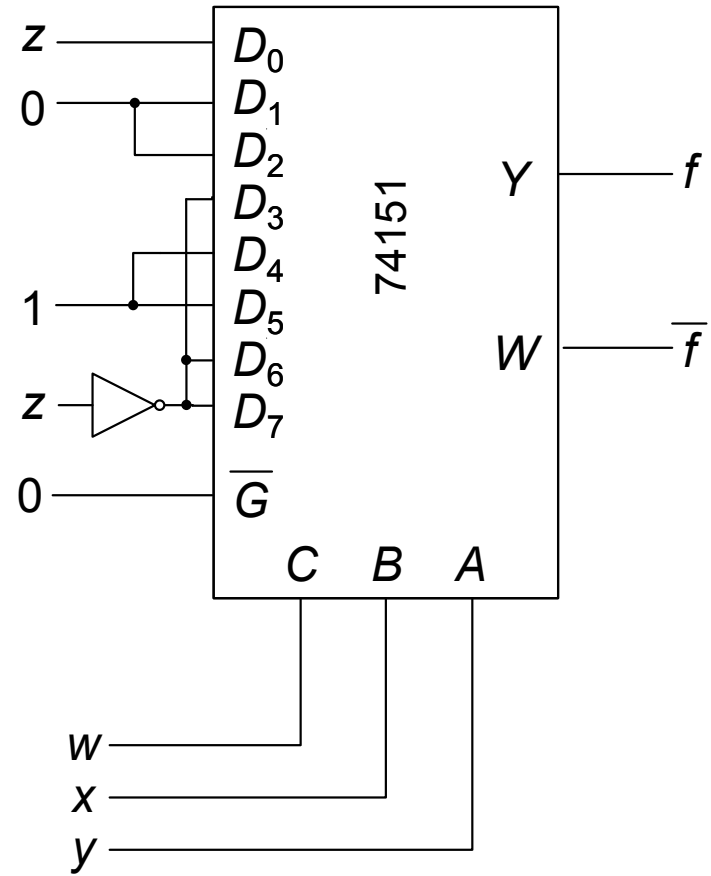
- Use 4-to-1 MUX to implement the same function:

x	y	z	f	
0	0	0	1	1
0	0	1	1	
0	1	0	0	0
0	1	1	0	
1	0	0	0	z
1	0	1	1	
1	1	0	1	$\overline{z}$
1	1	1	0	



Example for Logic Function using MUX

w	x	y	z	f	
0	0	0	0	0	z
0	0	0	1	1	
0	0	1	0	0	0
0	0	1	1	0	
0	1	0	0	0	0
0	1	0	1	0	
0	1	1	0	1	$\overline{z}$
0	1	1	1	0	
1	0	0	0	1	1
1	0	0	1	1	
1	0	1	0	1	1
1	0	1	1	1	
1	1	0	0	1	$\overline{z}$
1	1	0	1	0	
1	1	1	0	1	$\overline{z}$
1	1	1	1	0	

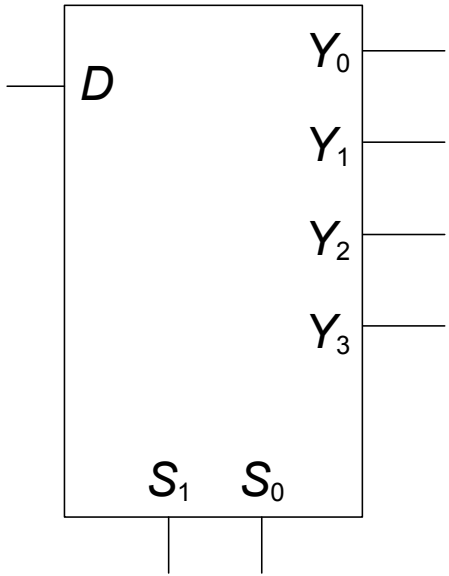


Demultiplexer (DMUX)

- It reverses the operation of MUX
- 1 data input line
- n select signals
- 2^n data output lines
- e.g. 1-to-4 DMUX

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

Logic Symbol



Use Decoders as DMUX

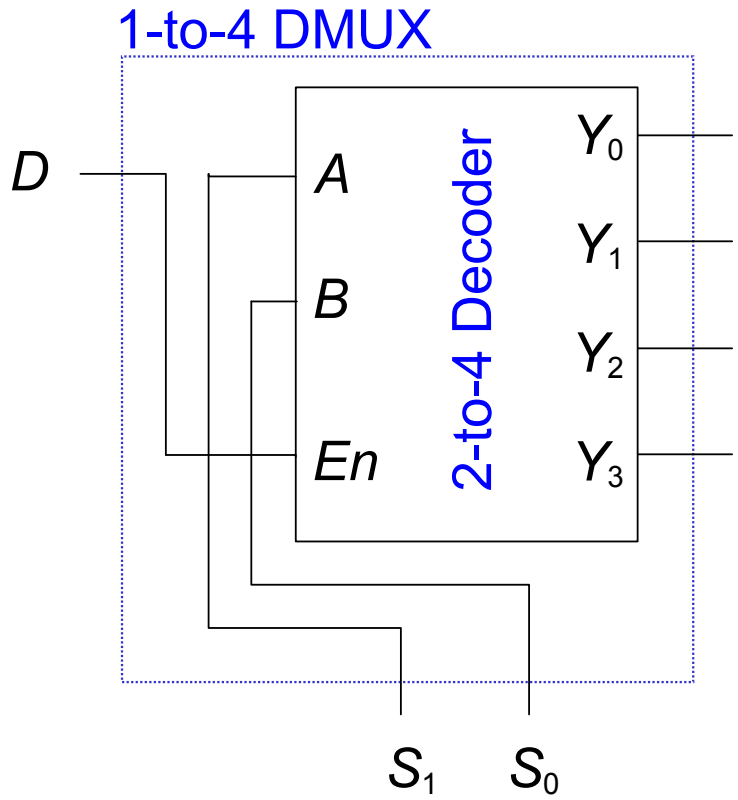
- Decoders can be used as Demultiplexers

2-to-4 Decoder

En	A	B	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

1-to-4 DMUX

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

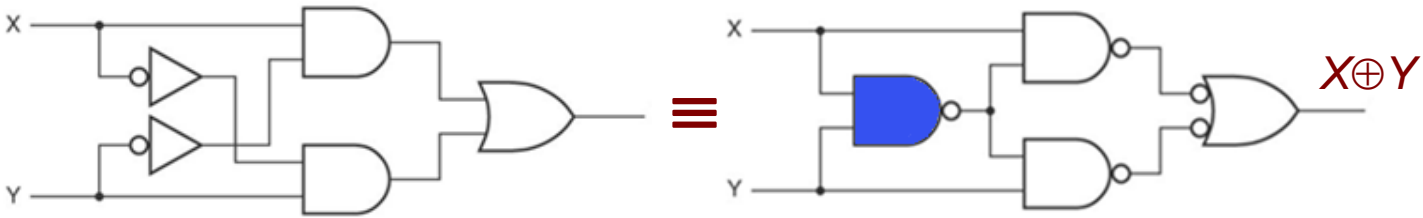


Review of XOR and XNOR Gate

- XOR produces 1 if the inputs are different
- XNOR produces 1 if the inputs are the same
- Can be expressed in terms of AND, OR and NOT

X	Y	$X \oplus Y$ (XOR)	$(X \oplus Y)'$ (XNOR)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

$$X \oplus Y = \overline{X} \cdot Y + X \cdot \overline{Y}$$
$$= Y \cdot \overline{Y} + \overline{X} \cdot Y + X \cdot \overline{Y} + X \cdot \overline{X}$$
$$= Y \cdot (\overline{Y} + \overline{X}) + X \cdot (\overline{Y} + \overline{X})$$
$$= Y \cdot \overline{(X \cdot Y)} + X \cdot \overline{(X \cdot Y)}$$
$$= \overline{Y \cdot \overline{(X \cdot Y)} \cdot X \cdot \overline{(X \cdot Y)}}$$
$$\overline{X \oplus Y} = X \cdot Y + \overline{X} \cdot \overline{Y}$$



XOR and XNOR Equivalent Gates

X	Y	X ⊕ Y (XOR)	(X⊕Y)' (XNOR)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

XOR



XNOR



Comparator

- Comparator compares binary numbers
- Logic for comparing 2 bits: a, b

a	*b*	*a=b*	*a>b*	*a<b*
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$a=b$: $\bar{a}\bar{b} + ab$ (XNOR)
$a>b$: $a\bar{b}$
$a<b$: $\bar{a}b$

- The logic for a=b is also : $\overline{\bar{a}b + a\bar{b}}$ (NOT XOR)

Magnitude Comparator

- Compares the magnitudes of binary numbers
- 4-bit Magnitude Comparator:
- Inputs are two numbers: $A_3A_2A_1A_0$ & $B_3B_2B_1B_0$
- Outputs: $Y_{A>B}$, $Y_{A<B}$, $Y_{A=B}$
- For each bit (i = 3, 2, 1, 0), let:

$$S_i = A_iB_i + \overline{A}_i\overline{B}_i = \overline{\overline{A}_iB_i + A_i\overline{B}_i}$$

- S_i is true when $A_i = B_i$
- For A = B, we must have:

$$A_3 = B_3,\ A_2 = B_2,\ A_1 = B_1, \text{ and } A_0 = B_0$$

- Hence, $Y_{A=B} = S_3 \bullet S_2 \bullet S_1 \bullet S_0$

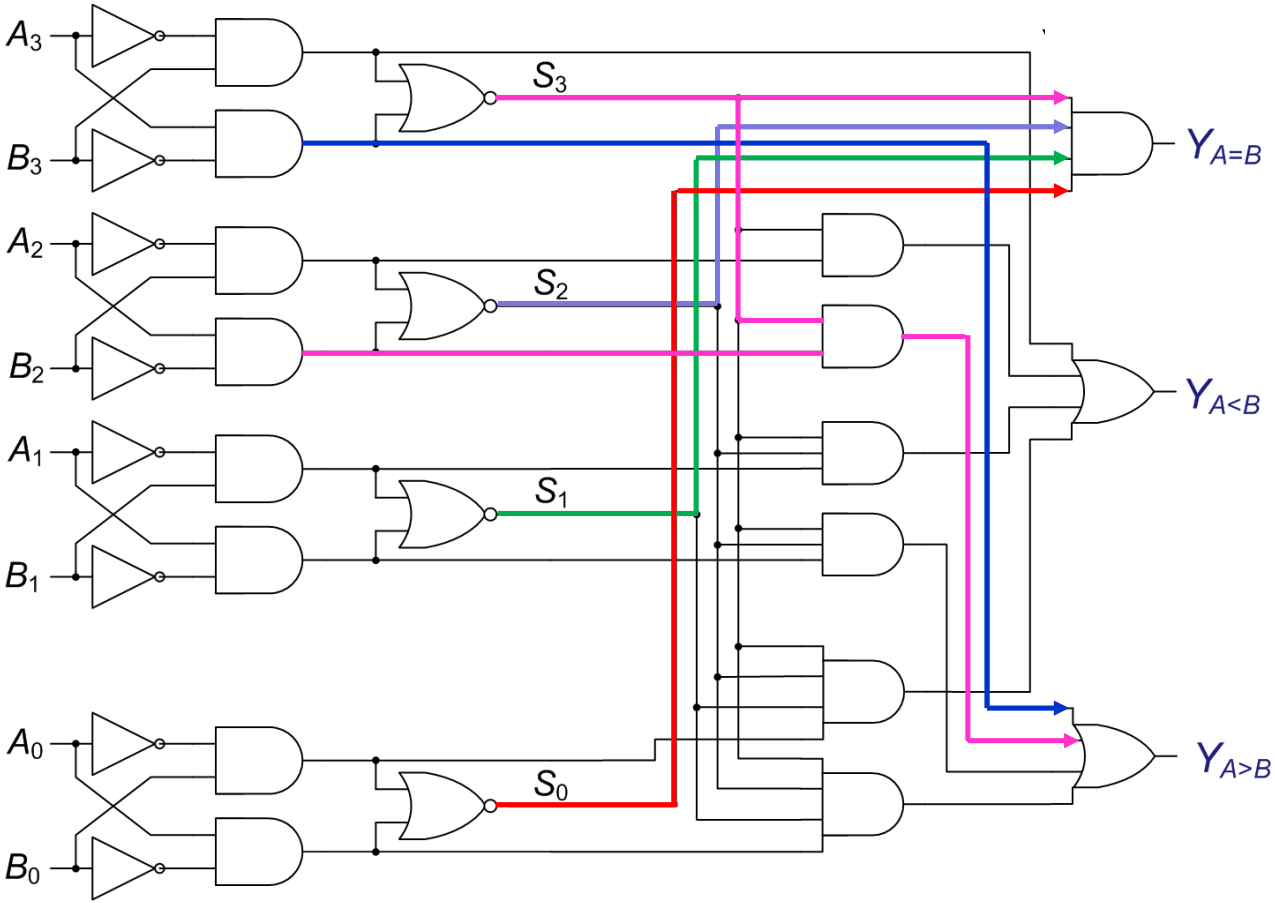
Logic for A>B

- For $A>B$, there are 4 and only 4 cases:
  1. A_3B_3 is 10 and $A_2A_1A_0$ & $B_2B_1B_0$ can be anything, i.e., A = 1xxx, B = 0xxx
  2. $A_3=B_3$ and A_2B_2 is 10 and A_1A_0 & B_1B_0 can be anything, i.e., A = 11xx, B = 10xx or A = 01xx, B = 00xx
  3. $A_3=B_3$ and $A_2=B_2$ and A_1B_1 is 10 and A_0 & B_0 can be anything, e.g., A = 011x, B = 010x
  4. $A_3=B_3$ and $A_2=B_2$ and $A_1=B_1$ and A_0B_0 is 10, e.g. A = 1011, B = 1010
- $Y_{A>B} = A_3\overline{B_3} + S_3A_2\overline{B_2} + S_3S_2A_1\overline{B_1} + S_3S_2S_1A_0\overline{B_0}$

Logic for A<B

- For $A<B$, there are 4 and only 4 cases:
  1. A_3B_3 is 01 and $A_2A_1A_0$ & $B_2B_1B_0$ can be anything, i.e., A = 0xxx, B = 1xxx
  2. $A_3=B_3$ and A_2B_2 is 01 and A_1A_0 & B_1B_0 can be anything, i.e., A = 10xx, B = 11xx or A = 00xx, B = 01xx
  3. $A_3=B_3$ and $A_2=B_2$ and A_1B_1 is 01 and A_0 & B_0 can be anything, e.g., A = 110x, B = 111x
  4. $A_3=B_3$ and $A_2=B_2$ and $A_1=B_1$ and A_0B_0 is 01, e.g. A = 1000, B = 1001
- $Y_{A<B} = \overline{A_3}B_3 + S_3\overline{A_2}B_2 + S_3S_2\overline{A_1}B_1 + S_3S_2S_1\overline{A_0}B_0$

4-bit Comparator Logic Circuit

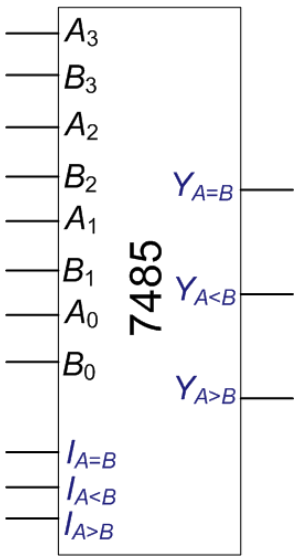


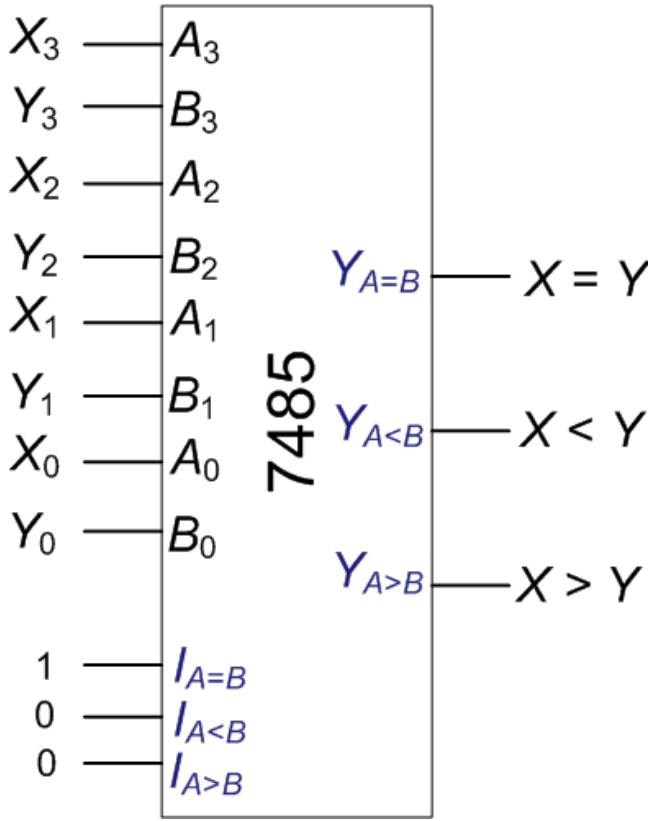
MSI: 7485 4-bit Magnitude Comparator

Input Data				Cascading Inputs			Outputs		
A_3B_3	A_2B_2	A_1B_1	A_0B_0	$I_{A>B}$	$I_{A<B}$	$I_{A=B}$	$Y_{A>B}$	$Y_{A<B}$	$Y_{A=B}$
$A_3>B_3$	X	X	X	X	X	X	H	L	L
$A_3<B_3$	X	X	X	X	X	X	L	H	L
$A_3=B_3$	$A_2>B_2$	X	X	X	X	X	H	L	L
$A_3=B_3$	$A_2<B_2$	X	X	X	X	X	L	H	L
$A_3=B_3$	$A_2=B_2$	$A_1>B_1$	X	X	X	X	H	L	L
$A_3=B_3$	$A_2=B_2$	$A_1<B_1$	X	X	X	X	L	H	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0>B_0$	X	X	X	H	L	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0<B_0$	X	X	X	L	H	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	H	L	L	H	L	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	L	H	L	L	H	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	X	X	H	L	L	H
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	L	L	L	H	H	L
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	H	H	L	L	L	L

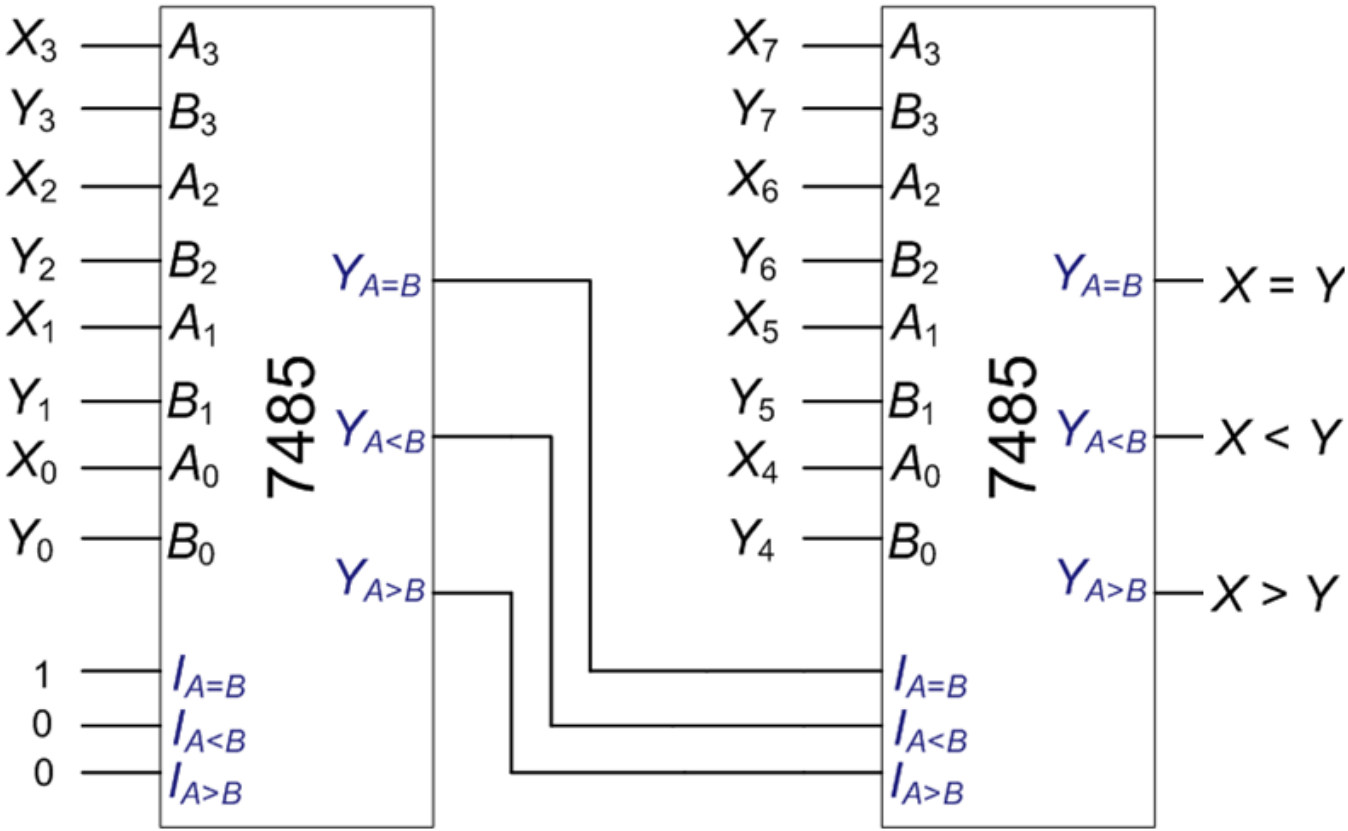
Comparison of 4-bit Numbers

- Compare two numbers:
  - $X = X_3X_2X_1X_0$
  - $Y = Y_3Y_2Y_1Y_0$
  - Logic symbol



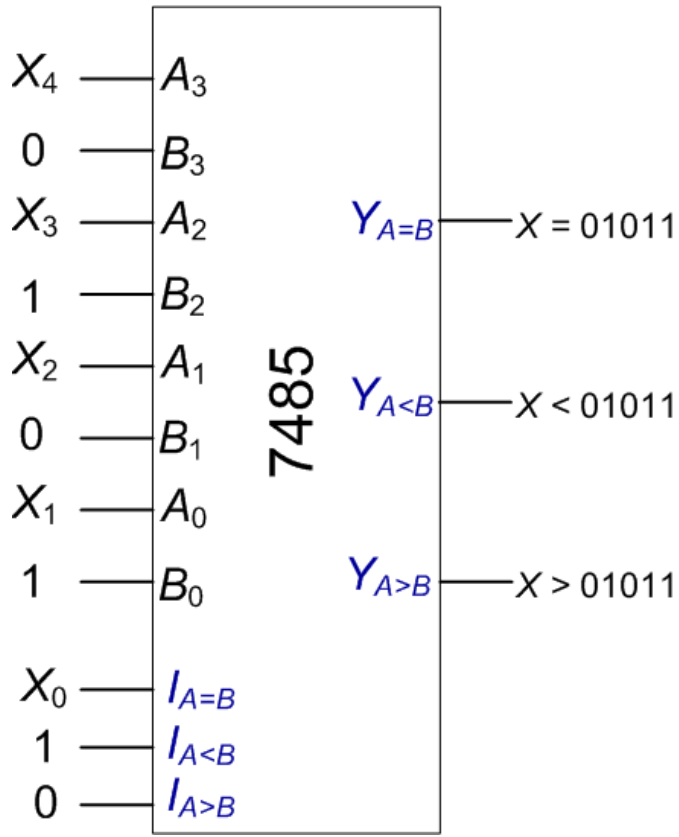


Comparison of 8-bit Numbers



Using Single 7485 for 5-bit Comparison

- Compare $X_4X_3X_2X_1X_0$ with 01011
- If $X_4X_3X_2X_1$ is greater or smaller than 0101, the cascading inputs have no effect
- If $X_4X_3X_2X_1$ = 0101, it depends on X_0
- If X_0 = 1, X = 01011
- If X_0 = 0, X = 01010 < 01011
- Exercise: How to compare $X_4X_3X_2X_1X_0$ with 01010?



Binary Adders

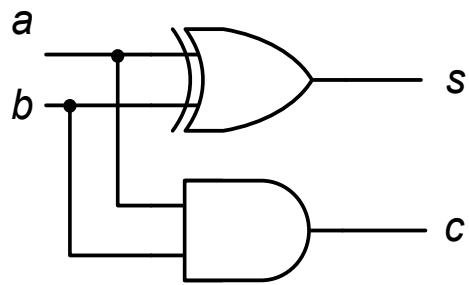
- Binary Adder is for binary number addition
- Logic Circuits to be discussed are:
  - Half Adder
  - Full Adder
  - Ripple Adder
  - Carry Look Ahead Adder

Half Adder

- Half Adder (HA) is for addition of 2 single bits
- It has two 1-bit inputs and two 1-bit outputs
- The inputs are the 2 bits to be added (a,b)
- The outputs are 1-bit sum (s) & 1-bit carry (c)
- The logic is:

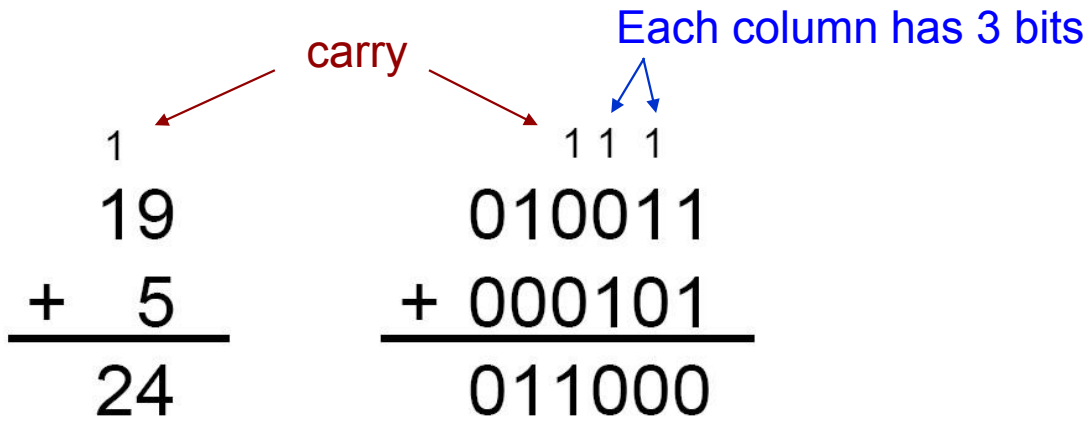
a	*b*	*s*	*c*
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$s = a \oplus b$$
$$c = a \bullet b$$



Binary Addition

- The half adder (HA) adds 2 single-bit inputs
- It cannot complete a full addition
- e.g.



- To complete a full addition, the adder needs to take in 3 inputs in each column: a, b and the carry from the previous bit.

Full Adder

- To carry the addition, an adder with 3 inputs is required
- A Full Adder (FA) takes in 3 inputs (a, b and c_i) and produce 2 outputs (s, c_o)
- a & b are the 2 bits to be added, c_i is the carry input (carry over from the previous bit) and c_o is the carry output (to the next bit)

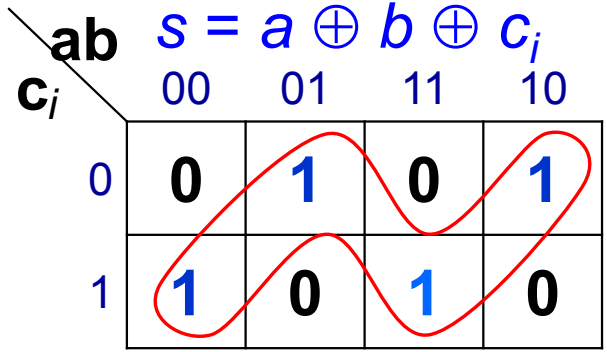
Truth Table

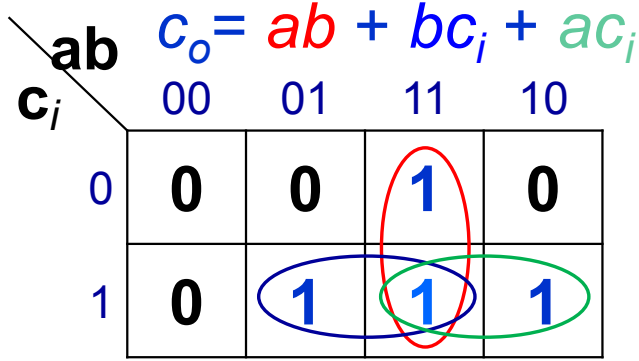
a	b	c_i	s	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logic for Full Adder

- Logic equation can be derived from the truth table:

a	b	c_i	s	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





- $s = a \oplus b \oplus c_i$
- $c_o = ab + bc_i + ac_i$

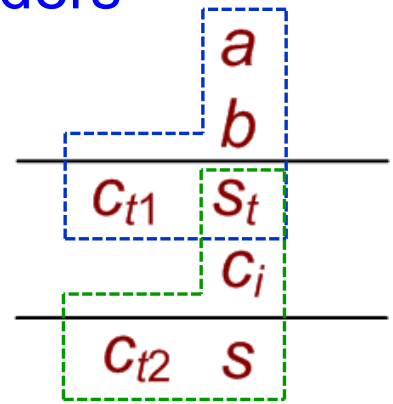
Full Adder Circuit

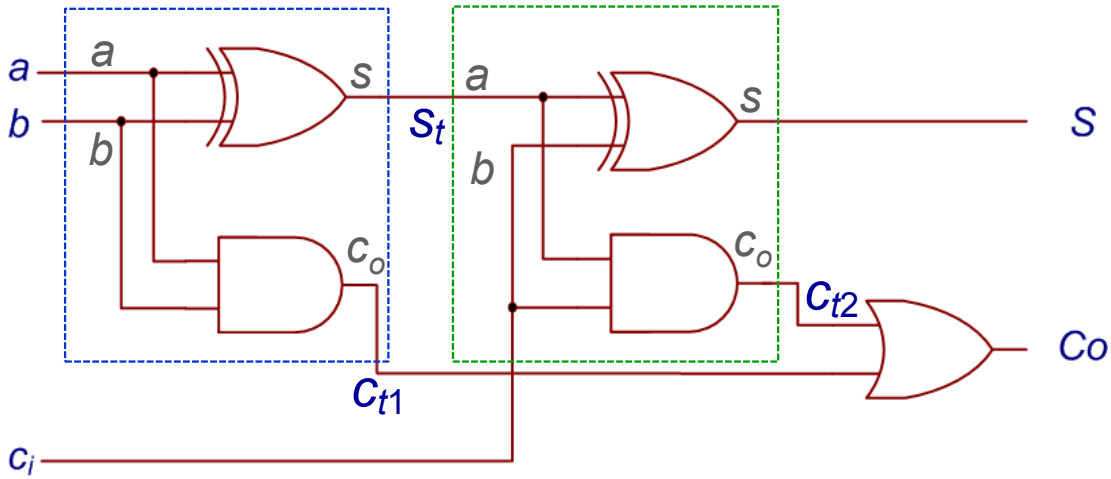
- Full adder can be built from 2 half adders
- $s = a \oplus b \oplus c_i$
- $c_o = ab + bc_i + ac_i$

$$= ab + (\bar{a}bc_i + a\underline{bc_i}) + (\underline{abc_i} + a\bar{b}c_i)$$

$$= ab + \underline{abc_i} + c_i(\bar{a}b + a\bar{b})$$

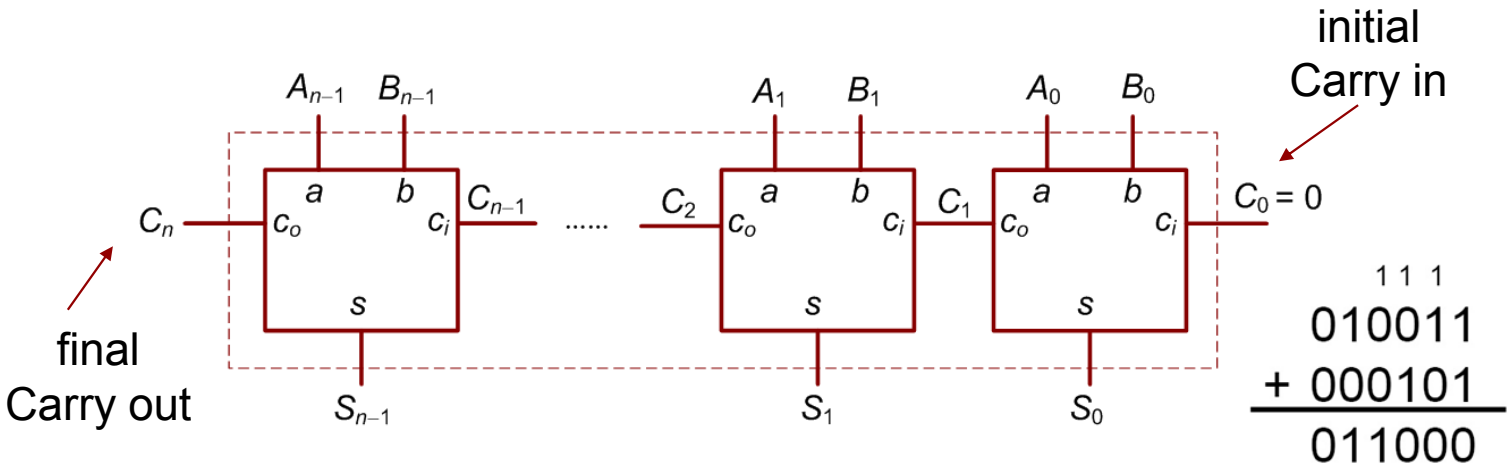
$$= ab + c_i(a \oplus b)$$





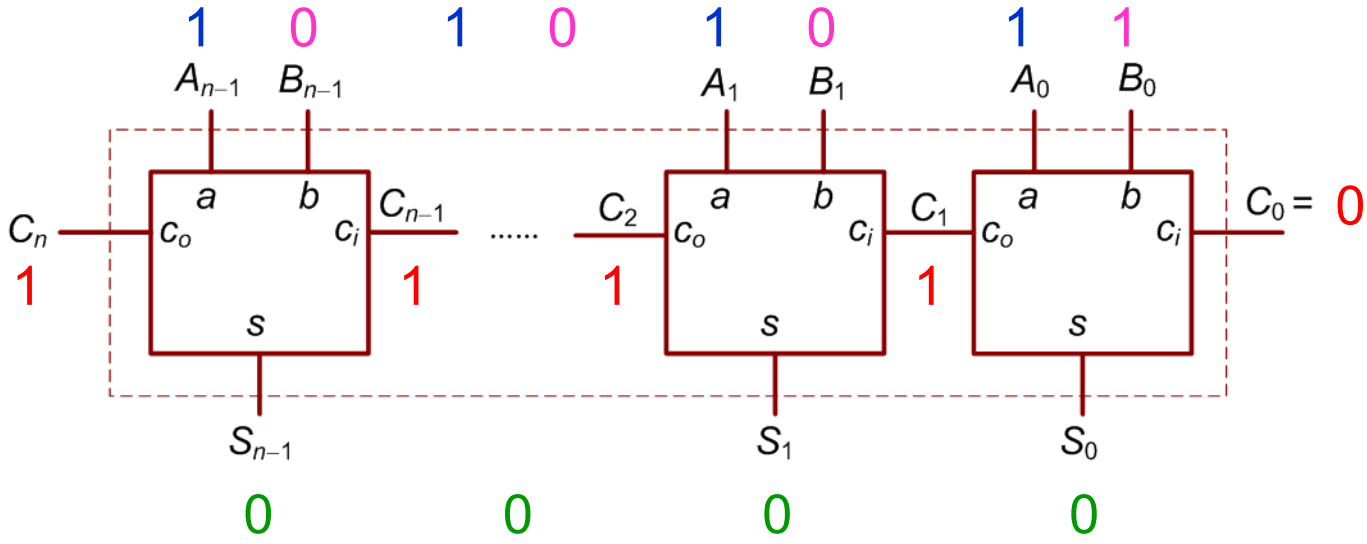
n-bit Ripple Adder

- To perform an addition of 2 *n*-bit numbers $A_{n-1}\ldots A_1A_0$ & $B_{n-1}\ldots B_1B_0$, we need a *n*-bit adder, which can be built from *n* full adders
- A_{n-1} & B_{n-1} are the MSB and A_0 & B_0 are the LSB
- Ripple Adder: Carry ripples through the chain



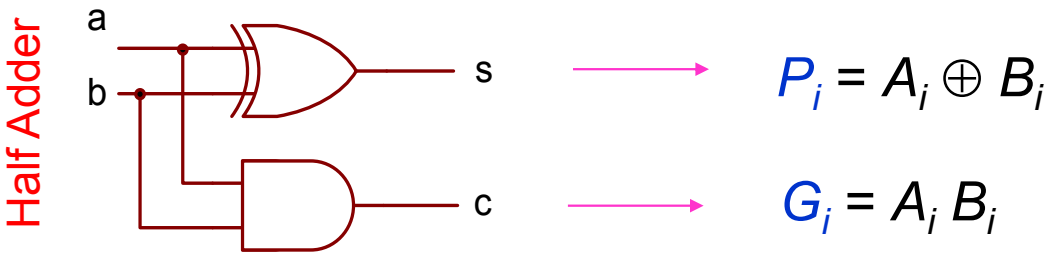
Delay in Ripple Adder

- Ripple adder is simple but slow
- Carries have to propagate through the chain
- E.g. 1111 + 0001 = (1) 0000



Carry-Look-Ahead Adder

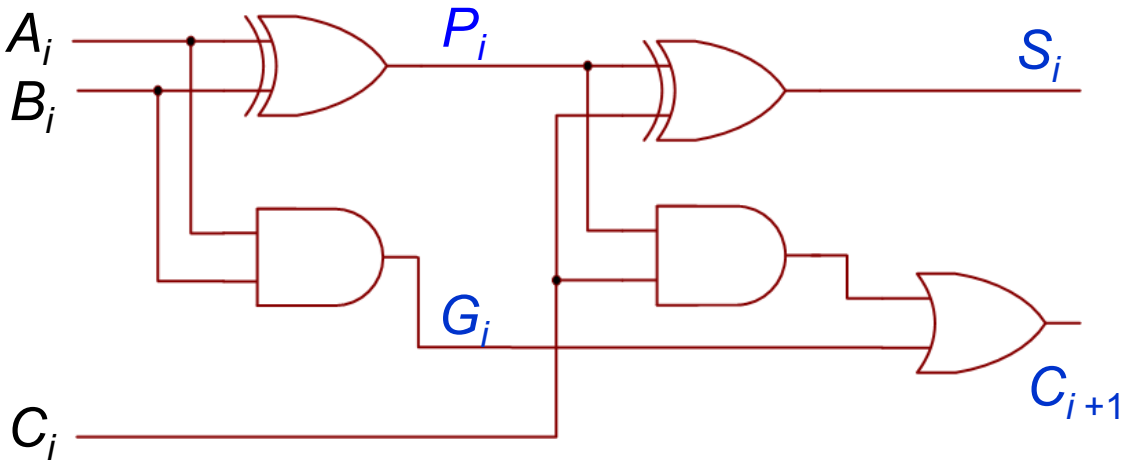
- The ripple adder is simple but slow due to the long carry chain
- To increase the speed, we have to reduce the delay on the carry chain
- Carry-Look-Ahead technique can be used
- In this technique, we redefine s & c as P_i & G_i



- P_i is called Carry Propagate
- G_i is called Carry Generate

Logic for Full Adder

- With P_i and G_i, we obtain the sum S_i & carry C_{i+1} for the full adder:



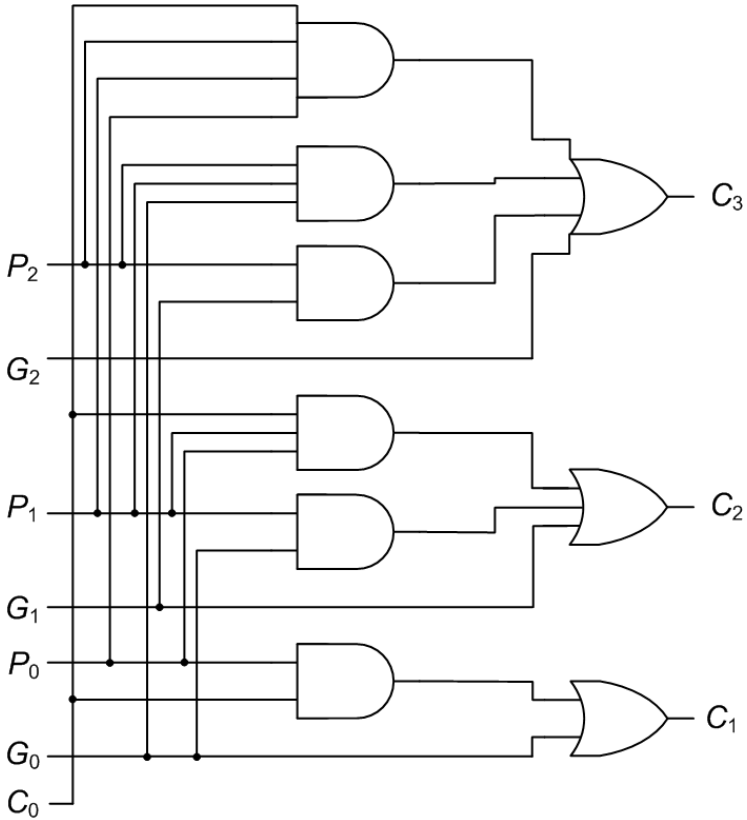
$$P_i = A_i \oplus B_i \qquad S_i = P_i \oplus C_i$$

$$G_i = A_i B_i \qquad C_{i+1} = G_i + P_i C_i$$

Carry-Look-Ahead Technique

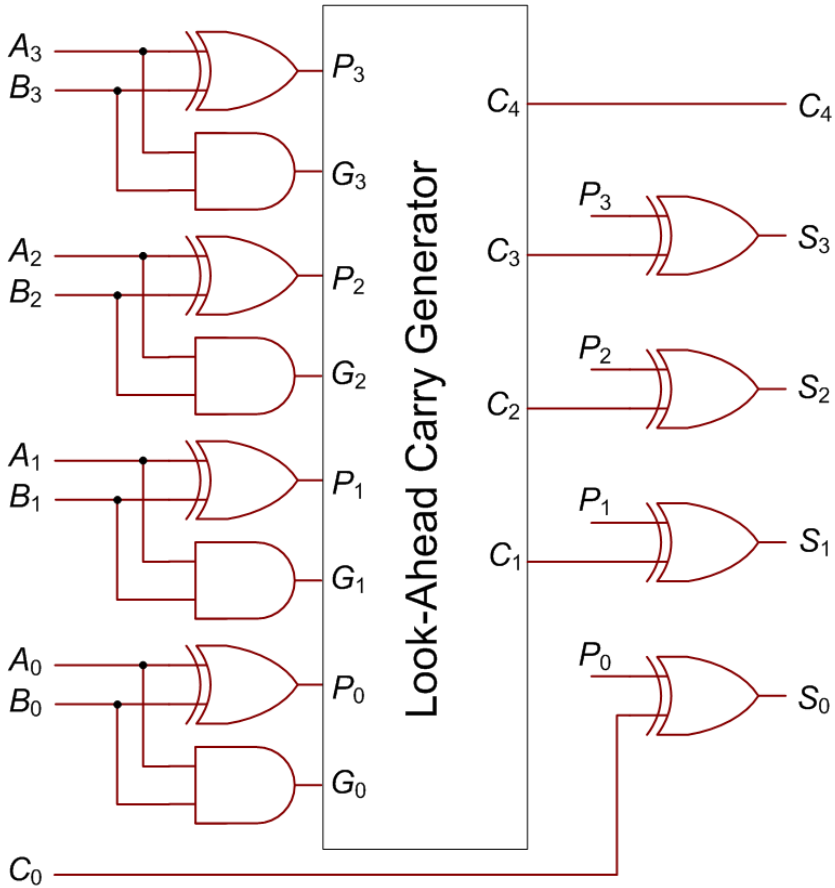
- $C_{i+1} = G_i + P_i C_i$
- $C_1 = G_0 + P_0 C_0$
- $C_2 = G_1 + P_1 C_1$
  $= G_1 + P_1(G_0 + P_0 C_0)$
  $= G_1 + P_1 G_0 + P_1 P_0 C_0$
- $C_3 = G_2 + P_2 C_2$
  $= G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0)$
  $= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$
- Carry no longer depend on its previous stage

Look-Ahead Carry Generator



- Speed:
  2 gate delay for all carry, independent of the number of bit in the adder
  $t_{Ci} = 2t_{gate}$
- Cost:
  more gates

4-bit Carry-Look-Ahead Adder



$P_i = A_i \oplus B_i$

$G_i = A_i \cdot B_i$

$S_i = P_i \oplus C_i$

$C_{i+1} = G_i + P_i C_i$

Delays:

$t_{Si} = 4t_{gate}$

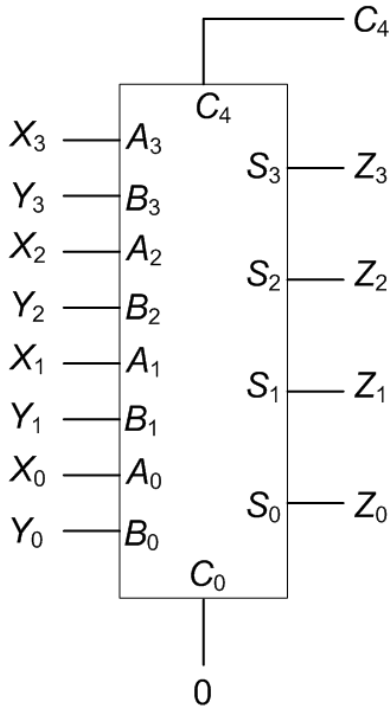
$t_{Ci} = 3t_{gate}$

MSI Adders

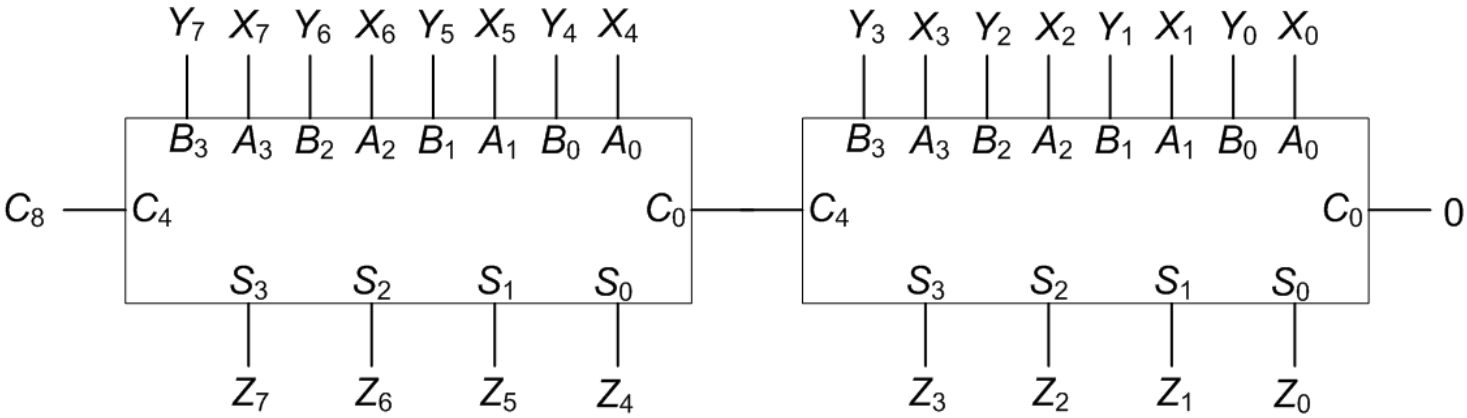
- Adders are available in MSI devices
- Both TTL and CMOS are available, e.g.
  - 74183: TTL 1-bit Full Adder
  - 7482: TTL 2-bit Carry-Look-Ahead Adder
  - 7483 and 74283: TTL 4-bit arry-Look-Ahead Adder
  - 74182: TTL 4-bit Look-Ahead Carry Generator
  - 4008: CMOS 4-bit Carry-Look-Ahead Adder

4-bit Addition

- To add 2 4-bit numbers: Z = X + Y



8-bit Addition



What do we know by now?

- Decoders and Encoders
  - What the logic of decoder and encode is
  - How to implement logic functions using decoders
- Multiplexers and Demultiplexers
  - What the logic of MUX and DMUX is
  - How to implement logic functions using MUX
- Comparators
  - What the logic of magnitude comparator is
  - How to read the function table of 7485 and how to use it
- Adders
  - What the logic of Half adder and Full adder is
  - How to build ripple adder using full adder
  - What Carry-look-ahead adder is and its logic